

50 CÂU LỆNH SQL

SẴN BUG THỰC CHIẾN

Dành cho QA · Tester — Dialect MySQL

```
ecommerce_test.sql

-- Tìm bug: email bị trùng (Bug-D)
SELECT email, COUNT(*) AS so_lan
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;

-- Kết quả phát hiện:
trung_email@email.com | 2
```

50

Câu lệnh SQL

6

Nhóm chủ đề

19

Bug cài sẵn

MySQL

Dialect

- Mỗi câu lệnh: tình huống bug điển hình — SQL — phân tích mệnh đề — kết quả — góc soi lỗi
- Phần chuẩn bị: script tạo DB, sơ đồ ER, dữ liệu mẫu với 19 bug cố ý cài sẵn để thực hành

Cách dùng cuốn cẩm nang này

Đây không phải tài liệu dạy SQL từ đầu. Nó là một bộ công cụ thực chiến: 50 câu lệnh để một người làm kiểm thử có thể tự mình đi vào database và tìm ra những lỗi mà giao diện không phơi bày. Mỗi câu lệnh được trình bày theo cùng một khuôn để bạn tra cứu nhanh:

- **Dữ liệu trước query** — bảng mẫu cho thấy trạng thái dữ liệu; dòng tô đỏ là dòng lỗi hoặc dòng trọng tâm của câu lệnh. Bảng có thể chỉ trích các dòng liên quan cho gọn.
- **Câu lệnh SQL** — sẵn sàng copy, viết theo dialect MySQL.
- **Phân tích từng mệnh đề** — mỗi từ khóa SQL được giải thích riêng.
- **Kết quả sau query** — bảng output sau khi chạy lệnh trên dữ liệu mẫu.
- **Góc soi lỗi của Tester** — cạm bẫy, hiểu lầm và bước tiếp theo.

ĐỌC TRƯỚC KHI CHẠY

Mọi câu lệnh trong sách đều là truy vấn ĐỌC (SELECT) — không sửa, không xóa dữ liệu. Dù vậy, hãy luôn chạy trên bản sao test/staging. Tên bảng/cột dùng schema ecommerce_test ở phần Chuẩn bị môi trường — đổi cho khớp schema thật của bạn.

NGUYÊN TẮC XUYỀN SUỐT

Săn bug bằng SQL thực chất là đi tìm những điều LỖ RA LUÔN ĐÚNG nhưng dữ liệu lại nói ngược lại: tồn kho không thể âm, tổng dòng con phải bằng header, email trong hệ thống phải là duy nhất. Với mỗi bất biến đó, bạn viết một câu lệnh tìm các bản ghi VI PHẠM. Cuốn sách này là 50 ví dụ của đúng một tư duy đó.

Mục lục

CHUẨN BỊ · Môi trường thực hành	5
PHƯƠNG PHÁP · Dịch yêu cầu thành câu lệnh sẵn bug	10
PHẦN 1 · Toàn vẹn và trùng lặp dữ liệu	12
01. Khám phá schema bằng INFORMATION_SCHEMA khi chưa có tài liệu	12
02. Kiểm tra ràng buộc UNIQUE/FOREIGN KEY có thực sự được enforce	14
03. Tìm email bị trùng	15
04. Tìm trùng theo nhiều cột (composite key)	17
05. Tìm NULL ở cột bắt buộc	18
06. Phát hiện khoảng trống thừa và ký tự ẩn	20
07. Kiểm tra bản ghi mồ côi (foreign key orphan)	21
08. Tìm trùng không phân biệt hoa/thường	22
09. Đếm giá trị NULL theo từng cột	24
10. Tìm bản ghi trùng hoàn toàn (full duplicate)	25
11. Kiểm tra giá trị ngoài danh sách cho phép (ENUM check)	26
12. Tìm bản ghi xóa mềm vẫn tham gia tính toán	27
PHẦN 2 · Ràng buộc nghiệp vụ	30
13. Đối soát tổng tiền đơn hàng với chi tiết items	30
14. Phát hiện tồn kho âm hoặc NULL	32
15. Phát hiện đơn hàng PENDING đang chờ sản phẩm hết hàng	33
16. Tìm đơn hàng không có dòng chi tiết nào	34
17. Tìm sản phẩm chưa bao giờ được bán	35
18. Tìm khách hàng chưa có đơn hàng nào	37
19. Tổng hợp chi tiêu theo từng khách hàng	38
20. Phát hiện sản phẩm đã bán vượt quá tồn kho	40
PHẦN 3 · Đối soát và tính toán	42
21. Tính doanh thu thực theo từng đơn từ Order_Items	42
22. Xếp hạng sản phẩm bán chạy nhất theo doanh số	43
23. Kiểm tra giá trị tồn kho (stock × price)	45
24. So sánh giá bán lịch sử với giá niêm yết hiện tại	46
25. Tổng doanh thu chỉ tính đơn hàng COMPLETED	48
26. Phân tích tỷ lệ đơn hàng theo trạng thái	49
27. Tổng doanh thu theo danh mục sản phẩm	50
28. Đối soát tồn kho hiện tại với lượng đã bán ra	52
29. Phát hiện đơn hàng bị tạo trùng (double order)	53
30. Phát hiện đơn hàng có giá trị bất thường (outlier)	54
PHẦN 4 · Biên và dữ liệu bất thường	57
31. Tìm tên khách hàng chứa ký tự bất thường	57
32. Tìm email không đúng định dạng cơ bản	58
33. Kiểm tra số lượng sản phẩm bất thường trong Order_Items	60
34. Kiểm tra ngày đặt hàng bất thường (tương lai hoặc quá xa quá khứ)	61
35. Tìm tên sản phẩm trùng sau khi chuẩn hóa	62
36. Phát hiện dữ liệu test/demo còn sót trong dữ liệu thật	63

37. Dò bản ghi gần-trùng bằng SOUNDEX (lỗi gõ chính tả)	65
38. Phát hiện đơn có tổng tiền nhưng không có sản phẩm nào	67
39. Phát hiện tổng items vượt quá 1.5 lần total_amount	68
PHẦN 5 · Audit, log và dấu vết	71
40. Truy vết item còn sót của đơn đã xóa mềm	71
41. Đối chiếu trạng thái hủy với cờ xóa mềm	72
42. Phát hiện đơn treo (PENDING) tồn đọng quá lâu	74
43. Dựng dòng thời gian đơn hàng — khoảng cách giữa các đơn	76
44. Phát hiện item_id bị nhảy — dấu vết của bản ghi bị xóa	78
45. Phân tích đơn hàng bị hủy — khách nào hay hủy nhất	80
PHẦN 6 · Truy vấn nâng cao cho QA	82
46. Dùng ROW_NUMBER() phát hiện item trùng trong cùng một đơn	82
47. Dùng CTE + RANK() xếp hạng sản phẩm bán chạy	84
48. Dùng lại một CTE nhiều lần: tìm khách chi tiêu trên mức trung bình	86
49. Tính doanh thu tích lũy theo thời gian với SUM() OVER()	88
50. Báo cáo tổng hợp: nhiều loại lỗi trong một câu UNION ALL	90
CASE STUDY · Một ngày điều tra dữ liệu của QA	93
KIỂM THỬ GHI DỮ LIỆU · Hành động trên app → xác minh DB	95
CHẠY SQL AN TOÀN TRÊN PRODUCTION	97
PHỤ LỤC · Tra cứu nhanh	98
A · Cheat sheet cú pháp lỗi	98
B · Đáp án bài tập tự luyện	100
C · Giải thích thuật ngữ	106

Chuẩn bị môi trường thực hành

Tất cả 50 câu lệnh trong sách đều chạy được ngay trên cơ sở dữ liệu **ecommerce_test** dưới đây. Hệ thống mô phỏng một sàn thương mại điện tử nhỏ gồm 4 bảng, với **19 bug được cố ý cài cắm** để bạn thực hành phát hiện.

Sơ đồ quan hệ 4 bảng (Entity Relationship)

Customers PK customer_id customer_name email membership_tier status	Products PK product_id product_name category price stock
Orders PK order_id FK customer_id total_amount status order_date deleted_at	Order_Items PK item_id FK order_id FK product_id quantity price

Customers 1 : N Orders | Orders 1 : N Order_Items | Products 1 : N Order_Items

PK = Primary Key (khóa chính) **FK** = Foreign Key (khóa ngoại, liên kết bảng khác)

Bảng	Lưu gì	Ví dụ trong sách
Customers	Danh sách khách hàng: họ tên, email, hạng thành viên (Standard / Silver / Gold), trạng thái tài khoản.	C001 = Nguyen Van A, Silver, ACTIVE.
Products	Danh mục sản phẩm: tên, danh mục, giá niêm yết, số lượng tồn kho.	PROD_001 = iPhone 15 Pro Max, giá 30.000.000, tồn 50.
Orders	Mỗi đơn hàng: ai đặt, trạng thái (PENDING / COMPLETED / CANCELLED), ngày đặt, và total_amount — tổng tiền toàn đơn. Giá trị này phải bằng SUM(price × quantity) của tất cả dòng Order_Items cùng order_id.	ORD_001 của C001, tổng 32.000.000, COMPLETED.
Order_Items	Chi tiết từng sản phẩm trong đơn — bảng nối Orders ↔ Products. Mỗi dòng = 1 sản phẩm: mua gì, số lượng, và price (giá 1 đơn vị lúc mua — lưu riêng vì giá sản phẩm có thể thay đổi sau).	ORD_001 có 2 sản phẩm → 2 dòng cùng order_id = ORD_001.

Lưu ý: **total_amount** (Orders) và **price** (Order_Items) là hai cột khác nhau. total_amount = tổng cả đơn; price = giá 1 đơn vị. Nếu hai con số này không khớp là có bug — ví dụ: ORD_002 ghi total_amount = 20.000.000 nhưng SUM(price × quantity) trong Order_Items = 31.000.000 → lệch 11.000.000 (Bug-B trong data mẫu).

Script tạo Database

Thay vì copy-paste thủ công, hãy tải file SQL sẵn có và chạy một lần duy nhất — database **ecommerce_test** cùng toàn bộ dữ liệu mẫu sẽ được tạo tự động.

Tải file: maiqai.com/books/ecommerce_test_setup.sql

Chạy trong MySQL Workbench: File → Open SQL Script → chọn file vừa tải → nhấn **Ctrl+Shift+Enter**.

Chạy bằng CLI: `mysql -u root -p < ecommerce_test_setup.sql`

Script tự xóa và tạo lại data mỗi lần chạy — tiện để reset sau khi thực hành.

PHIÊN BẢN MYSQL

Khuyến nghị **MySQL 8.0 trở lên**. Câu 1-45 chạy được trên MySQL 5.7, nhưng PHẦN 6 (Câu 46-50) dùng window function và CTE — hai tính năng chỉ có từ MySQL 8.0. Kiểm tra phiên bản đang dùng: `SELECT VERSION();`

Lỗi đã cài sẵn trong dữ liệu mẫu

Data mẫu chứa **19 lỗi cố ý** (đánh mã Bug-A → Bug-S) cài sẵn để bạn bắt bằng SQL, gom theo sáu nhóm dưới đây. Mọi câu lệnh trong sách đều trả về ít nhất một kết quả bất thường — bạn luôn có thứ để kiểm tra. Bản đồ đầy đủ từng lỗi ↔ câu phát hiện nằm ở phần chú thích đầu file **ecommerce_test_setup.sql**.

Trùng lặp & toàn vẹn dữ liệu

Email trùng tuyệt đối (C004/C005), tổ hợp khóa trùng trong Order_Items (item 1 và 7), email trùng không phân biệt hoa/thường (C001/C009), tên sản phẩm trùng hoàn toàn (PROD_002/PROD_005).
ORD_006 trùng hoàn toàn với ORD_003 (cùng khách hàng, tổng tiền, ngày đặt — đơn nhân đôi).

NULL & dữ liệu thiếu

C006: email = NULL.
C007: email = chuỗi rỗng.
PROD_006: price = NULL.
PROD_007: stock = NULL.

Định dạng không hợp lệ

C008: customer_name có khoảng trắng thừa ở đầu và cuối (' Pham Van D ').
C010: membership_tier = 'VIP' — ngoài danh sách Standard/Silver/Gold/Platinum.

Mô côi & ràng buộc tham chiếu

ORD_004: customer_id = 'C999' không tồn tại trong bảng Customers — đồng thời là đơn không có dòng item nào.

Giá trị nghiệp vụ sai & lệch số

item_id bỏ trống số 3 — gap trong chuỗi ID liên tục.

PROD_003: stock = -5 (tồn kho âm).

ORD_001: total_amount = 32.000.000 nhưng Order_Items cộng = 62.000.000 (do item trùng).

ORD_002: total_amount = 20.000.000 nhưng Order_Items cộng = 31.000.000 (lệch 11.000.000).

ORD_007: order_date = 2027-01-01 (ngày đặt hàng trong tương lai).

Item 12: quantity = 0 (số lượng biên dưới không hợp lệ).

Item 12: price = 1.500.000 nhưng PROD_002 niêm yết 2.000.000 (giá ghi sai vào đơn hàng).

Audit, xóa mềm & dấu vết

ORD_005: đã xóa mềm (deleted_at có giá trị) nhưng item 8, 9 chưa được dọn và đơn vẫn lọt vào tính toán.

ORD_003: đã CANCELLED nhưng deleted_at vẫn trống — hai cột dấu vết lệch nhau.

Dữ liệu mẫu sau khi chạy script

Bảng Customers (10 dòng — dòng đỏ: lỗi trùng email, NULL, khoảng trắng, tier sai)

customer_id	customer_name	email	membership_tier	status
C001	Nguyen Van A	a.nguyen@email.com	Silver	ACTIVE
C002	Tran Van B	b.tran@email.com	Standard	ACTIVE
C003	Le Thi C	c.le@email.com	Gold	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	Standard	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	Standard	ACTIVE
C006	Pham Van X	(NULL)	Standard	ACTIVE
C007	Nguyen Thi Y		Standard	ACTIVE
C008	Pham Van D	d.pham@email.com	Gold	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	Silver	ACTIVE
C010	Khach Test VIP	vip@email.com	VIP	ACTIVE

Bảng Products (7 dòng — dòng đỏ: stock âm, NULL, tên trùng)

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

Bảng Orders (7 dòng — dòng đỏ: total_amount sai, khách hàng mờ côi, xóa mềm, đơn nhân đôi, ngày tương lai)

order_id	customer_id	total_amount	status	order_date	deleted_at
ORD_001	C001	32.000.000	COMPLETED	2026-06-20	(NULL)
ORD_002	C002	20.000.000	COMPLETED	2026-06-22	(NULL)
ORD_003	C003	8.000.000	CANCELLED	2026-06-23	(NULL)
ORD_004	C999	5.000.000	PENDING	2026-06-24	(NULL)
ORD_005	C001	15.000.000	CANCELLED	2026-06-25	2026-06-25 10:30:00
ORD_006	C003	8.000.000	PENDING	2026-06-23	(NULL)
ORD_007	C001	20.000.000	PENDING	2027-01-01	(NULL)

Bảng Order_Items (11 dòng — dòng đỏ: tổ hợp khóa trùng; item_id=3 bỏ qua; qty=0; giá sai)

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

item_id nhảy từ 2 lên 4 (thiếu 3) và từ 9 lên 12 (thiếu 10, 11).

Item 7 trùng (order_id, product_id) với item 1.

ORD_002: tổng items = 31.000.000 nhưng total_amount = 20.000.000 (lệch 11.000.000).

Item 8 và 9 thuộc ORD_005 — đơn đã xóa mềm nhưng dòng chi tiết vẫn còn.

Item 12: quantity = 0 và price = 1.500.000 (PROD_002 niêm yết 2.000.000).

Phương pháp: Dịch một yêu cầu thành câu lệnh sẵn bug

50 câu trong sách là 50 lời giải có sẵn. Nhưng hệ thống bạn kiểm thử luôn có những quy tắc riêng mà không câu nào đoán trước được. Kỹ năng thật sự không nằm ở việc thuộc lòng câu lệnh, mà ở chỗ **biến một yêu cầu nghiệp vụ thành một truy vấn kiểm chứng**. Dưới đây là quy trình năm bước áp dụng được cho mọi nghiệp vụ.

Bước 1 — Tìm 'bất biến' (điều luôn phải đúng)

Mọi quy tắc nghiệp vụ đều ẩn một điều **LUÔN PHẢI ĐÚNG**. Viết nó ra thành một câu khẳng định, ví dụ:

- 'tổng tiền đơn = tổng tiền các dòng item'
- 'mỗi email chỉ thuộc một khách'
- 'tồn kho không bao giờ âm'

Bước 2 — Đảo thành câu hỏi vi phạm

Bug là nơi bất biến bị phá vỡ. Đổi 'X luôn đúng' thành 'tìm những bản ghi mà X SAI'. Đây là bước biến tư duy kiểm thử thành một mục tiêu truy vấn rõ ràng.

Bước 3 — Khoanh vùng bảng và cột

Bất biến đụng tới bảng và cột nào?

- Một bảng → WHERE đơn giản
- Nhiều bảng → cần JOIN hoặc truy vấn loại trừ (NOT EXISTS / NOT IN)
- So sánh hai tổng → cần GROUP BY và hàm tổng hợp

Bước 4 — Chọn khuôn mẫu SQL

Mỗi loại bất biến ứng với một mẫu quen thuộc (tra nhanh ở Phụ lục A). *Lưu ý: các mẫu dưới đây tìm chỗ vi phạm — tức là nơi quy tắc bị phá vỡ, không phải nơi quy tắc đúng.*

- 'không được trùng' → GROUP BY ... HAVING COUNT(*) > 1
- 'phải tồn tại ở bảng kia' → NOT EXISTS (quy tắc nói 'phải có', ta tìm chỗ 'không có' — nên dùng NOT EXISTS)
- 'hai tổng phải bằng nhau' → JOIN + GROUP BY + HAVING so sánh
- 'phải nằm trong danh sách' → NOT IN (quy tắc nói 'phải nằm trong', ta tìm chỗ 'nằm ngoài' — nên dùng NOT IN)

Bước 5 — Chạy và diễn giải

- Kết quả RỖNG → bất biến đang được giữ; đó là tin tốt, không phải thất bại.
- Có dòng trả về → DANH SÁCH CẦN ĐIỀU TRA, không phải bản án: luôn xác minh thủ công trước khi kết luận bug.

Ví dụ áp dụng — từ một câu yêu cầu đến câu lệnh

YÊU CẦU (trích từ tài liệu nghiệp vụ)

“Mỗi khách hàng phải đăng ký bằng một địa chỉ email duy nhất; không hai tài khoản nào được dùng chung một email.”

Bước 1 — Bất biến: email là duy nhất giữa các khách hàng.

Bước 2 — Câu hỏi vi phạm: tìm email nào đang bị từ 2 tài khoản trở lên cùng sử dụng.

Bước 3 — Bảng/cột: chỉ bảng Customers, cột email — một bảng nên không cần JOIN; so theo nhóm nên dùng GROUP BY.

Bước 4 — Khuôn mẫu: 'không được trùng' → GROUP BY email HAVING COUNT(*) > 1.

Bước 5 — Diễn giải: kết quả ra 2 email trùng — trung_email@email.com (C004 và C005 trùng hoàn toàn) và a.nguyen@email.com (C001 và C009 trùng khác hoa/thường, do collation mặc định của MySQL không phân biệt case). Cả hai cặp cần điều tra.

```
SELECT email,  
       COUNT(*) AS so_lan  
FROM   Customers  
GROUP BY email  
HAVING COUNT(*) > 1;
```

PHẦN 1 — Toàn vẹn và trùng lặp dữ liệu

Trước khi kiểm thử bất kỳ điều gì, QA cần biết hệ thống mình đang đối mặt. Hai câu mở đầu là bước thám sát schema: đọc cấu trúc bảng và kiểm tra xem các ràng buộc UNIQUE/FOREIGN KEY đã thực sự tồn tại chưa. Tiếp theo là nhóm lỗi kinh điển: bản ghi nhân đôi, ô bắt buộc bị bỏ trống, khóa ngoại trỏ vào nơi không tồn tại — những lỗi dữ liệu mà QA bắt gặp ngay tuần đầu tiên.

PHẠM VI

Câu hỏi cốt lõi: **DB có đúng cấu trúc không?** — Dữ liệu có định danh được duy nhất, liên kết hợp lệ giữa các bảng, và các ô bắt buộc không bị để trống?

01

Khám phá schema bằng INFORMATION_SCHEMA khi chưa có tài liệu

TÌNH HUỐNG

QA mới nhận một hệ thống không có tài liệu database — chỉ có quyền SELECT, không biết có bao nhiêu bảng, cột nào kiểu gì, cột nào bắt buộc. Trước khi viết được bất kỳ câu lệnh soi lỗi nào, cần một cách tra cứu schema ngay trong chính SQL, không phụ thuộc tài liệu hay sơ đồ ER có thể đã lỗi thời.

Những gì QA nhìn thấy qua **SELECT *** — không biết cấu trúc đúng sau:

customer_id	customer_name	email	membership_tier	status
C001	Nguyen Van A	a.nguyen@email.com	Silver	ACTIVE
C002	Tran Van B	b.tran@email.com	Standard	ACTIVE
C003	Le Thi C	c.le@email.com	Gold	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	Standard	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	Standard	ACTIVE
C006	Pham Van X	(NULL)	Standard	ACTIVE
C007	Nguyen Thi Y		Standard	ACTIVE
C008	Pham Van D	d.pham@email.com	Gold	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	Silver	ACTIVE
C010	Khach Test VIP	vip@email.com	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT table_name,
       COUNT(*) AS so_cot,
       SUM(CASE WHEN is_nullable='YES'
                THEN 1 ELSE 0 END) AS so_cot_nullable,
       SUM(CASE WHEN column_key='PRI'
                THEN 1 ELSE 0 END) AS so_khoa_chinh
FROM   information_schema.columns
WHERE  table_schema = 'ecommerce_test'
GROUP BY table_name
ORDER BY table_name;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM information_schema.columns	information_schema là schema hệ thống có sẵn trong mọi DB MySQL — chứa metadata mô tả chính các bảng/cột trong DB, không cần quyền đặc biệt ngoài SELECT.
WHERE table_schema = 'ecommerce_test'	Lọc đúng database đang khảo sát — information_schema chứa metadata của TẤT CẢ database trên server, không lọc sẽ trả về lẫn lộn.
SUM(CASE WHEN ... THEN 1 ELSE 0 END)	Đếm có điều kiện: đếm số cột cho phép NULL và số cột là khóa chính, cho cái nhìn nhanh về độ "chặt" của từng bảng.
GROUP BY table_name	Gom theo từng bảng để có một dòng tổng quan mỗi bảng.

Giải thích tổng thể

information_schema.columns là bản đồ schema luôn cập nhật — không như tài liệu viết tay có thể đã lỗi thời từ lâu.

DB ecommerce_test có 4 bảng (Customers, Order_Items, Orders, Products), mỗi bảng có 1 khóa chính. Orders nhiều cột nullable nhất (3/6) — gợi ý nhiều trường tùy chọn, đáng kiểm tra kỹ.

Lưu ý: **table_name có thể trả về chữ thường** dù tên gốc viết hoa (tùy hệ điều hành server MySQL chạy) — luôn kiểm tra case thật trước khi lọc WHERE table_name = '...'.
 Chưa cần hiểu hết cú pháp SUM(CASE WHEN...) ở đây — kỹ thuật đếm có điều kiện sẽ được giải thích kỹ ở Câu 9.

Kết quả sau khi query (minh họa)

table_name	so_cot	so_cot_nullable	so_khoa_chinh
customers	5	3	1
order_items	5	2	1
orders	6	3	1
products	5	3	1

4 bảng, mỗi bảng 1 khóa chính. Orders có 6 cột (nhiều nhất, vì có thêm order_date và deleted_at) với 3 cột nullable — nên soi kỹ khi tìm dữ liệu thiếu.

GÓC SOI LỖI CỦA TESTER

Khi thấy một bảng có quá nhiều cột nullable, hỏi dev: những cột đó có rule 'bắt buộc' nào ở tầng application không? Nếu có, kiểm tra tầng DB có enforce bằng NOT NULL không — nếu không, tầng application bỏ sót một lần là dữ liệu sai ghi vào ngay. Câu 5 sẽ kiểm tra cụ thể hơn từng cột.

02

Kiểm tra ràng buộc UNIQUE/FOREIGN KEY có thực sự được enforce

TÌNH HUỐNG

QA nhận bàn giao một hệ thống: tài liệu ghi "email là duy nhất", dev bảo "đã có FOREIGN KEY cho customer_id". Nhưng dữ liệu thực tế lại có email trùng và đơn hàng trở tới khách không tồn tại. Vậy những ràng buộc đó **thực sự đã được khai báo hay chưa**? Đừng dựa vào tài liệu hay lời nói miệng — tra trực tiếp metadata của DB để có câu trả lời chắc chắn.

Bảng Customers — dòng đỏ: 2 email trùng (C004/C005) lẽ ra phải bị UNIQUE chặn:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT tc.table_name,
       tc.constraint_type,
       kcu.column_name
FROM   information_schema.table_constraints tc
LEFT JOIN information_schema.key_column_usage kcu
      ON tc.constraint_name = kcu.constraint_name
      AND tc.table_schema = kcu.table_schema
      AND tc.table_name = kcu.table_name
WHERE  tc.table_schema = 'ecommerce_test'
ORDER BY tc.table_name, tc.constraint_type;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM information_schema .table_constraints tc	Bảng hệ thống liệt kê MỌI ràng buộc đã khai báo: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK.
LEFT JOIN information_schema .key_column_usage kcu	Nối thêm để biết ràng buộc đó áp dụng trên cột nào . Bắt buộc dùng LEFT JOIN thay vì JOIN thường: ràng buộc CHECK không có dòng tương ứng trong key_column_usage (CHECK không gắn với một cột theo kiểu khóa), nên INNER JOIN sẽ ÂM THÂM loại CHECK constraint khỏi kết quả — đúng loại lỗi khó nhận ra vì không báo gì cả.
WHERE tc.table_schema = 'ecommerce_test'	Giới hạn đúng database đang kiểm tra.

Giải thích tổng thể
Kết quả chỉ trả về **4 PRIMARY KEY** — đúng 1 cho mỗi bảng — và **không có UNIQUE hay FOREIGN KEY nào**.
Không có UNIQUE trên cột email có nghĩa là DB không chặn hai tài khoản dùng chung email — dữ liệu trùng (C004/C005) lọt qua được chính vì lý do này.
Không có FOREIGN KEY trên Orders.customer_id có nghĩa là DB chấp nhận bất kỳ giá trị nào cho trường đó — kể cả C999 không tồn tại trong bảng Customers.

Kết quả sau khi query (minh họa)

table_name	constraint_type	column_name
customers	PRIMARY KEY	customer_id
order_items	PRIMARY KEY	item_id
orders	PRIMARY KEY	order_id
products	PRIMARY KEY	product_id

Chỉ có khóa chính. Tính duy nhất (email không trùng) và tính tham chiếu (đơn hàng phải có khách hàng thật) chưa được bảo vệ ở tầng DB — nếu tầng ứng dụng bỏ sót một lần kiểm tra, dữ liệu sai ghi thẳng vào mà không có gì ngăn lại.

GÓC SOI LỖI CỦA TESTER

Khi phát hiện thiếu UNIQUE hay FOREIGN KEY, đừng chỉ nói 'nên thêm' — kèm theo kết quả câu này như bằng chứng để team không tranh cãi 'chắc đã có rồi'. Lưu ý: thêm UNIQUE sẽ thất bại nếu dữ liệu trùng chưa được dọn — cần xử lý dữ liệu trước, rồi mới ALTER TABLE. MySQL chỉ enforce CHECK constraint từ phiên bản 8.0.16 — hệ thống cũ khai báo nhưng DB im lặng bỏ qua.

03 Tìm email bị trùng

TÌNH HUỐNG

Hệ thống đăng ký yêu cầu mỗi email chỉ thuộc về một tài khoản. Tuy nhiên do thiếu ràng buộc **UNIQUE** trên cột email, hai bản ghi C004 và C005 đã lọt vào bảng với cùng một địa chỉ. Hậu quả: gửi email nhầm người, đăng nhập nhầm nhằng, chiến dịch marketing tính trùng người dùng.

Bảng Customers — dòng đỏ: email bị trùng (Bug-D):

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE

customer_id	customer_name	email	status
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT email,
      COUNT(*) AS so_lan
FROM   Customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL bắt đầu từ đây: tải toàn bộ bảng Customers vào vùng xử lý — 10 dòng trong dữ liệu mẫu.
GROUP BY email	Gom nhóm tất cả dòng có cùng giá trị email lại với nhau. NULL và chuỗi rỗng mỗi loại thành một nhóm riêng.
HAVING COUNT(*) > 1	Lọc nhóm : chỉ giữ nhóm có nhiều hơn 1 bản ghi — tức là email xuất hiện từ 2 lần trở lên. HAVING khác WHERE ở chỗ nó lọc SAU khi gom nhóm.
SELECT email, COUNT(*) AS so_lan	Chỉ đến bước này MySQL mới chiếu kết quả : lấy email và đặt tên cột đếm là so_lan .
ORDER BY so_lan DESC	Sắp xếp kết quả cuối cùng: email trùng nhiều nhất nổi lên đầu.

Giải thích tổng thể

Kỹ thuật dùng ở đây là **GROUP BY + HAVING COUNT**: gom nhóm theo giá trị cần kiểm tra, rồi lọc những nhóm vi phạm tính duy nhất.
Đây là mẫu truy vấn nền tảng để phát hiện mọi loại trùng lặp trong bất kỳ cột nào.

Kết quả sau khi query (minh họa)

email	so_lan
a.nguyen@email.com	2
trung_email@email.com	2

2 cặp trùng: **trung_email@email.com** (C004 + C005 — Bug-D) và **a.nguyen@email.com** (C001 + C009). Lưu ý: cặp C001/C009 chỉ xuất hiện trên MySQL 8.0 với collation mặc định (case-insensitive); trên DB phân biệt hoa/thường, cặp này sẽ không xuất hiện — chỉ còn 1 cặp. Cần xử lý cả hai trường hợp và thêm ràng buộc UNIQUE trước khi đưa lên production.

GÓC SOI LỖI CỦA TESTER

Kết quả của câu này có thể thay đổi tùy cách database **so sánh chuỗi ký tự** (database gọi cài đặt này là *collation*). Kiểm tra nhanh: `SHOW CREATE TABLE Customers;`

(1) **Vấn đề hoa/thường**: Database thường chạy theo một trong hai chế độ:

- **Không phân biệt hoa/thường** (mặc định MySQL 8.0, tên kỹ thuật `utf8mb4_0900_ai_ci`) → 'A.NGUYEN@EMAIL.COM' và 'a.nguyen@email.com' được coi là **một**, câu này phát hiện được ngay.
- **Phân biệt hoa/thường** (tên kỹ thuật `utf8mb4_bin`) → hai email trên bị coi là **khác nhau**, câu này sẽ BỎ SÓT cặp C001/C009 → cần dùng Câu 8.

(2) **Vấn đề khoảng trắng thừa**: ' test@mail.com' (có dấu cách ở đầu) và 'test@mail.com' bị coi là hai email khác nhau dù trông giống nhau → câu này bỏ sót → dùng Câu 6 để phát hiện.

04 Tìm trùng theo nhiều cột (composite key)**TÌNH HUỐNG**

Quy tắc nghiệp vụ: mỗi đơn hàng không được chứa cùng một sản phẩm ở hai dòng riêng biệt. Tổ hợp (**order_id, product_id**) phải là duy nhất trong Order_Items. Nếu app bị double-submit hoặc thiếu kiểm tra, một sản phẩm có thể lọt vào hai lần.

Bảng Order_Items — dòng đỏ: cặp (order_id, product_id) xuất hiện lần 2:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT order_id,
       product_id,
       COUNT(*) AS so_lan
FROM   Order_Items
GROUP BY order_id, product_id
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items — 11 dòng trong dữ liệu mẫu.
GROUP BY order_id, product_id	Gom nhóm theo tổ hợp hai cột. Mỗi nhóm đại diện cho một cặp (đơn hàng, sản phẩm) duy nhất.
HAVING COUNT(*) > 1	Lọc nhóm: chỉ giữ những tổ hợp xuất hiện nhiều hơn 1 lần — vi phạm tính duy nhất nghiệp vụ.
SELECT order_id, product_id, COUNT(*) AS so_lan	Chiều kết quả: tên đơn, tên sản phẩm và số lần trùng.

Giải thích tổng thể

Mẫu **GROUP BY + HAVING COUNT** áp dụng cho khóa tổ hợp.

Thay vì GROUP BY một cột như email, ta GROUP BY nhiều cột để đại diện cho quy tắc duy nhất phức tạp hơn.

Thêm bớt cột trong GROUP BY tùy theo quy tắc nghiệp vụ cần kiểm tra.

Kết quả sau khi query (minh họa)

order_id	product_id	so_lan
ORD_001	PROD_001	2

1 dòng: sản phẩm PROD_001 bị thêm hai lần vào ORD_001. Cần xóa dòng trùng và thêm UNIQUE(order_id, product_id) vào bảng.

GÓC SOI LỖI CỦA TESTER

Trước khi xóa dòng trùng, hãy kiểm tra:

- (1) Hai dòng có item_id khác nhau không? Nếu có, xóa theo item_id cụ thể — không xóa theo điều kiện mờ.
 - (2) Số lượng và đơn giá của hai dòng có giống nhau không? Nếu khác nhau, cần xác nhận nghiệp vụ dòng nào đúng trước khi xóa — không thể tự suy.
- Đừng xóa dựa trên giả định — sai item_id sẽ phá vỡ các bảng liên kết.

05 Tìm NULL ở cột bắt buộc

TÌNH HUỐNG

Cột email được quy định là bắt buộc trên giao diện đăng ký, nhưng luồng import dữ liệu cũ hoặc API nội bộ lại không kiểm tra. Kết quả: một số tài khoản không có email — không thể gửi thông báo đặt hàng, không thể reset mật khẩu, email marketing gửi hàng loạt bị thiếu người nhận.

Bảng Customers — dòng đỏ: email NULL hoặc chuỗi rỗng:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE

customer_id	customer_name	email	status
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  email IS NULL
       OR TRIM(email) = '';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE email IS NULL OR TRIM(email) = ''	Lọc hai kiểu rỗng: IS NULL bắt giá trị chưa có, TRIM(email) = '' bắt chuỗi rỗng hoặc chỉ có khoảng trắng. Thiếu một trong hai điều kiện sẽ bỏ sót một kiểu lỗi.
SELECT customer_id, customer_name, email	Chiếu ra các cột để QA xác minh và báo cáo số lượng bị ảnh hưởng.

Giải thích tổng thể

Có hai kiểu rỗng hoàn toàn khác nhau trong SQL — dễ nhầm nhưng hành xử rất khác:

- (1) **NULL** — ô chưa được điền, hệ thống *không biết* email là gì. Giống như tờ form bỏ trống hoàn toàn.
- (2) **Chuỗi rỗng ""** — ô đã được điền nhưng người dùng không gõ gì, chỉ bấm Submit. Hệ thống *biết* có email, nhưng nội dung là trống.

Tại sao không thể dùng email != "" để bắt cả hai?

Dùng **email != ""** sẽ bỏ sót mọi dòng có email là NULL — vì SQL không dùng **!=** để so sánh với NULL, bắt buộc phải dùng **IS NULL**. Đây là lý do câu lệnh cần hai điều kiện riêng kết hợp bằng OR.

Kết quả sau khi query (minh họa)

customer_id	customer_name	email
C006	Pham Van X	(NULL)
C007	Nguyen Thi Y	

Số dòng trả về = quy mô dữ liệu thiếu. Dùng con số này để viết defect report mức độ ảnh hưởng.

GÓC SOI LỖI CỦA TESTER

Đừng chỉ kiểm tra **IS NULL** — đó chỉ là một nửa bức tranh.

- (1) **Chuỗi rỗng**: hệ thống cũ thường lưu "" thay cho NULL → cần thêm điều kiện **TRIM(email) = ""**.
- (2) **Ô toàn khoảng trắng**: giá trị như " " (chỉ có dấu cách) trông giống có dữ liệu nhưng thực chất rỗng — **TRIM(email) = ""** cũng bắt được cả trường hợp này. Kết hợp cả hai điều kiện vào một câu lệnh để không bỏ sót kiểu nào.

06

Phát hiện khoảng trắng thừa và ký tự ẩn

TÌNH HUỐNG

Người dùng copy-paste tên từ Excel hoặc nhập trên điện thoại dễ kéo theo dấu cách thừa ở đầu/cuối. Trong dữ liệu mẫu, C008 được lưu là 'Pham Van D ' (có hai dấu cách ở đầu và cuối) — nhìn bề ngoài trông bình thường nhưng hệ thống coi đây là tên khác với 'Pham Van D', khiến kiểm tra UNIQUE thông thường không phát hiện được trùng lặp.

Bảng Customers — dòng đỏ: tên có khoảng trắng thừa:

customer_id	customer_name	status
C001	Nguyen Van A	ACTIVE
C002	Tran Van B	ACTIVE
C003	Le Thi C	ACTIVE
C004	Khach Hang Ao Bug	ACTIVE
C005	Khach Hang Trung	ACTIVE
C006	Pham Van X	ACTIVE
C007	Nguyen Thi Y	ACTIVE
C008	Pham Van D	ACTIVE
C009	Nguyen Van A (2)	ACTIVE
C010	Khach Test VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       CHAR_LENGTH(customer_name)      AS do_dai_goc,
       CHAR_LENGTH(TRIM(customer_name)) AS do_dai_sau_trim
FROM   Customers
WHERE  customer_name != TRIM(customer_name);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE customer_name != TRIM(customer_name)	Lọc dòng mà độ dài gốc khác độ dài sau khi cắt trắng — tức là tồn tại khoảng trắng thừa ở đầu hoặc cuối.
SELECT ..., CHAR_LENGTH(customer_name), CHAR_LENGTH(TRIM(...))	Hiển thị độ dài gốc và sau trim để thấy chênh lệch bao nhiêu ký tự .

Giải thích tổng thể

TRIM chỉ cắt khoảng trắng ở hai đầu, không xóa khoảng trắng ở giữa tên.

So sánh CHAR_LENGTH trước và sau TRIM để đo chính xác có bao nhiêu ký tự thừa.

Kết quả sau khi query (minh họa)

customer_id	customer_name	do_dai_goc	do_dai_sau_trim
C008	Pham Van D	14	10

Mỗi dòng trả về là một tên cần được chuẩn hóa — đề xuất dev chạy `UPDATE ... SET customer_name = TRIM(customer_name)` trên các dòng này.

GÓC SOI LỖI CỦA TESTER

TRIM chỉ bắt được dấu cách thông thường — loại dấu cách bàn phím tạo ra khi nhấn Space.

Bẫy từ Excel/Word: khi copy-paste từ các phần mềm văn phòng, đôi khi xuất hiện **dấu cách ẩn** (trông y hệt dấu cách bình thường nhưng là một ký tự khác, tên kỹ thuật là *non-breaking space*). TRIM không nhận ra loại này và bỏ sót hoàn toàn.

Cách nhận biết: chạy TRIM xong mà **CHAR_LENGTH** vẫn dài hơn số ký tự nhìn thấy → có khả năng cao đang chứa dấu cách ẩn.

Cách xử lý: thay thế dấu cách ẩn trước, rồi mới TRIM:

```
TRIM(REPLACE(customer_name, CHAR(160), ''))
```

Trong đó CHAR(160) là mã số của dấu cách ẩn đó.

07 Kiểm tra bản ghi mồ côi (foreign key orphan)

TÌNH HUỐNG

Bảng Orders có cột `customer_id` dùng để liên kết sang bảng Customers — mỗi đơn hàng phải thuộc về một khách hàng có thật. Nếu ràng buộc này bị tắt hoặc dữ liệu được import thủ công bỏ qua kiểm tra, có thể xuất hiện đơn hàng trỏ đến `customer_id` không tồn tại trong Customers. Đây gọi là **bản ghi mồ côi** — bản ghi tham chiếu đến một đối tượng không còn (hoặc chưa từng) tồn tại. Báo cáo doanh thu sẽ sai vì không lấy được thông tin khách hàng để ghép vào.

Bảng Orders — dòng đỏ: `customer_id` không có trong Customers:

order_id	customer_id	total_amount	status
ORD_001	C001	32.000.000	COMPLETED
ORD_002	C002	20.000.000	COMPLETED
ORD_003	C003	8.000.000	CANCELLED
ORD_004	C999	5.000.000	PENDING
ORD_005	C001	15.000.000	CANCELLED

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id
FROM   Orders o
LEFT JOIN Customers c ON o.customer_id = c.customer_id
WHERE  c.customer_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o LEFT JOIN Customers c ON o.customer_id = c.customer_id	LEFT JOIN giữ lại TẤT CẢ dòng trong Orders — kể cả dòng không khớp với bất kỳ dòng nào trong Customers. Với INNER JOIN, đơn hàng không có khách hàng tương ứng sẽ bị loại khỏi kết quả và không phát hiện được.
WHERE c.customer_id IS NULL	Sau LEFT JOIN, những dòng Orders không khớp sẽ có c.customer_id = NULL. Lọc đúng các dòng đó.
SELECT o.order_id, o.customer_id	Chiếu ra order_id và customer_id để xác định đơn hàng nào bị mồ côi.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + WHERE IS NULL** là cách chuẩn để tìm bản ghi mồ côi.

Nguyên lý: LEFT JOIN giữ nguyên tất cả dòng bên trái. Những dòng không có cặp bên phải sẽ có giá trị NULL ở mọi cột của bảng phải.

Ta lọc đúng điều kiện đó để xác định bản ghi mồ côi.

Kết quả sau khi query (minh họa)

order_id	customer_id
ORD_004	C999

ORD_004 không có chủ. Cần xác minh: customer C999 có thực tồn tại nhưng bị xóa nhầm, hay đây là đơn hàng được tạo bởi bug?

GÓC SOI LỖI CỦA TESTER

Khi tìm thấy đơn mồ côi, hỏi dev hai câu: customer C999 có thực từng tồn tại rồi bị xóa nhầm không, hay đơn được tạo do bug? Nếu customer bị xóa cứng (hard delete) mà vẫn còn đơn, đó là lỗi hỏng toàn vẹn tham chiếu (referential integrity — dữ liệu con trỏ tới bản ghi cha không còn tồn tại) — cần xem xét thêm FOREIGN KEY hoặc đổi sang soft-delete. Câu lệnh tương tự áp dụng được cho mọi cặp bảng cha-con: đổi cặp bảng và cột khóa là xong.

08 Tìm trùng không phân biệt hoa/thường

TÌNH HUỐNG

Database lưu email theo đúng kiểu người dùng nhập. Trong dữ liệu mẫu, C001 đăng ký 'a.nguyen@email.com' nhưng C009 lại nhập 'A.NGUYEN@EMAIL.COM' — cùng một địa chỉ, chỉ khác hoa/thường. Trên server dùng collation phân biệt hoa/thường, kiểm tra UNIQUE trên cột gốc sẽ bỏ qua dạng trùng này vì coi hai chuỗi trên là khác nhau.

Bảng Customers — dòng đỏ: email trùng khi so sánh không phân biệt hoa/thường:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE

customer_id	customer_name	email	status
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT LOWER(email) AS email_chuan,
       COUNT(*) AS so_lan
FROM Customers
GROUP BY LOWER(email)
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
GROUP BY LOWER(email)	LOWER(email) chuyển về chữ thường trước khi gom nhóm. Nhờ vậy 'A.NGUYEN@EMAIL.COM' và 'a.nguyen@email.com' rơi vào cùng một nhóm.
HAVING COUNT(*) > 1	Chỉ giữ nhóm có nhiều hơn 1 bản ghi — email trùng sau khi chuẩn hóa.
SELECT LOWER(email) AS email_chuan, COUNT(*) AS so_lan	Chiếu email đã chuẩn hóa và số lần trùng.
ORDER BY so_lan DESC	Email trùng nhiều nhất hiện lên đầu.

Giải thích tổng thể

Bọc cột trong hàm **LOWER()** trước GROUP BY là kỹ thuật chuẩn hóa trước khi gom nhóm.

Áp dụng tương tự với **UPPER()**, **TRIM()**, hay kết hợp cả hai tùy theo loại dữ liệu cần kiểm tra.

Kết quả sau khi query (minh họa)

email_chuan	so_lan
a.nguyen@email.com	2
trung_email@email.com	2

2 cặp trùng sau khi chuẩn hóa hoa/thường: **a.nguyen@email.com** (C001 + C009) và **trung_email@email.com** (C004 + C005).

GÓC SOI LỖI CỦA TESTER

Câu 3 và Câu 8 cho cùng kết quả trên MySQL 8.0 mặc định (collation không phân biệt hoa/thường). Điểm khác biệt: Câu 8 dùng **LOWER()** nên chạy đúng trên mọi loại collation — kể cả khi DB được cài chế độ phân biệt hoa/thường. Khi không chắc DB đang dùng collation nào, dùng Câu 8 là an toàn hơn.

09

Đếm giá trị NULL theo từng cột

TÌNH HUỐNG

Thay vì kiểm tra NULL từng cột riêng lẻ, câu lệnh này cho ra một bảng tổng hợp — mỗi cột một ô — để QA thấy ngay bức tranh toàn cảnh: cột nào bị thiếu nhiều nhất, ưu tiên xử lý cột nào trước.

Bảng Products — dòng đỏ: có giá trị NULL trong cột số:

product_id	product_name	price	stock
PROD_001	iPhone 15 Pro Max	30.000.000	50
PROD_002	Bàn phím cơ Logitech	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	-5
PROD_004	Sạc du phong Anker	1.000.000	20
PROD_005	Bàn phím cơ Logitech	2.000.000	30
PROD_006	Loa Bluetooth JBL	(NULL)	10
PROD_007	Chuot gaming Razer	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT
  SUM(CASE WHEN product_name IS NULL THEN 1 ELSE 0 END) AS null_ten,
  SUM(CASE WHEN price      IS NULL THEN 1 ELSE 0 END) AS null_gia,
  SUM(CASE WHEN stock      IS NULL THEN 1 ELSE 0 END) AS null_ton
FROM Products;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
SUM(CASE WHEN ... IS NULL THEN 1 ELSE 0 END)	Với mỗi dòng, CASE WHEN trả về 1 nếu cột là NULL, 0 nếu không. SUM cộng lại — kết quả là tổng số NULL của cột đó. Áp dụng lặp lại cho từng cột cần kiểm tra.
SELECT null_ten, null_gia, null_ton	Kết quả là 1 dòng duy nhất chứa số NULL của mỗi cột — dạng báo cáo nhanh cho QA.

Giải thích tổng thể

Kỹ thuật **CASE WHEN + SUM** (đã thoáng gặp ở Câu 1, nay phân tích kỹ) là cách đếm có điều kiện: chuyển mỗi điều kiện thành số 1/0 rồi cộng lại.

Không cần GROUP BY vì toàn bộ bảng là một nhóm duy nhất — kết quả trả về chỉ 1 dòng.

Mở rộng bằng cách thêm cột vào SELECT để kiểm tra nhiều trường trong một lần chạy.

Kết quả sau khi query (minh họa)

null_ten	null_gia	null_ton
0	1	1

Cột price và stock đều có 1 NULL — cần làm rõ với dev/PO: NULL ở đây có nghĩa là gì? Chưa nhập? Miễn phí? Hết hàng? NULL và 0 là hai trạng thái khác nhau — xác định đúng nghĩa trước khi viết test case.

GÓC SOI LỖI CỦA TESTER

Câu này chỉ đếm NULL, bỏ sót chuỗi rỗng — dữ liệu thiếu ở hệ thống cũ thường lưu dưới dạng chuỗi rỗng thay vì NULL, nên kết quả = 0 chưa có nghĩa là sạch.

Kết quả là tổng hợp toàn bảng (1 dòng duy nhất) — dùng Câu 5 để xem cụ thể bản ghi nào bị thiếu.

10

Tìm bản ghi trùng hoàn toàn (full duplicate)

TÌNH HUỐNG

Khác với trùng ID, full duplicate là hai dòng có toàn bộ giá nghiệp vụ giống nhau nhưng ID tự sinh khác nhau — thường xảy ra khi người dùng bấm nút 'Lưu' hai lần hoặc import dữ liệu không kiểm tra trùng trước.

Bảng Products — dòng đỏ: cùng tên và giá với dòng khác:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_name,
       price,
       COUNT(*) AS so_lan
FROM   Products
GROUP BY product_name, price
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
GROUP BY product_name, price	Gom nhóm theo tổ hợp các cột nghiệp vụ — ở đây là tên sản phẩm và giá. Thêm category vào GROUP BY nếu muốn kiểm tra chặt hơn.
HAVING COUNT(*) > 1	Lọc nhóm có nhiều hơn 1 dòng — tức là cùng tên và giá xuất hiện lặp lại.
SELECT product_name, price, COUNT(*) AS so_lan	Chiếu tên, giá và số lần trùng để xác định bản ghi cần xóa.

Giải thích tổng thể

Định nghĩa 'trùng hoàn toàn' phụ thuộc vào nghiệp vụ: có thể trùng theo 2, 3 hoặc toàn bộ cột.

Hãy thêm bớt cột trong GROUP BY tùy theo ngữ cảnh — chỉ đưa vào những cột đại diện cho giá trị nghiệp vụ, không phải ID tự sinh.

Kết quả sau khi query (minh họa)

product_name	price	so_lan
Ban phím cơ Logitech	2.000.000	2

PROD_002 và PROD_005 trùng tên + giá nhưng có product_id khác nhau — hệ thống đang có hai bản ghi cho cùng một sản phẩm. Cần xác định bản nào có đơn hàng liên kết để tránh xóa nhầm khi dọn dẹp.

GÓC SOI LỖI CỦA TESTER

Bẫy 1 — trùng theo 2 cột chưa đủ bằng chứng: câu lệnh chỉ so sánh product_name và price. Hai sản phẩm cùng tên cùng giá nhưng khác nhà cung cấp hoặc khác phiên bản vẫn có thể là sản phẩm hợp lệ, không phải trùng thật — cần xác nhận thêm với PO trước khi báo bug.

Bẫy 2 — kết quả không nói bản nào là gốc: câu trả về số lần trùng, không biết PROD_002 hay PROD_005 mới là bản được tạo đúng. Có đơn hàng liên kết không đồng nghĩa là bản gốc — có thể người dùng đặt nhầm vào bản trùng.

Báo cáo bug đúng cách: ghi rõ cả hai product_id, số đơn hàng liên kết từng bản, và đề nghị dev/PO xác nhận bản nào giữ lại — không tự ý kết luận.

11**Kiểm tra giá trị ngoài danh sách cho phép (ENUM check)****TÌNH HUỐNG**

Cột membership_tier chỉ được nhận một trong bốn giá trị: Standard, Silver, Gold, Platinum. Nếu tầng ứng dụng không validate, hoặc dữ liệu được nhập thẳng vào DB qua script, các giá trị lạ sẽ lọt vào và làm vỡ logic ưu đãi.

Bảng Customers — dòng đỏ: membership_tier ngoài danh sách hợp lệ:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       membership_tier
FROM   Customers
WHERE  membership_tier NOT IN ('Standard','Silver','Gold','Platinum');
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE membership_tier NOT IN ('Standard','Silver', 'Gold','Platinum')	NOT IN lọc tất cả dòng có giá trị không nằm trong danh sách. Lưu ý: NULL sẽ KHÔNG khớp NOT IN — cần thêm OR membership_tier IS NULL nếu muốn bắt cả NULL.
SELECT customer_id, customer_name, membership_tier	Chiếu đủ thông tin để QA xác minh và cập nhật về giá trị đúng.

Giải thích tổng thể

Kỹ thuật **NOT IN + danh sách tường minh**: lọc ra mọi giá trị không nằm trong tập hợp lệ đã định nghĩa. Áp dụng được cho mọi cột dạng ENUM — chỉ đổi tên cột và danh sách.

Kết quả rỗng không phải thất bại — đó là xác nhận dữ liệu đang đúng. Nên chạy định kỳ hoặc sau mỗi lần migrate để phát hiện sớm giá trị lạ.

Kết quả sau khi query (minh họa)

customer_id	customer_name	membership_tier
C010	Khách Test VIP	VIP

1 bản ghi có tier không hợp lệ. Cần cập nhật về giá trị đúng và bổ sung CHECK constraint hoặc ENUM type trong DB.

GÓC SOI LỖI CỦA TESTER

Cạm bẫy: không để NULL lọt vào danh sách NOT IN — nếu có, toàn bộ điều kiện trả về UNKNOWN và bỏ sót mọi dòng. Bẫy này áp cho cả danh sách viết tay lẫn **NOT IN (subquery)** khi cột trong subquery có thể NULL — khi đó dùng NOT EXISTS an toàn hơn.

Nếu cần bắt thêm cả dòng chưa điền tier, thêm riêng: OR membership_tier IS NULL

12

Tìm bản ghi xóa mềm vẫn tham gia tính toán

TÌNH HUỐNG

Hệ thống dùng cơ chế **soft-delete** — khi hủy đơn hàng, cột **deleted_at** được ghi thời điểm xóa thay vì xóa hẳn bản ghi. Tuy nhiên nhiều query báo cáo (tổng doanh thu, số đơn trong tháng...) quên điều kiện **WHERE deleted_at IS NULL**, khiến đơn đã hủy vẫn bị tính vào kết quả. Đây là bug rất phổ biến và khó phát hiện bằng mắt thường vì dữ liệu 'trông có vẻ đúng'.

Bảng Orders — dòng đỏ: đơn ORD_005 đã bị xóa mềm (deleted_at có giá trị) nhưng vẫn nằm trong bảng:

order_id	customer_id	total_amount	status	deleted_at
ORD_001	C001	32.000.000	COMPLETED	None
ORD_002	C002	20.000.000	COMPLETED	None
ORD_003	C003	8.000.000	CANCELLED	None
ORD_004	C999	5.000.000	PENDING	None
ORD_005	C001	15.000.000	CANCELLED	2026-06-25 10:30:00

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount,
       o.status,
       o.deleted_at
FROM   Orders o
WHERE  o.deleted_at IS NOT NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o	Quét toàn bộ bảng Orders — bao gồm cả bản ghi đã bị xóa mềm.
WHERE o.deleted_at IS NOT NULL	deleted_at IS NOT NULL: lọc ra các đơn hàng đã bị đánh dấu xóa. Bản ghi vẫn tồn tại trong bảng nhưng lẽ ra không được tham gia vào bất kỳ phép tính nào.
SELECT o.order_id, o.total_amount, o.status, o.deleted_at	Chiếu đủ thông tin để QA đối chiếu: đơn nào bị xóa, số tiền bao nhiêu, trạng thái gì — từ đó kiểm tra xem query báo cáo có đang cộng nhầm những đơn này không.

Giải thích tổng thể

Cơ chế **soft-delete** giữ lại bản ghi để phục vụ **audit trail** (nhật ký truy vết — lưu lại lịch sử mọi thao tác để tra cứu khi cần), nhưng yêu cầu **mọi query nghiệp vụ** đều phải thêm điều kiện `WHERE deleted_at IS NULL`.

Bug xảy ra khi developer viết query mới mà quên điều kiện này — đơn đã xóa lặng lẽ cộng vào tổng doanh thu.

QA nên chạy câu này để biết có bao nhiêu bản ghi soft-delete, sau đó đối soát với báo cáo tổng hợp để phát hiện chênh lệch.

Kết quả sau khi query (minh họa)

order_id	total_amount	status	deleted_at
ORD_005	15.000.000	CANCELLED	2026-06-25 10:30:00

ORD_005 đã bị xóa mềm nhưng vẫn nằm trong bảng. Nếu query báo cáo không lọc `deleted_at`, tổng doanh thu sẽ bị thổi phồng thêm 15 triệu.

GÓC SOI LỖI CỦA TESTER

Soft-delete là pattern phổ biến trong hầu hết hệ thống e-commerce, CRM, ERP. Khi test, QA cần kiểm tra mọi màn hình báo cáo: chạy lại query báo cáo không có điều kiện `WHERE deleted_at IS NULL` rồi so sánh kết quả — nếu tổng tiền hoặc số lượng thay đổi, đó là bug cần báo cáo ngay. Bước sau khi tìm thấy: đối chiếu với log để xác nhận đơn được xóa mềm có đúng thời điểm và đúng lý do không.

BÀI TẬP TỰ LUYỆN — PHẦN 1

Bài 1.1. Đếm số khách theo từng hạng thành viên (`membership_tier`) và chỉ ra hạng nào nằm NGOÀI danh sách hợp lệ (Standard, Silver, Gold, Platinum).

Gợi ý: *GROUP BY membership_tier để đếm; đối chiếu danh sách hợp lệ như Câu 11.*

Bài 1.2. Tìm những khách hàng có `customer_name` chứa khoảng trắng thừa ở đầu hoặc cuối.

Gợi ý: *So sánh customer_name với TRIM(customer_name); khác nhau là có khoảng trắng thừa.*

Bài 1.3. Sản phẩm nào được mua trong nhiều hơn một đơn hàng khác nhau?

Gợi ý: *Đếm số order_id PHÂN BIỆT cho mỗi product_id: COUNT(DISTINCT order_id).*

→ Lời giải đầy đủ ở **Phụ lục B** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 2 — Ràng buộc nghiệp vụ

Mỗi hệ thống đều có những quy tắc bất thành văn: tổng tiền ghi trên đơn phải khớp tổng từng dòng chi tiết, tồn kho không thể âm, đơn nào cũng phải có sản phẩm và thuộc về khách hàng thật. Khi tầng ứng dụng quên kiểm tra, dữ liệu sai sẽ lặn lẽ trôi vào database — và không có gì ở tầng DB ngăn lại.

PHẠM VI

Câu hỏi cốt lõi: **Dữ liệu có tuân theo luật chơi của hệ thống này không?** — Vi phạm ở phần này thường không gây lỗi ngay, nhưng âm thầm làm sai báo cáo tài chính, gây lỗi xử lý đơn hàng, hoặc bỏ sót khách hàng thật.

13

Đối soát tổng tiền đơn hàng với chi tiết items

TÌNH HUỐNG

Hệ thống lưu **total_amount** trong Orders nhưng không tự đối soát với tổng tính từ Order_Items. Khi có dữ liệu nhân đôi hoặc bug logic, hai con số lặn lẽ lệch nhau mà không có cảnh báo nào.

Bảng Orders — dòng đỏ: total_amount lệch với tổng Order_Items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount           AS ghi_trong_don,
       SUM(i.quantity * i.price) AS tinh_tu_items,
       o.total_amount
       - SUM(i.quantity * i.price) AS chenh_lech
FROM   Orders o
JOIN   Order_Items i ON o.order_id = i.order_id
GROUP BY o.order_id, o.total_amount
HAVING SUM(i.quantity * i.price) != o.total_amount;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Order_Items i ON o.order_id = i.order_id	INNER JOIN ghép mỗi đơn hàng với tất cả items của nó. Đơn không có item sẽ bị bỏ qua khỏi kết quả — đây là ý muốn: đơn rỗng không có gì để đối soát. Nếu cần phát hiện riêng đơn 0 items, viết thêm truy vấn dùng LEFT JOIN lọc WHERE i.item_id IS NULL.
GROUP BY o.order_id, o.total_amount	Gom toàn bộ items của cùng một đơn thành một nhóm. total_amount cần có trong GROUP BY vì được dùng trong HAVING.
HAVING SUM(i.quantity * i.price) != o.total_amount	HAVING lọc sau khi đã tính aggregate — chỉ giữ đơn có tổng items khác với total_amount đã ghi.
SELECT o.order_id, o.total_amount AS ghi_trong_don, SUM(i.quantity * i.price) AS tinh_tu_items, o.total_amount - SUM(...) AS chenh_lech	Hiển thị cả ba con số: ghi_trong_don (ghi trong đơn), tinh_tu_items (tính từ từng dòng sản phẩm), chenh_lech (hiệu giữa hai cột) — đủ thông tin để QA viết defect report mà không cần tra thêm.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY + HAVING** để đối soát header-detail: total_amount trong Orders phải bằng SUM(quantity × price) từ Order_Items.

Kết quả phát hiện **ba nguyên nhân khác nhau**:

- (1) ORD_001 lệch vì item bị nhân đôi (item 1 và item 7 cùng order + product).
 - (2) ORD_002 lệch vì total_amount bị ghi sai thủ công (Bug-B).
 - (3) ORD_005 lệch vì là đơn đã xóa mềm nhưng item chưa dọn, vẫn lọt vào đối soát (Câu 12).
- Cùng triệu chứng nhưng cách xử lý khác nhau — cần tra log để phân biệt.

Kết quả sau khi query (minh họa)

order_id	ghi_trong_don	tinh_tu_items	chenh_lech
ORD_001	32.000.000	62.000.000	-30.000.000
ORD_002	20.000.000	31.000.000	-11.000.000
ORD_005	15.000.000	3.000.000	12.000.000

3 đơn lệch: ORD_001 do item trùng (62M vs 32M); ORD_002 do total_amount ghi sai (31M vs 20M); ORD_005 đã bị xóa mềm nhưng vẫn tham gia đối soát — minh họa bug Câu 12 (soft-delete leak).

GÓC SOI LỖI CỦA TESTER

Hai bẫy cần biết khi dùng câu này: (1) INNER JOIN bỏ qua đơn rỗng — đơn không có item không xuất hiện trong kết quả, không phải không có lỗi mà bị lọt. (2) Câu này không lọc deleted_at, nên đơn đã xóa mềm vẫn lọt vào kết quả — nếu chỉ muốn đối soát đơn còn hiệu lực, thêm WHERE o.deleted_at IS NULL. Khi tìm thấy chênh lệch, hỏi dev: nguyên nhân là item bị nhân đôi hay total_amount bị ghi sai thủ công? Hai nguyên nhân có cách xử lý hoàn toàn khác nhau.

14

Phát hiện tồn kho âm hoặc NULL

TÌNH HUỐNG

Cột **stock** có hai kiểu lỗi cần soi cùng lúc: giá trị **âm** (bán quá số lượng thực có) và giá trị **NULL** (chưa từng được nhập kho). Hai lỗi này có nguyên nhân và cách xử lý khác hẳn nhau, nhưng đều khiến trang sản phẩm hiển thị sai tình trạng còn hàng.

Bảng Products — dòng đỏ: stock âm (Bug-C) và stock NULL:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       stock
FROM   Products
WHERE  stock < 0
       OR stock IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
WHERE stock < 0 OR stock IS NULL	Hai điều kiện gộp bằng OR: stock < 0 bắt tồn kho âm; stock IS NULL bắt dữ liệu chưa nhập. Không thể dùng = NULL vì NULL = NULL trả về UNKNOWN, không phải TRUE — đây là lỗi phổ biến nhất khi mới học SQL.
SELECT product_id, product_name, stock	Chiếu đủ thông tin để QA xác định sản phẩm và mức độ lệch.

Giải thích tổng thể

Một câu gộp hai loại lỗi: **tồn kho âm** (bán quá số lượng — thường do thiếu CHECK constraint hoặc hai người đặt cùng lúc) và **tồn kho NULL** (chưa có thông tin — khác với stock = 0 là hết hàng đã biết rõ). Lưu ý: phải dùng **IS NULL** để bắt NULL, viết **stock = NULL** không bao giờ cho kết quả.

Kết quả sau khi query (minh họa)

product_id	product_name	stock
PROD_003	Tai nghe Sony WH-1000XM5	-5
PROD_007	Chuot gaming Razer	(NULL)

2 sản phẩm lỗi, hai nguyên nhân khác nhau: PROD_003 tồn kho = -5 (bán quá số lượng — điều tra race condition hoặc logic trừ kho); PROD_007 tồn kho = NULL (chưa nhập kho — bổ sung dữ liệu hoặc thêm DEFAULT 0 vào schema).

GÓC SOI LỖI CỦA TESTER

Khi tìm thấy stock âm — hỏi dev hai câu trước khi viết defect report:

- (1) Luồng nào trừ kho? (đặt hàng, xác nhận thanh toán, hay xuất kho?) → xác định đúng điểm xảy ra lỗi.
- (2) DB có ràng buộc ngăn giá trị âm không? → nếu không có, đây là thiếu sót cần ghi vào defect.

Khi tìm thấy stock NULL — phân biệt hai tình huống trước khi báo lỗi:

- (1) Sản phẩm mới tạo chưa nhập kho → nhắc nhở bổ sung dữ liệu, chưa phải bug nghiêm trọng.
- (2) Sản phẩm đang bán mà stock = NULL → bug thật: hệ thống không biết còn hàng không, có thể cho phép đặt hàng tùy tiện.

15 Phát hiện đơn hàng PENDING đang chờ sản phẩm hết hàng

TÌNH HUỐNG

Khi hệ thống nhận đơn hàng mà không kiểm tra tồn kho tức thì, đơn PENDING vẫn được tạo dù sản phẩm đã hết hoặc tồn kho âm. QA cần phát hiện sớm trước khi đội vận hành phải xử lý thủ công — hoặc tệ hơn, khách chờ lâu mà không được giao hàng.

Đơn PENDING ghép với tồn kho sản phẩm (tổng hợp Orders + Order_Items + Products) — dòng đỏ:
ORD_006 đặt PROD_003 đang hết hàng (stock = -5):

order_id	customer_id	product_id	tồn_kho	status
ORD_004	C999	(rỗng — 0 items)	—	PENDING
ORD_006	C003	PROD_003	-5	PENDING
ORD_007	C001	PROD_004	20	PENDING

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       oi.product_id,
       p.product_name,
       p.stock      AS tồn_kho,
       oi.quantity  AS so_luong_dat
FROM   Orders o
JOIN   Order_Items oi ON o.order_id = oi.order_id
JOIN   Products  p  ON oi.product_id = p.product_id
WHERE  o.status = 'PENDING'
AND    p.stock <= 0;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o	Bắt đầu từ bảng Orders (mỗi đơn là một dòng), gán alias o . Đây là bảng gốc để lần lượt nối thêm chi tiết đơn và sản phẩm.
JOIN Order_Items oi ON o.order_id = oi.order_id	Nối Orders với chi tiết sản phẩm — bỏ qua đơn rỗng (ORD_004) vì không có dòng nào trong Order_Items để nối.
JOIN Products p ON oi.product_id = p.product_id	Nối tiếp sang bảng Products để lấy tồn kho hiện tại của từng sản phẩm trong đơn hàng.
WHERE o.status = 'PENDING' AND p.stock <= 0	o.status = 'PENDING' : chỉ xét đơn đang chờ xử lý. p.stock <= 0 : lọc ra sản phẩm hết hàng hoặc tồn kho âm — kết hợp với điều kiện trên để tìm đơn đang chờ nhưng không thể giao.

Giải thích tổng thể

Câu lệnh kết hợp ba bảng để đặt câu hỏi liên bảng: **đơn hàng nào đang chờ mà sản phẩm trong đó đã hết?**

Điều này khác với Câu 14 (chỉ hỏi 'sản phẩm nào hết hàng?') — ở đây ta hỏi thêm 'và nó đang nằm trong đơn PENDING không?' — cần JOIN để trả lời.

Đây là lỗi thường gặp khi hệ thống không có bước *reserve stock* (đặt chỗ tồn kho) ngay lúc tạo đơn.

Kết quả sau khi query (minh họa)

order_id	customer_id	product_id	product_name	ton_kho	so_luong_dat
ORD_006	C003	PROD_003	Tai nghe Sony WH-1000XM5	-5	1

ORD_006 đang PENDING nhưng PROD_003 có stock = -5. Đơn này không thể giao cho đến khi nhập thêm hàng.

GÓC SOI LỖI CỦA TESTER

Khi tìm thấy đơn PENDING + hết hàng, QA cần xác nhận thêm hai điều:

- (1) Hệ thống có cảnh báo cho đội vận hành không, hay đơn tồn đọng im lặng?
 - (2) Nếu DB dùng soft-delete, lọc thêm **AND o.deleted_at IS NULL** để tránh tính nhầm đơn đã hủy.
- Câu 14 tìm sản phẩm hết hàng; câu này tìm đúng đơn hàng bị ảnh hưởng — giúp ưu tiên đúng chỗ khi viết defect report.

16 Tìm đơn hàng không có dòng chi tiết nào**TÌNH HUỐNG**

Mỗi đơn hàng phải có ít nhất một sản phẩm. Đơn rỗng — tồn tại trong Orders nhưng không có dòng nào trong Order_Items — là dấu hiệu bug luồng xử lý: đơn được tạo nhưng bước thêm sản phẩm bị lỗi hoặc bị bỏ qua.

Bảng Orders — dòng đỏ: ORD_004 không có item nào trong Order_Items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.status
FROM   Orders o
LEFT JOIN Order_Items i
      ON o.order_id = i.order_id
WHERE  i.order_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o LEFT JOIN Order_Items i ON o.order_id = i.order_id	LEFT JOIN giữ TẤT CẢ đơn hàng, kể cả đơn không có dòng nào trong Order_Items. Với INNER JOIN, đơn rỗng sẽ biến mất.
WHERE i.order_id IS NULL	Sau LEFT JOIN, đơn không khớp có toàn bộ cột của Order_Items = NULL. Lọc những dòng đó là cách nhận diện đơn rỗng — đây là kỹ thuật truy vấn loại trừ.
SELECT o.order_id, o.customer_id, o.total_amount, o.status	Hiển thị đủ thông tin để QA điều tra: ai đặt, số tiền bao nhiêu, trạng thái gì.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + WHERE IS NULL** là cách chuẩn để tìm bản ghi không có con.
Nguyên lý: sau LEFT JOIN, mọi cột từ bảng bên phải sẽ là NULL với những dòng không khớp.
Trong data mẫu, ORD_004 (khách C999 không tồn tại) cũng không có item nào — đây là đơn có **hai lỗi chồng nhau**: orphan customer và rỗng items.

Kết quả sau khi query (minh họa)

order_id	customer_id	total_amount	status
ORD_004	C999	5.000.000	PENDING

ORD_004 không có item nào và customer_id C999 không tồn tại. Hai lỗi chồng nhau — cần điều tra lịch sử tạo đơn.

GÓC SOI LỖI CỦA TESTER

Đơn rỗng thường xuất hiện do bug luồng xử lý — đơn được tạo nhưng bước thêm sản phẩm bị lỗi hoặc timeout. Khi tìm thấy, hỏi dev: luồng tạo đơn có bước rollback không nếu thêm item thất bại? Nếu không có, đơn rỗng sẽ tiếp tục xuất hiện sau mỗi lần lỗi mạng hoặc lỗi timeout.

17 Tìm sản phẩm chưa bao giờ được bán

TÌNH HUỐNG

Sản phẩm có trong danh mục nhưng không xuất hiện trong bất kỳ đơn hàng nào. Có thể là sản phẩm mới chưa ra mắt, sản phẩm bị ẩn nhưng dữ liệu chưa dọn, hoặc sản phẩm bị lỗi khiến không ai thêm được vào giỏ hàng.

Bảng Products — dòng đỏ: chưa có trong Order_Items:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Điện thoại	30.000.000	50

product_id	product_name	category	price	stock
PROD_002	Bàn phím cơ Logitech	Phụ kiện	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phụ kiện	8.000.000	-5
PROD_004	Sạc du phong Anker	Phụ kiện	1.000.000	20
PROD_005	Bàn phím cơ Logitech	Phụ kiện	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phụ kiện	(NULL)	10
PROD_007	Chuột gaming Razer	Phụ kiện	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.price,
       p.stock
FROM   Products p
LEFT JOIN Order_Items oi
      ON p.product_id = oi.product_id
WHERE  oi.product_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p LEFT JOIN Order_Items oi ON p.product_id = oi.product_id	LEFT JOIN giữ TẤT CẢ sản phẩm, kể cả sản phẩm không có dòng nào trong Order_Items.
WHERE oi.product_id IS NULL	Sau LEFT JOIN, sản phẩm không khớp có oi.product_id = NULL. Đây chính xác là sản phẩm chưa bao giờ được bán.
SELECT p.product_id, p.product_name, p.price, p.stock	Chiều thêm price và stock để QA đánh giá: chưa bán vì lỗi, chưa có giá, hay chưa có hàng?

Giải thích tổng thể

Cùng kỹ thuật LEFT JOIN + WHERE IS NULL với Câu 7 và Câu 16, áp dụng cho chiều Products → Order_Items.

Trong data mẫu, 3 sản phẩm chưa bán — mỗi cái có lý do khác nhau:

(1) PROD_005: trùng tên PROD_002 — sản phẩm bị nhân đôi.

(2) PROD_006: price = NULL — chưa định giá.

(3) PROD_007: stock = NULL — chưa nhập kho.

Kết hợp với Câu 9, Câu 10 và Câu 14 để phân tích từng trường hợp.

Kết quả sau khi query (minh họa)

product_id	product_name	price	stock
PROD_005	Bàn phím cơ Logitech	2.000.000	30
PROD_006	Loa Bluetooth JBL	(NULL)	10
PROD_007	Chuột gaming Razer	1.500.000	(NULL)

3 sản phẩm chưa bán: PROD_005 trùng tên PROD_002, PROD_006 chưa có giá, PROD_007 chưa có tồn kho.

GÓC SOI LỖI CỦA TESTER

Sản phẩm chưa bán không nhất thiết là lỗi — có thể là hàng mới chưa ra mắt. Để phân biệt trước khi báo bug: xem thêm cột price và stock trong kết quả. Sản phẩm chưa bán mà price = NULL hoặc stock = NULL là dấu hiệu dữ liệu chưa hoàn chỉnh — ưu tiên điều tra trước. Sản phẩm chưa bán mà trùng tên sản phẩm đã có thì nghi nhân đôi — xác nhận bằng Câu 10.

18 Tìm khách hàng chưa có đơn hàng nào**TÌNH HUỐNG**

Khách hàng đã đăng ký tài khoản nhưng chưa đặt đơn nào. Có thể là dữ liệu test, tài khoản tạo nhầm, hoặc khách thật nhưng gặp lỗi khi checkout. Nhận diện nhóm này giúp làm sạch dữ liệu trước khi kiểm thử tính năng phân tích người dùng.

Bảng Customers — dòng đỏ: chưa có đơn hàng nào trong Orders:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       c.membership_tier,
       c.status
FROM   Customers c
LEFT JOIN Orders o
      ON c.customer_id = o.customer_id
WHERE  o.customer_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id	LEFT JOIN giữ TẤT CẢ khách hàng, kể cả người chưa có đơn hàng nào.
WHERE o.customer_id IS NULL	Sau LEFT JOIN, khách không có đơn sẽ có cột o.customer_id = NULL. Lọc đúng những khách đó.
SELECT c.customer_id, c.customer_name, c.membership_tier, c.status	Chiếu thêm tier và status để QA phân loại: tài khoản test, tài khoản lỗi, hay khách thật chưa mua?

Giải thích tổng thể

Cùng kỹ thuật LEFT JOIN + WHERE IS NULL, chiều Customers → Orders.
Trong data mẫu, 7 trong 10 khách chưa có đơn — phần lớn mang dấu hiệu tài khoản test (tên chứa 'Test', 'Ao Bug', khoảng trắng thừa, đánh số thủ công).
Khi làm sạch dữ liệu trước sprint, QA nên chạy câu này và đánh dấu tài khoản nào là test — tránh đưa vào báo cáo làm sai số liệu thật.

Kết quả sau khi query (mình họa)

customer_id	customer_name	membership_tier	status
C004	Khách Hang Ao Bug	Standard	ACTIVE
C005	Khách Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khách Test VIP	VIP	ACTIVE

7 khách chưa có đơn — phần lớn mang dấu hiệu tài khoản test. Cần đánh dấu để loại khỏi báo cáo production.

GÓC SOI LỖI CỦA TESTER

Câu này hay bị nhầm với Câu 7 (đơn hàng mờ côi). Điểm khác nhau:
(1) **Câu 7:** Orders không có Customers — đơn hàng mờ côi, thiếu chủ.
(2) **Câu 18:** Customers không có Orders — khách hàng chưa mua gì.
Hai hướng bổ sung cho nhau: Câu 7 bắt lỗi tăng Orders, Câu 18 bắt lỗi tăng Customers.

19 Tổng hợp chi tiêu theo từng khách hàng

TÌNH HUỐNG

QA cần bảng tổng hợp để kiểm tra: khách nào đã mua, bao nhiêu đơn, tổng tiền bao nhiêu. Bảng này là nền để phát hiện bất thường — khách chi tiêu quá cao/thấp, hoặc số đơn không khớp với dữ liệu loyalty.

Bảng Orders — dữ liệu đầu vào để tổng hợp chi tiêu:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25
ORD_006	C003	8.000.000	PENDING	2026-06-23
ORD_007	C001	20.000.000	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS so_don,
       SUM(o.total_amount) AS tong_chi_tieu
FROM   Customers c
JOIN   Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY tong_chi_tieu DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id	INNER JOIN: chỉ lấy khách đã có ít nhất một đơn. Khách chưa mua (C004–C010) sẽ không xuất hiện — dùng LEFT JOIN nếu muốn hiện cả họ với so_don = 0. ORD_004 (C999) cũng bị loại vì C999 không có trong Customers.
GROUP BY c.customer_id, c.customer_name	Gom tất cả đơn hàng của cùng một khách thành một nhóm để tính tổng.
SELECT ..., COUNT, SUM	COUNT(o.order_id) đếm số đơn; SUM(o.total_amount) cộng tổng tiền. Lưu ý: SUM dùng total_amount từ Orders là con số đã ghi sẵn trong đơn, không tính lại từ từng item — nếu dữ liệu bị chênh lệch giữa tổng đơn và tổng items, câu lệnh này sẽ không phát hiện được.
ORDER BY tong_chi_tieu DESC	Khách chi tiêu nhiều nhất lên đầu — để phát hiện outlier bất thường. Trên production, thêm LIMIT 10–20 để chỉ lấy nhóm khách chi tiêu cao nhất.

Giải thích tổng thể

Câu lệnh không tìm bug trực tiếp mà tạo **bảng nền để phát hiện bất thường**.

QA so sánh kết quả với dữ liệu hệ thống loyalty, CRM hoặc báo cáo tài chính — nếu con số lệch là có vấn đề.

Lưu ý: vì dùng total_amount từ Orders, kết quả bao gồm Bug-B (ORD_002 ghi sai).

Cần chạy Câu 13 trước để phát hiện và sửa dữ liệu lệch, sau đó Câu 19 mới phản ánh đúng thực tế.

Kết quả sau khi query (minh họa)

customer_id	customer_name	so_don	tong_chi_tieu
C001	Nguyen Van A	3	67.000.000
C002	Tran Van B	1	20.000.000
C003	Le Thi C	2	16.000.000

Chỉ 3/10 khách có đơn (C001, C002, C003); 7 khách C004–C010 chưa mua nên không xuất hiện. C001 dẫn đầu 67M (3 đơn: ORD_001 + ORD_005 soft-delete + ORD_007 ngày tương lai). C003 có 2 đơn (ORD_003 + ORD_006 — cặp double order từ Câu 29). Con số C002 (20M) là Bug-B — thực tế items cộng lại 31M.

GÓC SOI LỖI CỦA TESTER

Câu này dùng total_amount từ Orders — con số ghi sẵn, chưa chắc đúng. Trước khi dùng kết quả này để báo cáo, chạy Câu 13 trước để xác nhận không có đơn nào bị lệch. Bộ số tổng hợp ở đây chỉ đáng tin khi dữ liệu nền đã được kiểm tra sạch — đừng báo cáo con số C001 = 67M mà chưa kiểm tra đơn nào trong 3 đơn của C001 có lỗi không.

20

Phát hiện sản phẩm đã bán vượt quá tồn kho

TÌNH HUỐNG

Tổng số lượng đã bán (từ Order_Items) lớn hơn tồn kho hiện tại (từ Products). Đây là dấu hiệu của race condition, thiếu kiểm tra stock trước khi trừ, hoặc dữ liệu kho bị cập nhật sai sau khi xác nhận đơn. Lưu ý: phép so sánh này chỉ có nghĩa khi kho chưa được nhập thêm kể từ đầu kỳ (đúng với DB mẫu); hệ thống có nhập hàng định kỳ phải đối soát với bảng nhập–xuất kho.

Bảng Products — dòng đỏ: tồn kho có thể bị vượt quá:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.stock,
       SUM(oi.quantity) AS tong_da_ban
FROM   Products p
JOIN   Order_Items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.stock
HAVING SUM(oi.quantity) > p.stock;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id	INNER JOIN ghép mỗi sản phẩm với tất cả dòng trong Order_Items. Sản phẩm chưa có dòng nào trong Order_Items sẽ bị loại khỏi kết quả — đây là ý muốn: không bán được thì không có gì để so sánh với tồn kho. Nếu cần liệt kê sản phẩm chưa bán, dùng LEFT JOIN và lọc WHERE oi.item_id IS NULL.
GROUP BY p.product_id, p.product_name, p.stock	Gom tất cả dòng Order_Items của cùng một sản phẩm để tính tổng số đã bán.
HAVING SUM(oi.quantity) > p.stock	HAVING so sánh sau aggregate: tổng đã bán > tồn kho hiện tại là vi phạm ràng buộc nghiệp vụ.
SELECT ..., tong_da_ban	Hiển thị cả stock hiện tại và tổng đã bán để QA thấy ngay mức độ vượt quá.

Giải thích tổng thể

Câu lệnh phát hiện vi phạm ràng buộc: **tổng bán ra không được vượt tồn kho** (với giả định kho chưa nhập thêm — xem lưu ý ở phần Tình huống).

PROD_003 bị phát hiện vì stock = -5 và tong_da_ban = 2 (2 > -5).

PROD_004 cũng bị phát hiện: stock = 20 nhưng tong_da_ban = 22 ($22 > 20$) — hệ thống đã bán quá số lượng tồn kho hiện có.

Với hệ thống thực, câu này giúp phát hiện overselling (bán vượt tồn kho) trước khi khách hàng phàn nàn vì không nhận được hàng.

Kết quả sau khi query (minh họa)

product_id	product_name	stock	tong_da_ban
PROD_004	Sac du phong Anker	20	22
PROD_003	Tai nghe Sony WH-1000XM5	-5	2

PROD_003: tồn kho = -5 nhưng tổng đã bán = 2 — kho âm vẫn bị bán thêm. PROD_004: tồn kho = 20 nhưng tổng đã bán = 22 — vượt 2 đơn vị so với tồn kho hiện tại.

GÓC SOI LỖI CỦA TESTER

INNER JOIN làm câu này bỏ qua sản phẩm chưa có đơn nào — sản phẩm chưa bán không xuất hiện dù tồn kho đang âm. Khi tìm thấy overselling (bán vượt tồn kho), hỏi dev: hệ thống có bước reserve stock (đặt chỗ tồn kho) ngay khi tạo đơn không? Nếu không, race condition xảy ra khi hai người đặt cùng lúc — đây là lỗi đồng thời (concurrency bug), không chỉ là lỗi dữ liệu đơn thuần.

BÀI TẬP TỰ LUYỆN — PHẦN 2

Bài 2.1. Liệt kê các sản phẩm có giá cao hơn giá trung bình của toàn bộ sản phẩm.

Gợi ý: Dùng subquery (`SELECT AVG(price) FROM Products`) ngay trong mệnh đề `WHERE`.

Bài 2.2. Trong các đơn COMPLETED, đơn nào có total_amount KHÁC tổng tiền tính từ Order_Items?

Gợi ý: Lọc `WHERE status='COMPLETED'` trước, `JOIN + GROUP BY`, rồi `HAVING` so sánh hai tổng.

Bài 2.3. Sản phẩm nào CHƯA từng được bán nhưng vẫn còn tồn kho lớn hơn 0?

Gợi ý: `NOT EXISTS` để tìm sản phẩm chưa bán, kết hợp điều kiện `stock > 0`.

Bài 2.4. Tìm tất cả sản phẩm có giá (price) bằng NULL hoặc bằng 0.

Gợi ý: Dùng `IS NULL` để bắt giá chưa nhập và `<= 0` để bắt giá không hợp lệ — kết hợp bằng `OR`.

→ Lời giải đầy đủ ở **Phụ lục B** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 3 — Đối soát và tính toán

Phần này chuyển từ kiểm tra từng bản ghi sang tổng hợp và thống kê: tính doanh thu thực từ chi tiết đơn, đối soát tồn kho với lượng đã bán, xếp hạng sản phẩm, đo tỷ lệ trạng thái và phát hiện giá trị bất thường so với mặt bằng chung.

PHẠM VI

Câu hỏi cốt lõi: **Các con số tổng hợp có phản ánh đúng dữ liệu chi tiết không?** — Doanh thu, xếp hạng, tỷ lệ phần trăm đều được cộng dồn từ nhiều bản ghi; chỉ một nhóm bản ghi bấn cũng đủ làm cả báo cáo sai.

21 Tính doanh thu thực theo từng đơn từ Order_Items

TÌNH HUỐNG

Câu 19 tổng hợp chi tiêu dùng **total_amount** — con số ghi sẵn trong Orders, có thể sai như Bug-B ở ORD_002. Doanh thu thật phải tính trực tiếp từ Order_Items: **SUM(quantity × price)**. Câu này là điểm khởi đầu để QA xác minh độ tin cậy của dữ liệu trước khi đưa vào báo cáo.

Bảng Order_Items — dòng đỏ: item_id=7 nhân đôi PROD_001 trong ORD_001:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT order_id,
       COUNT(*)           AS so_dong_items,
       SUM(quantity)       AS tong_so_luong,
       SUM(quantity * price) AS doanh_thu_thuc
FROM   Order_Items
GROUP BY order_id
ORDER BY order_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items — 11 dòng trong dữ liệu mẫu.
GROUP BY order_id	Gom tất cả dòng có cùng order_id thành một nhóm. Mỗi nhóm đại diện cho một đơn hàng.
COUNT(*), SUM(quantity), SUM(quantity * price)	COUNT(*) đếm số dòng items của đơn. SUM(quantity) tổng số lượng sản phẩm. SUM(quantity × price) tính tiền thực từ items.
ORDER BY order_id	Sắp xếp kết quả theo mã đơn để so sánh dễ hơn với bảng Orders.

Giải thích tổng thể

Kỹ thuật **GROUP BY + aggregate** tổng hợp từ bảng chi tiết lên level đơn hàng.

COUNT(*) là chỉ số cảnh báo sớm: nếu con số lớn bất thường so với dự kiến, rất có thể có item bị trùng. Trong data mẫu, ORD_001 có 3 dòng thay vì 2 — **COUNT(*) = 3** là tín hiệu đầu tiên trước khi đối soát doanh_thu_thuc với total_amount.

Kết quả sau khi query (minh họa)

order_id	so_dong_items	tong_so_luong	doanh_thu_thuc
ORD_001	3	3	62.000.000
ORD_002	2	2	31.000.000
ORD_003	2	1	8.000.000
ORD_005	2	2	3.000.000
ORD_006	1	1	8.000.000
ORD_007	1	20	20.000.000

ORD_001: 3 dòng items → COUNT bất thường, doanh thu = 62M (lẽ ra 32M). ORD_002: tổng = 31M, khác total_amount 20M (Bug-B). ORD_003: 2 dòng — item 12 có quantity=0 nên tong_so_luong vẫn = 1 và doanh thu = 8M. ORD_006 (double order) và ORD_007 (ngày tương lai) xuất hiện thêm. ORD_004 vắng mặt vì không có item nào.

GÓC SOI LỖI CỦA TESTER

Khi thấy COUNT(*) của một đơn cao bất thường, không cần tính doanh thu ngay — tra trực tiếp các item của đơn đó ORDER BY item_id để xem dòng nào bị nhân đôi, rồi đối chiếu item_id cụ thể trước khi báo bug.

22

Xếp hạng sản phẩm bán chạy nhất theo doanh số

TÌNH HUỐNG

Báo cáo sản phẩm bán chạy là cơ sở để ra quyết định nhập hàng và marketing. Nhưng nếu dữ liệu Order_Items có item trùng, bảng xếp hạng sẽ sai — sản phẩm bình thường đột ngột lên top vì được tính nhiều lần.

Bảng Order_Items — dòng đỏ: item_id=7 làm PROD_001 bị tính 3 lần:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000

item_id	order_id	product_id	quantity	price
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       SUM(oi.quantity)           AS tong_so_luong,
       SUM(oi.quantity * oi.price) AS tong_doanh_so
FROM   Products p
JOIN   Order_Items oi
      ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY tong_doanh_so DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id	INNER JOIN ghép mỗi sản phẩm với tất cả items bán ra. Sản phẩm chưa bán (PROD_005, 006, 007) bị loại — dùng LEFT JOIN nếu muốn hiện cả chúng với doanh_so = NULL.
GROUP BY p.product_id, p.product_name	Gom tất cả dòng Order_Items của cùng một sản phẩm thành một nhóm để tính tổng số lượng và doanh số.
ORDER BY tong_doanh_so DESC	Sản phẩm doanh số cao nhất nổi lên đầu — dễ phát hiện outlier nếu chênh lệch bất thường.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY + ORDER BY** là nền của mọi báo cáo xếp hạng.
Kết quả hiện tại bị méo vì dữ liệu đầu vào có lỗi: PROD_001 xuất hiện 3 lần trong items (item 1, 4, 7) thay vì 2.
Doanh số thực của PROD_001 chỉ là 60M — con số 90M là ảo do item trùng.
Bảng xếp hạng chính xác phải bắt đầu bằng việc xác nhận dữ liệu sạch (Câu 4).

Kết quả sau khi query (minh họa)

product_id	product_name	tong_so_luong	tong_doanh_so
PROD_001	iPhone 15 Pro Max	3	90.000.000
PROD_004	Sac du phong Anker	22	22.000.000
PROD_003	Tai nghe Sony WH-1000XM5	2	16.000.000
PROD_002	Ban phim co Logitech	2	4.000.000

PROD_001 dẫn đầu với 90M — thực ra chỉ 60M nếu không có item 7 trùng. PROD_004 vọt lên hạng 2 (22 đơn vị, 22M) do item 14 có quantity=20 — đây là dấu hiệu đáng ngờ. Chạy Câu 4 để xác nhận trùng, sửa dữ liệu rồi mới tin vào kết quả xếp hạng này.

GÓC SOI LỖI CỦA TESTER

Trước khi dùng kết quả xếp hạng để ra quyết định kinh doanh, QA cần kiểm tra:

(1) **Câu 4:** có item nào bị trùng (order_id + product_id) không?

(2) **Câu 21:** COUNT(*) theo order có dòng nào bất thường không?

Dữ liệu bản ở Order_Items lan trực tiếp lên báo cáo cấp quản lý — đây là loại bug dễ bị bỏ qua nhất vì không gây lỗi hệ thống.

23

Kiểm tra giá trị tồn kho (stock × price)

TÌNH HUỐNG

Giá trị tồn kho = **stock × price** là chỉ số tài chính quan trọng trong quản lý kho. Nếu stock âm (Bug-C) hoặc price = NULL, giá trị tính ra sẽ bị âm hoặc NULL — báo cáo kế toán sẽ sai lệch.

Bảng Products — dòng đỏ: stock âm hoặc NULL sẽ cho gia_tri_kho sai:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       price,
       stock,
       price * stock AS gia_tri_kho
FROM   Products
ORDER BY gia_tri_kho;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products — 7 sản phẩm trong dữ liệu mẫu.
price * stock AS gia_tri_kho	MySQL tính tích hai cột và đặt tên kết quả là gia_tri_kho . Quy tắc: NULL × bất kỳ = NULL; âm × dương = âm.
ORDER BY gia_tri_kho	Sắp xếp tăng dần: MySQL đặt NULL trước tiên, rồi đến âm, rồi dương. Cách hiển thị này đưa dòng lỗi lên đầu bảng kết quả.

Giải thích tổng thể

Phép nhân đơn giản nhưng phản ánh ngay **hai loại lỗi dữ liệu** đã gặp ở Câu 9 và Câu 14:

- (1) **stock âm:** PROD_003 cho gia_tri_kho = -40.000.000 — không thể có kho giá trị âm.

(2) **NULL**: PROD_006 (price NULL) và PROD_007 (stock NULL) cho gia_tri_kho = NULL — không thể tính được giá trị kho, ảnh hưởng trực tiếp đến báo cáo tổng tài sản.

Kết quả sau khi query (minh họa)

product_id	product_name	price	stock	gia_tri_kho
PROD_006	Loa Bluetooth JBL	(NULL)	10	(NULL)
PROD_007	Chuot gaming Razer	1.500.000	(NULL)	(NULL)
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	-5	-40.000.000
PROD_004	Sac du phong Anker	1.000.000	20	20.000.000
PROD_005	Ban phim co Logitech	2.000.000	30	60.000.000
PROD_002	Ban phim co Logitech	2.000.000	100	200.000.000
PROD_001	iPhone 15 Pro Max	30.000.000	50	1.500.000.000

3 dòng bất thường ở đầu: 2 NULL (không tính được) và 1 âm -40M (tồn kho âm do Bug-C). ORDER BY gia_tri_kho tự động đưa các dòng lỗi lên đầu — không cần WHERE để lọc riêng.

GÓC SOI LỖI CỦA TESTER

Khi thấy giá trị kho âm (-40M), không phải chỉ báo bug 'giá trị âm' — cần hỏi dev: logic trừ kho xảy ra ở đâu trong luồng (đặt đơn, xác nhận, xuất kho) và có ràng buộc ngăn stock < 0 không? Câu trả lời quyết định severity của bug.

24

So sánh giá bán lịch sử với giá niêm yết hiện tại

TÌNH HUỐNG

Cột **price** trong Order_Items là giá tại thời điểm mua — snapshot của giá khi giao dịch xảy ra. Cột **price** trong Products là giá niêm yết hiện tại. Nếu hai con số khác nhau, có thể bình thường (giá thay đổi) hoặc là bug (ghi nhầm giá lúc tạo đơn). QA cần kiểm tra để phân biệt hai trường hợp.

Bảng Order_Items — so sánh oi.price (lúc mua) với Products.price (hiện tại):

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT oi.order_id,
       oi.product_id,
       oi.price      AS gia_luc_ban,
       p.price       AS gia_hien_tai,
       oi.price - p.price AS chenh_lech
FROM   Order_Items oi
JOIN   Products p
       ON oi.product_id = p.product_id
WHERE  oi.price <> p.price;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id	INNER JOIN ghép mỗi item với thông tin sản phẩm tương ứng. Kết quả có cả oi.price (lúc mua) và p.price (hiện tại) trên cùng một dòng.
WHERE oi.price <> p.price	Lọc chỉ những dòng có giá bán khác giá hiện tại. Kết quả rỗng = tất cả items được ghi đúng giá.
SELECT ..., oi.price - p.price AS chenh_lech	Tính độ lệch: dương = bán đắt hơn giá hiện tại; âm = bán rẻ hơn giá hiện tại.

Giải thích tổng thể

Order_Items.price là **giá snapshot** — lưu lại giá đúng lúc khách mua, không đổi kể cả khi Products.price thay đổi sau đó.

Item 12 (ORD_003/PROD_002): ghi giá 1.500.000 nhưng giá hiện tại của PROD_002 là 2.000.000 → chênh lệch -500.000. Đây là dấu hiệu giá snapshot bị ghi sai lúc tạo đơn, không phải thay đổi giá hợp lệ sau đó.

Câu này quan trọng khi team sửa giá sản phẩm: cần xác nhận đơn cũ đã dùng đúng giá cũ, không phải giá mới bị gán ngược.

Kết quả sau khi query (minh họa)

order_id	product_id	gia_luc_ban	gia_hien_tai	chenh_lech
ORD_003	PROD_002	1.500.000	2.000.000	-500.000

Item 12 (ORD_003/PROD_002) ghi giá 1.500.000 nhưng giá niêm yết PROD_002 là 2.000.000 — chênh -500.000. Cần xác nhận: giá PROD_002 từng là 1.500.000 trước đây (hợp lý), hay item này bị ghi sai giá ngay từ đầu (bug). Câu này nên chạy lại sau mỗi lần cập nhật bảng giá.

GÓC SOI LỖI CỦA TESTER

Không phải mọi chênh lệch giá đều là bug — khi có dòng trả về (như item 12 ở đây), cần phân biệt hai tình huống:

- (1) **Chênh lệch hợp lý**: giá sản phẩm tăng/giảm sau khi đơn đã đặt — oi.price đúng (giá tại thời điểm mua), p.price là giá mới. Không phải bug.
- (2) **Chênh lệch bất thường**: item vừa tạo đã khác giá niêm yết — có thể bug khi tạo đơn hoặc có discount không được ghi nhận đúng cách.

25

Tổng doanh thu chỉ tính đơn hàng COMPLETED

TÌNH HUỐNG

Doanh thu thật chỉ đến từ đơn đã hoàn tất. Nếu tổng hợp cả đơn CANCELLED hay PENDING, con số sẽ bị thổi phồng. QA cần xác minh hệ thống lọc đúng trạng thái khi tính doanh thu trước khi chốt số báo cáo cuối kỳ.

Bảng Orders — dòng đỏ: ORD_002 có total_amount sai (Bug-B):

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT c.customer_name,
       o.order_id,
       o.total_amount,
       o.order_date
FROM   Orders o
JOIN   Customers c
      ON o.customer_id = c.customer_id
WHERE  o.status = 'COMPLETED'
ORDER BY o.total_amount DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id	INNER JOIN lấy tên khách hàng cho mỗi đơn. ORD_004 (customer_id = C999 không tồn tại) tự động bị loại bởi JOIN này.
WHERE o.status = 'COMPLETED'	Lọc chỉ đơn đã hoàn tất. ORD_003 (CANCELLED) và ORD_004 (PENDING) không được tính vào doanh thu.
ORDER BY o.total_amount DESC	Đơn có giá trị lớn nhất lên đầu — để phát hiện đơn bất thường hoặc Bug-B khi đối chiếu với Câu 13.

Giải thích tổng thể

Thêm **WHERE status = 'COMPLETED'** là điều kiện bắt buộc trong mọi câu báo cáo doanh thu.

Trong data mẫu, tổng total_amount hai đơn COMPLETED = 32M + 20M = 52M.

Nhưng ORD_002 ghi total_amount = 20M trong khi tổng items thực = 31M (Bug-B) — tức 52M này đã thấp hơn thực tế. Đừng vội chốt một con số 'doanh thu thực' ở đây: cần làm sạch dữ liệu lệch (Câu 13) trước khi đưa vào báo cáo.

Kết quả sau khi query (minh họa)

customer_name	order_id	total_amount	order_date
Nguyen Van A	ORD_001	32.000.000	2026-06-20
Tran Van B	ORD_002	20.000.000	2026-06-22

2 đơn COMPLETED, tổng 52M — nhưng ORD_002 ghi 20M trong khi tổng items thực là 31M (Bug-B), nên con số 52M chưa đáng tin.

GÓC SOI LỖI CỦA TESTER

Ngoài chuyện total_amount có thể sai, cần thống nhất với spec định nghĩa 'doanh thu':

(1) Chỉ tính COMPLETED, hay gồm cả đơn đã giao nhưng status chưa kịp cập nhật?

(2) Đơn COMPLETED nhưng sau đó bị hoàn trả (refund) có bị trừ khỏi doanh thu không?

Trả lời sai một trong hai câu này thì báo cáo lệch dù câu SQL viết đúng.

26 Phân tích tỷ lệ đơn hàng theo trạng thái

TÌNH HUỐNG

Tỷ lệ CANCELLED cao là dấu hiệu vấn đề UX, luồng thanh toán, hoặc bug nghiệp vụ. QA cần bảng thống kê trạng thái đơn để theo dõi xu hướng theo sprint và phát hiện khi tỷ lệ một trạng thái thay đổi bất thường.

Bảng Orders — dữ liệu đầu vào để phân tích trạng thái:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25
ORD_006	C003	8.000.000	PENDING	2026-06-23
ORD_007	C001	20.000.000	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT status,
       COUNT(*) AS so_don,
       ROUND(COUNT(*) * 100.0 /
            (SELECT COUNT(*) FROM Orders), 1)
       AS phan_tram
FROM   Orders
GROUP BY status
ORDER BY so_don DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
GROUP BY status	Gom tất cả đơn có cùng trạng thái thành một nhóm. Kết quả là 1 dòng cho mỗi giá trị status tồn tại trong bảng.
COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Orders)	Subquery đếm tổng số đơn. Chia COUNT nhóm cho tổng rồi nhân 100 để ra phần trăm. ROUND(..., 1) làm tròn 1 chữ số thập phân.
ORDER BY so_don DESC	Trạng thái phổ biến nhất lên đầu.

Giải thích tổng thể

Subquery (**SELECT COUNT(*) FROM Orders**) trong SELECT là scalar subquery — trả về một giá trị dùng làm mẫu số cho tất cả các dòng.

Trên MySQL, toán tử / luôn cho số thập phân nên viết 100 hay 100.0 đều ra đúng. Dùng **100.0** là thói quen an toàn khi mang câu lệnh sang SQL Server hoặc PostgreSQL — nơi chia hai số nguyên bị cắt mất phần thập phân (kết quả thành 0).

Với data mẫu (7 đơn): PENDING = 42.9%, COMPLETED = 28.6%, CANCELLED = 28.6%.

Kết quả sau khi query (minh họa)

status	so_don	phan_tram
PENDING	3	42.9
COMPLETED	2	28.6
CANCELLED	2	28.6

PENDING chiếm tỷ lệ cao nhất (3/7 đơn, 42.9%) — đáng lo nếu trên production. Bao gồm ORD_004 (C999-orphan), ORD_006 (double order), ORD_007 (ngày tương lai). Câu này có giá trị thật khi chạy trên dữ liệu production đủ lớn để phát hiện xu hướng bất thường.

GÓC SOI LỖI CỦA TESTER

Hai lưu ý khi dùng câu này trên production:

- (1) **Trạng thái lạ**: nếu xuất hiện status không trong danh sách chuẩn, đây là dòng bất thường cần điều tra — kết hợp với Câu 11 (kỹ thuật ENUM check).
- (2) **Tỷ lệ thay đổi đột ngột**: CANCELLED tăng từ 10% lên 30% sau một sprint là tín hiệu cần xem lại luồng checkout hoặc thanh toán.

27

Tổng doanh thu theo danh mục sản phẩm

TÌNH HUỐNG

Quản lý sản phẩm cần biết danh mục nào đóng góp bao nhiêu vào doanh thu để ra quyết định nhập hàng và phân bổ ngân sách. Câu lệnh này tổng hợp từ Order_Items lên level danh mục qua bảng Products.

Bảng Order_Items — dòng đỏ: item_id=7 làm danh mục Điện thoại bị thổi phồng:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT p.category,
       COUNT(DISTINCT oi.order_id) AS so_don_co_sp,
       SUM(oi.quantity)           AS tong_so_luong,
       SUM(oi.quantity * oi.price) AS tong_doanh_so
FROM   Order_Items oi
JOIN   Products p
      ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY tong_doanh_so DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id	INNER JOIN lấy thông tin category từ Products cho mỗi dòng Order_Items. Sản phẩm chưa bán không xuất hiện — danh mục chỉ tồn tại nếu có đơn.
GROUP BY p.category	Gom tất cả items về cùng danh mục thành một nhóm. Mỗi dòng kết quả = một danh mục.
COUNT(DISTINCT oi.order_id), SUM(oi.quantity * oi.price)	COUNT(DISTINCT order_id) đếm số đơn có ít nhất 1 sản phẩm thuộc danh mục này (không tính trùng). SUM(quantity × price) tính tổng doanh số.
ORDER BY tong_doanh_so DESC	Danh mục doanh số cao nhất lên đầu.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY category** tổng hợp dữ liệu qua hai bảng lên một level cao hơn.

Kết quả bị ảnh hưởng bởi item trùng: 'Dien thoai' cho 90M thay vì 60M thực tế.

COUNT(DISTINCT order_id) thay vì **COUNT(*)**: nếu một đơn có 2 item cùng danh mục, đơn đó chỉ được đếm 1 lần — phản ánh đúng số đơn có mua sản phẩm danh mục này.

Kết quả sau khi query (minh họa)

category	so_don_co_sp	tong_so_luong	tong_doanh_so
Dien thoai	2	3	90.000.000
Phu kien	6	26	42.000.000

'Dien thoai' dẫn đầu 90M — bị thổi phồng do item 7 trùng. Thực tế là 60M. 'Phu kien' tăng mạnh lên 42M / 26 đơn vị (6 đơn) do item 14 có quantity=20 của ORD_007. Đây là ví dụ điển hình: một dòng dữ liệu bất thường (qty=20) đã kéo lệch toàn bộ báo cáo danh mục.

GÓC SOI LỖI CỦA TESTER

Kết quả chỉ có 2 danh mục vì INNER JOIN loại sản phẩm chưa bán — nếu báo cáo không đề cập đến điều này, người đọc có thể tưởng 'Dien thoai' và 'Phu kien' là toàn bộ danh mục. Trước khi tin vào doanh số 90M của 'Dien thoai', kiểm tra item trùng bằng Câu 4 — item 7 đang thổi phồng con số này lên 30M.

28

Đối soát tồn kho hiện tại với lượng đã bán ra

TÌNH HUỐNG

Tồn kho hiện tại (stock) + lượng đã bán (SUM items) phải khớp với tồn kho ban đầu. Nếu con số ước tính không hợp lý — âm, quá thấp hoặc quá cao — có thể có giao dịch bị bỏ sót, dữ liệu kho bị sửa thủ công, hoặc bug trong logic trừ kho.

Bảng Products — dòng đỏ: PROD_003 stock âm sẽ cho uoc_tinh_ban_dau bất thường:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.stock AS ton_kho_hien_tai,
       COALESCE(SUM(oi.quantity), 0) AS tong_da_ban,
       p.stock
       + COALESCE(SUM(oi.quantity), 0)
       AS uoc_tinh_ban_dau
FROM   Products p
LEFT JOIN Order_Items oi
      ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.stock
ORDER BY p.product_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p LEFT JOIN Order_Items oi ON p.product_id = oi.product_id	LEFT JOIN giữ TẤT CẢ sản phẩm, kể cả chưa bán. Sản phẩm chưa bán sẽ có SUM(oi.quantity) = NULL từ phía Order_Items.
COALESCE(SUM(oi.quantity), 0) AS tong_da_ban	COALESCE chuyển NULL thành 0 cho sản phẩm chưa bán. Không dùng COALESCE thì tong_da_ban = NULL — phép cộng tiếp theo sẽ bị NULL.
p.stock + COALESCE(SUM(oi.quantity), 0) AS uoc_tinh_ban_dau	Ước tính tồn kho ban đầu = tồn hiện tại + đã bán. Nếu con số này âm → có giao dịch khi kho đã âm.
GROUP BY p.product_id, p.product_name, p.stock	p.stock phải có trong GROUP BY vì được dùng trong SELECT nhưng không phải aggregate.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + COALESCE + aggregate** tạo bảng đối soát kho đầy đủ.

PROD_003: uoc_tinh_ban_dau = -5 + 2 = -3 — tồn kho ước tính ban đầu âm, vật lý không thể xảy ra → xác nhận dữ liệu tồn kho đã sai ở đâu đó, cần điều tra.

PROD_004: $20 + 22 = 42$ — ước tính ban đầu 42 đơn vị nhưng chỉ còn 20, cần xem xét.
PROD_001: $50 + 3 = 53$ — lẽ ra phải là 52 nếu chỉ bán 2 unit, chênh 1 đơn vị chính xác bằng item 7 trùng.

Kết quả sau khi query (minh họa)

product_id	product_name	ton_kho_hien_tai	tong_da_ban	uoc_tinh_ban_dau
PROD_001	iPhone 15 Pro Max	50	3	53
PROD_002	Ban phim co Logitech	100	2	102
PROD_003	Tai nghe Sony WH-1000XM5	-5	2	-3
PROD_004	Sac du phong Anker	20	22	42
PROD_005	Ban phim co Logitech	30	0	30
PROD_006	Loa Bluetooth JBL	10	0	10
PROD_007	Chuot gaming Razer	(NULL)	0	(NULL)

PROD_003: $uoc_tinh_ban_dau = -3$ → bất khả thi, xác nhận tồn kho không nhất quán. PROD_004: $uoc_tinh = 42$ (đã bán 22, còn 20) — hợp lý về mặt số học nhưng cần kiểm tra vì $tong_da_ban > stock$. PROD_001: 53 thay vì 52 → item trùng thời phỏng thêm 1 đơn vị.

GÓC SOI LỖI CỦA TESTER

COALESCE là hàm quan trọng khi LEFT JOIN: không dùng sẽ cho NULL không tính được.

Hai trường hợp kết quả bất thường cần điều tra thêm:

(1) **uoc_tinh_ban_dau âm**: tồn kho âm là bất khả thi về mặt vật lý — bug dữ liệu gốc cần điều tra.

(2) **uoc_tinh_ban_dau quá cao**: có thể items bị trùng hoặc kho không trừ đúng.

PROD_007 trả về NULL vì $stock = NULL$ — COALESCE chỉ xử lý phía $tong_da_ban$, không xử lý $p.stock$.

29

Phát hiện đơn hàng bị tạo trùng (double order)

TÌNH HUỐNG

Khi người dùng bấm nút Đặt hàng nhiều lần hoặc xảy ra lỗi mạng khiến request gửi hai lần, hệ thống có thể tạo hai đơn giống nhau trong tích tắc. Đây là lỗi double-charge nghiêm trọng — khách bị trừ tiền hai lần cho cùng một đơn mà không hay biết.

Bảng Orders — kiểm tra có cặp đơn nào trùng customer + tổng tiền + ngày không:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25
ORD_006	C003	8.000.000	PENDING	2026-06-23
ORD_007	C001	20.000.000	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT customer_id,
       total_amount,
       order_date,
       COUNT(*) AS so_lan
FROM   Orders
GROUP BY customer_id, total_amount, order_date
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
GROUP BY customer_id, total_amount, order_date	Gom theo bộ ba: cùng khách, cùng tổng tiền, cùng ngày. Nếu một nhóm có nhiều hơn 1 đơn — rất có thể là double order.
HAVING COUNT(*) > 1	Chỉ giữ nhóm xuất hiện từ 2 lần trở lên — dấu hiệu của đơn bị tạo trùng.

Giải thích tổng thể

Kỹ thuật **GROUP BY nhiều cột + HAVING COUNT** — cùng mẫu với Câu 3 và Câu 4, áp dụng cho bảng Orders để phát hiện double submission.

ORD_003 (CANCELLED) và ORD_006 (PENDING) cùng thuộc C003, cùng total_amount 8.000.000, cùng order_date 2026-06-23 → nhóm này có COUNT(*) = 2, vượt HAVING COUNT(*) > 1.

Câu này nên chạy sau mỗi đợt stress test hoặc khi khách phản ánh bị trừ tiền hai lần.

Kết quả sau khi query (minh họa)

customer_id	total_amount	order_date	so_lan
C003	8.000.000	2026-06-23	2

C003 có 2 đơn cùng ngày 2026-06-23, cùng tổng tiền 8M — dấu hiệu double submission điển hình. ORD_003 (CANCELLED) và ORD_006 (PENDING) — cần xác minh đây là hai đơn riêng biệt hay cùng một đơn bị tạo hai lần.

GÓC SOI LỖI CỦA TESTER

Khi tìm thấy double order, trước khi xóa bản nào hỏi khách hoặc kiểm tra payment log: cả hai đơn có phát sinh transaction thanh toán không? Nếu có, đây là lỗi double-charge nghiêm trọng — cần hoàn tiền trước khi xóa bản thừa. Nếu chỉ một đơn được thanh toán, giữ đơn đó và hủy đơn còn lại. Câu này gộp theo ngày — trên hệ thống lưu cả giờ phút, double order thực tế thường cách nhau vài giây đến vài phút.

30

Phát hiện đơn hàng có giá trị bất thường (outlier)

TÌNH HUỐNG

Đơn hàng có total_amount quá cao so với mức bình thường có thể do: nhập nhầm số lượng, giá sản phẩm bị set sai, hoặc bug logic tính tiền. Câu lệnh này tìm đơn vượt ngưỡng 1.5× trung bình như một bước kiểm tra nhanh.

Bảng Orders — dòng đỏ: ORD_001 vượt ngưỡng 1.5× trung bình:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22

order_id	customer_id	total_amount	status	order_date
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25
ORD_006	C003	8.000.000	PENDING	2026-06-23
ORD_007	C001	20.000.000	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       total_amount
FROM   Orders
WHERE  total_amount >
       (SELECT AVG(total_amount) * 1.5
        FROM Orders)
ORDER BY total_amount DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE total_amount > (SELECT AVG(total_amount) * 1.5 FROM Orders)	Subquery tính trung bình total_amount rồi nhân 1.5 làm ngưỡng. Trong data mẫu (7 đơn): AVG ≈ 15.430.000 → ngưỡng ≈ 23.140.000. Chỉ ORD_001 (32M) vượt ngưỡng này.
ORDER BY total_amount DESC	Đơn có giá trị cao nhất lên đầu để QA xem xét trước.

Giải thích tổng thể

Dùng **subquery trong WHERE** để so sánh từng dòng với một ngưỡng động tính từ chính bảng đó — không cần biết trước ngưỡng là bao nhiêu.
AVG trong data mẫu (7 đơn) ≈ 15.430.000 → ngưỡng 1,5× ≈ 23.140.000.
ORD_001 (32M) vượt ngưỡng và được flag. Điều này không có nghĩa là lỗi — đây là điểm khởi đầu để QA điều tra thêm bằng Câu 13.

Kết quả sau khi query (minh họa)

order_id	customer_id	total_amount
ORD_001	C001	32.000.000

ORD_001 (32M) vượt ngưỡng 1.5× trung bình (khoảng 23.140.000). Tổng tiền 32M là hợp lệ (30M + 2M), nhưng doanh thu từ items = 62M do item trùng — đây mới là bất thường thật sự, cần Câu 13 để phát hiện.

GÓC SOI LỖI CỦA TESTER

Ngưỡng 1.5× là điểm khởi đầu điều tra, không phải kết luận bug. Khi tìm thấy đơn vượt ngưỡng, hỏi: đơn B2B thường có giá trị lớn hơn B2C nhiều lần — cần phân tách hai nhóm trước khi so sánh. Bước tiếp theo khi flag được đơn: chạy Câu 13 để kiểm tra total_amount có khớp với tổng items không — đó mới là bằng chứng xác định lỗi, không phải chỉ vì giá trị lớn.

BÀI TẬP TỰ LUYỆN — PHẦN 3

Bài 3.1. Tính tổng doanh thu thực (số lượng × giá) của từng đơn hàng, sắp xếp giảm dần.

Gợi ý: *SUM(quantity * price) gom theo GROUP BY order_id.*

Bài 3.2. Mỗi danh mục (category) có bao nhiêu sản phẩm, kể cả sản phẩm chưa bán?

Gợi ý: *Đếm trực tiếp trên bảng Products (không JOIN Order_Items), GROUP BY category.*

Bài 3.3. Khách hàng nào có tổng chi tiêu cao hơn mức trung bình của các khách đã từng mua?

Gợi ý: *Tính tổng mỗi khách, rồi so với AVG của chính các tổng đó (subquery lồng).*

→ Lời giải đầy đủ ở **Phụ lục B** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 4 — Biên và dữ liệu bất thường

Người dùng thật luôn nhập những thứ ngoài dự đoán: chuỗi quá dài, ký tự lạ, số âm, ngày tháng vô lý. Đây là nhóm câu lệnh để tìm những giá trị outlier (nằm ngoài vùng bình thường) đã lọt vào database.

PHẠM VI

Câu hỏi cốt lõi: **Dữ liệu có nằm trong giới hạn nghiệp vụ cho phép không?** — Số âm, ngày tương lai, chuỗi rỗng, giá trị ngoài enum — những thứ form nhập liệu lẽ ra phải chặn từ đầu nhưng đã lọt qua.

31

Tìm tên khách hàng chứa ký tự bất thường

TÌNH HUỐNG

Form nhập liệu thường chỉ chặn ký tự đặc biệt ở frontend. Nếu backend API không validate, tên khách có thể chứa ký tự số, ngoặc, ký tự lạ — gây lỗi khi render, export PDF, hoặc tích hợp hệ thống bên ngoài vốn kỳ vọng dữ liệu sạch.

Bảng Customers — dòng đỏ: tên chứa ký tự ngoài chữ cái thường:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name
FROM   Customers
WHERE  customer_name REGEXP '[0-9()]';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE customer_name REGEXP '[0-9()]'	REGEXP kiểm tra chuỗi theo mẫu biểu thức chính quy. Pattern [0-9()] khớp với bất kỳ ký tự nào là chữ số (0–9) hoặc dấu ngoặc tròn.
SELECT customer_id, customer_name	Chiếu hai cột để QA xác minh và quyết định chuẩn hóa.

Giải thích tổng thể

REGEXP (Regular Expression) là công cụ mạnh để kiểm tra định dạng dữ liệu mà LIKE không làm được. Pattern **[0-9()]** khớp với CHỮ SỐ hoặc DẤU NGOẶC — hai loại ký tự không nên xuất hiện trong tên người thật.

C009 'Nguyen Van A (2)' bị bắt vì có cả '(' ')' và chữ số '2' — dấu hiệu tài khoản nhân bản hoặc dữ liệu test được đánh số bằng tay, không phải tên thật.

Kết quả sau khi query (minh họa)

customer_id	customer_name
C009	Nguyen Van A (2)

1 bản ghi: C009 có tên chứa ngoặc và số — dấu hiệu dữ liệu test hoặc tài khoản nhân bản được đánh số thủ công.

GÓC SOI LỖI CỦA TESTER

C009 'Nguyen Van A (2)' là ví dụ điển hình của tài khoản nhân bản, đánh số bằng tay ở cuối tên để tạo bản ghi mới. Khi tìm thấy loại này, hỏi dev: hệ thống có validate định dạng tên ở tầng backend không, hay chỉ validate ở frontend? Nếu chỉ ở frontend, người dùng có thể gửi thẳng qua API call để lách. Pattern REGEXP có thể mở rộng cho ký tự đặc biệt khác, nhưng với tên tiếng Việt nên dùng whitelist thủ công hơn là blacklist.

32 Tìm email không đúng định dạng cơ bản

TÌNH HUỐNG

Câu 5 đã bắt email NULL và chuỗi rỗng. Câu này đi thêm một bước: kiểm tra email **có nội dung** nhưng thiếu ký tự @ hoặc dấu chấm domain — dấu hiệu nhập sai hoặc bypass validation bằng cách nhập chữ bừa.

Bảng Customers — dòng đỏ: email NULL, rỗng hoặc không có @/dấu chấm:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  email IS NULL
       OR TRIM(email) = ''
       OR email NOT LIKE '%@%'
       OR email NOT LIKE '%.%';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE email IS NULL OR TRIM(email) = ''	Hai điều kiện này trùng với Câu 5 — nhắc lại ở đây để câu lệnh hoạt động độc lập, không cần chạy thêm câu khác.
OR email NOT LIKE '%@%' OR email NOT LIKE '%.%'	NOT LIKE '%@%' : email không chứa ký tự @ — không thể hợp lệ. NOT LIKE '%.%' : không có dấu chấm — thiếu phần domain (vd .com).
SELECT customer_id, customer_name, email	Chiếu đủ thông tin để QA liên hệ xác nhận email thật.

Giải thích tổng thể

Câu này kiểm tra email theo **ba tầng**:

- (1) Tầng tồn tại: NULL hoặc chuỗi rỗng → không có email.
- (2) Tầng cấu trúc: thiếu @ → không thể là email hợp lệ.
- (3) Tầng domain: thiếu dấu chấm → không có phần .com/.vn.

LIKE '%@%.%' không bắt được email như 'abc@' hay '@domain.com' — với dữ liệu nghiêm ngặt, dùng REGEXP thay thế.

Kết quả sau khi query (minh họa)

customer_id	customer_name	email
C006	Pham Van X	(NULL)
C007	Nguyen Thi Y	

C006: email NULL — chưa có. C007: email rỗng — nhập bỏ qua. Trên data mẫu, hai tầng kiểm tra @ và dấu chấm CHƯA bắt thêm dòng nào ngoài NULL/rỗng (các email còn lại đều đúng cấu trúc) — nhưng đó là tuyển phòng thủ quan trọng trên dữ liệu thật khi người dùng nhập email sai cấu trúc.

GÓC SOI LỖI CỦA TESTER

LIKE là kiểm tra nhanh nhưng lỏng lẻo — không bắt được các trường hợp như:

- (1) 'abc@': có @ nhưng không có domain → LIKE '%@%' vẫn pass.
- (2) '@domain.com': thiếu phần local → LIKE cũng pass.

Để kiểm tra chặt hơn, dùng REGEXP:

WHERE email NOT REGEXP '^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}\$'

33

Kiểm tra số lượng sản phẩm bất thường trong Order_Items

TÌNH HUỐNG

Cột quantity phải luôn ≥ 1 . Quantity = 0 là đơn trống về số lượng nhưng vẫn tồn tại trong DB — lỗi logic. Quantity âm có thể là bug trừ kho ngược chiều hoặc dữ liệu hoàn trả bị ghi sai bằng. Quantity quá lớn (> 1000) cũng cần xem xét.

Bảng Order_Items — kiểm tra quantity có trong ngưỡng hợp lệ không:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       quantity
FROM   Order_Items
WHERE  quantity <= 0
       OR quantity > 1000;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items .
WHERE quantity <= 0 OR quantity > 1000	Hai điều kiện biên: <= 0 bắt lỗi âm và zero; > 1000 bắt số lượng bất thường lớn (ngưỡng tùy ngành hàng).
SELECT item_id, order_id, product_id, quantity	Chiều đủ thông tin để QA xác định item nào vi phạm và thuộc đơn nào.

Giải thích tổng thể

Kiểm tra biên dưới (**quantity <= 0**) và biên trên (**quantity > 1000**) là cặp kiểm tra chuẩn cho mọi trường số lượng.

Item 12 (ORD_003/PROD_002) có quantity = 0 — vi phạm biên dưới. Đơn không thể tồn tại với 0 sản phẩm; đây là lỗi logic, không phải đơn bình thường hay đơn hoàn hàng (hoàn hàng phải dùng bảng riêng). Item 14 (ORD_007/PROD_004) có quantity = 20 — không vi phạm ($20 \leq 1000$), hợp lệ về kỹ thuật nhưng cần kiểm tra nghiệp vụ (mua 20 sạc dự phòng cùng lúc có bình thường không?).

Kết quả sau khi query (minh họa)

item_id	order_id	product_id	quantity
12	ORD_003	PROD_002	0

Item 12 (ORD_003/PROD_002) có quantity = 0 — vi phạm ràng buộc biên dưới. Một đơn hàng không thể chứa sản phẩm với số lượng bằng 0. Cần ADD CHECK CONSTRAINT hoặc validate tại tầng application để ngăn từ đầu.

GÓC SOI LỖI CỦA TESTER

Ngưỡng > 1000 chỉ là ví dụ — điều chỉnh tùy nghiệp vụ: bán lẻ B2C thì quantity > 10 đã đáng ngờ, bán buôn B2B thì có thể > 10.000 mới bất thường. Khi tìm thấy quantity = 0 (item 12), hỏi dev: đây là lỗi khi nhập đơn hay có luồng nào cố tình set về 0 (hoàn trả, điều chỉnh)? Nếu là hoàn trả, nên ghi vào bảng riêng thay vì để trong Order_Items với số lượng 0.

34**Kiểm tra ngày đặt hàng bất thường (tương lai hoặc quá xa quá khứ)****TÌNH HUỐNG**

Ngày đặt hàng trong tương lai là dấu hiệu đồng hồ server sai, dữ liệu được nhập thủ công với ngày sai, hoặc bug timezone. Ngày quá xa quá khứ (trước 2020) gợi ý dữ liệu migration bị lỗi hoặc giá trị placeholder chưa được cập nhật.

Bảng Orders — kiểm tra order_date có trong khoảng hợp lý không:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25
ORD_006	C003	8.000.000	PENDING	2026-06-23
ORD_007	C001	20.000.000	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       order_date
FROM   Orders
WHERE  order_date > CURDATE()
       OR order_date < '2020-01-01';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE order_date > CURDATE() OR order_date < '2020-01-01'	CURDATE() trả về ngày hiện tại của server — không cần hardcode ngày vào câu lệnh. Ngày < 2020-01-01 là ngưỡng tự chọn; điều chỉnh tùy thời điểm hệ thống bắt đầu hoạt động.
SELECT order_id, customer_id, order_date	Chiếu ngày để QA xác minh nguồn gốc dữ liệu.

Giải thích tổng thể

Dùng **CURDATE()** thay vì hardcoded ngày giúp câu lệnh luôn đúng mà không cần sửa theo thời gian.

ORD_007 có order_date = 2027-01-01 — vượt CURDATE() → ngày ở tương lai, là dấu hiệu đồng hồ server sai, nhập tay sai, hoặc bug timezone.

Câu này nên chạy định kỳ — đặc biệt sau khi deploy tính năng mới có liên quan đến xử lý thời gian hoặc timezone.

Lưu ý khi thực hành: kết quả trên đúng khi bạn chạy TRƯỚC ngày 2027-01-01. Nếu chạy sau ngày đó, ORD_007 không còn là 'tương lai' nữa và kết quả sẽ rỗng — hãy sửa order_date của ORD_007 thành một ngày xa hơn hiện tại để tái hiện.

Kết quả sau khi query (minh họa)

order_id	customer_id	order_date
ORD_007	C001	2027-01-01

ORD_007 có order_date = 2027-01-01 — hơn nửa năm trong tương lai so với hôm nay. Cần xác minh: đây là đặt hàng trước (pre-order) được phép, hay dữ liệu bị nhập sai ngày?

GÓC SOI LỖI CỦA TESTER

Cạm bẫy timezone: CURDATE() trả về ngày theo timezone của MySQL server.

Nếu app chạy ở timezone khác, ngày có thể lệch nhau quanh nửa đêm: đơn đặt lúc 06:30 sáng 25/06 giờ Việt Nam (UTC+7) = 23:30 đêm 24/06 giờ UTC — app hiển thị ngày 25 nhưng server UTC ghi ngày 24.

Khi nghi ngờ timezone gây lỗi, thêm điều kiện: **TIMESTAMPDIFF(HOUR, order_date, NOW()) < -8** để bắt đơn 'tương lai' trong vòng 8 giờ.

35**Tìm tên sản phẩm trùng sau khi chuẩn hóa****TÌNH HUỐNG**

Câu 10 đã bắt trùng chính xác theo tên và giá. Câu này đi sâu hơn: chuẩn hóa tên về chữ thường và cắt khoảng trắng trước khi so sánh. Bắt được cả các kiểu trùng tinh vi hơn mà UNIQUE constraint case-insensitive chưa chặn được.

Bảng Products — dòng đỏ: tên trùng sau khi LOWER + TRIM:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT LOWER(TRIM(product_name)) AS ten_chuan,
       COUNT(*)                  AS so_ban_ghi
FROM   Products
GROUP BY LOWER(TRIM(product_name))
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
GROUP BY LOWER(TRIM(product_name))	LOWER chuyển về chữ thường, TRIM cắt khoảng trắng hai đầu. Nhờ vậy 'Ban phim co Logitech' và ' ban phim co logitech ' rơi vào cùng một nhóm.
HAVING COUNT(*) > 1	Chỉ giữ nhóm có nhiều hơn 1 bản ghi — tên bị trùng sau chuẩn hóa.

Giải thích tổng thể

Kỹ thuật **LOWER + TRIM + GROUP BY** là cách chuẩn hóa trước khi so sánh — đã dùng ở Câu 8 (email), áp dụng tương tự cho product_name.

PROD_002 và PROD_005 đều là 'Ban phim co Logitech' → sau LOWER+TRIM cho 'ban phim co logitech' → COUNT = 2.

Câu 10 đã bắt cặp này theo (tên + giá), câu này bắt lại chỉ theo tên — phát hiện trùng ngay cả khi giá đã bị sửa khác nhau.

Kết quả sau khi query (minh họa)

ten_chuan	so_ban_ghi
ban phim co logitech	2

'ban phim co logitech' xuất hiện 2 lần (PROD_002 + PROD_005). Chạy thêm SELECT * FROM Products WHERE LOWER(TRIM(product_name)) = 'ban phim co logitech' để xem chi tiết cả hai dòng.

GÓC SOI LỖI CỦA TESTER

Kết quả câu này chỉ cho tên chuẩn hóa và số lần — không biết product_id nào.

Để xem đầy đủ thông tin hai sản phẩm trùng, kết hợp với subquery:

```
SELECT * FROM Products
WHERE LOWER(TRIM(product_name)) IN
(SELECT LOWER(TRIM(product_name)) FROM Products
GROUP BY LOWER(TRIM(product_name)) HAVING COUNT(*) > 1)
```

36 Phát hiện dữ liệu test/demo còn sót trong dữ liệu thật**TÌNH HUỐNG**

Trong lúc dựng môi trường hoặc thử nghiệm tính năng, dev/QA hay tạo tài khoản và bản ghi có tên gợi ý rõ ràng là test ('Test', 'Demo', 'Fake'...). Vấn đề là những bản ghi này đôi khi **quên dọn** trước khi lên production — làm sai lệch số liệu báo cáo, hoặc tệ hơn, biến thành lỗ hổng nếu tài khoản test có quyền cao.

Bảng Customers — dòng đỏ: tên gợi ý dữ liệu test/demo:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE

customer_id	customer_name	email	status
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  LOWER(customer_name) LIKE '%test%'
       OR LOWER(customer_name) LIKE '%demo%'
       OR LOWER(customer_name) LIKE '%fake%'
       OR LOWER(customer_name) LIKE '%ao bug%'
       OR LOWER(IFNULL(email, '')) LIKE '%test%'
       OR LOWER(IFNULL(email, '')) LIKE '%demo%';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	Duyệt bảng Customers — nơi có thể lẫn tài khoản test/demo giữa khách hàng thật.
WHERE LOWER(customer_name) LIKE '%test%'	LOWER chuẩn hóa trước khi so khớp — bắt được cả 'Test', 'TEST', 'test'. %...% tìm chuỗi con ở bất kỳ vị trí nào trong tên.
LOWER(IFNULL(email, '')) LIKE '%test%'	IFNULL(email, '') thay NULL bằng chuỗi rỗng trước khi LOWER — tránh LOWER(NULL) trả về NULL khiến LIKE luôn UNKNOWN và bỏ sót dòng có email NULL.
OR ... OR ...	Mỗi từ khóa nghi vấn ('test', 'demo', 'fake'...) là một nhánh OR riêng. Riêng 'ao bug' là từ khóa đặt riêng để bắt đúng dòng mẫu 'Khach Hang Ao Bug' trong DB thực hành — hệ thống thật cần thay bằng quy ước đặt tên test data của chính team bạn.

Giải thích tổng thể

Kỹ thuật **LIKE '%từ_khóa%'** trên nhiều cột — khác hẳn các câu trùng lặp/ENUM trước đó vì đây là tìm kiếm theo **ngữ nghĩa từ khóa**, không phải so khớp giá trị chính xác.
Bắt được 2 bản ghi: C004 'Khach Hang Ao Bug' (tên tự đặt là dữ liệu giả) và C010 'Khach Test VIP' (có chữ 'Test' rõ ràng) — cả hai đều là tài khoản có dấu hiệu rõ ràng của dữ liệu test.
Trên hệ thống thật, danh sách từ khóa cần mở rộng theo quy ước nội bộ: 'qa_', 'sample_', tên miền nội bộ dùng để test (vd '@company-test.com')...

Kết quả sau khi query (minh họa)

customer_id	customer_name	email
C004	Khach Hang Ao Bug	trung_email@email.com
C010	Khach Test VIP	vip@email.com

2 bản ghi nghi là dữ liệu test/demo. Trước khi xóa: xác nhận với team xem có phải seed data cố ý giữ lại cho mục đích nào đó không — đừng xóa chỉ vì tên 'nghe giống' test.

GÓC SOI LỖI CỦA TESTER

LIKE theo từ khóa có hai rủi ro cần lường trước:

(1) **Dương tính giả (false positive)**: khách hàng thật tên trùng từ khóa (vd công ty tên 'Testco') sẽ bị bắt nhầm — luôn xác nhận thủ công trước khi xóa hàng loạt.

(2) **Âm tính giả (false negative)**: dữ liệu test đặt tên khéo léo không chứa từ khóa nào sẽ lọt qua — đây là giới hạn cố hữu của tìm kiếm theo từ khóa, không phải lỗi kỹ thuật.

Phòng ngừa tận gốc: thêm cột **is_test_data BOOLEAN** để đánh dấu tường minh ngay từ lúc tạo, thay vì suy luận ngược từ tên gọi.

37 Dò bản ghi gần-trùng bằng SOUNDEX (lỗi gõ chính tả)**TÌNH HUỐNG**

Câu 8 và Câu 35 bắt trùng nhờ chuẩn hóa hoa/thường và khoảng trắng — nhưng cả hai đều cần chuỗi gốc **giống hệt nhau** sau khi chuẩn hóa. Lỗi gõ chính tả thật ('Nguyen Van A' gõ thành 'Nguyen Van Ah' hay đọc sai dấu) tạo ra chuỗi khác hẳn về mặt ký tự dù phát âm gần như nhau — LOWER/TRIM không bắt được loại trùng này.

Bảng Customers — dòng đồ: hai tên phát âm giống nhau (SOUNDEX trùng):

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT a.customer_id AS id_a,
       a.customer_name AS ten_a,
       b.customer_id AS id_b,
       b.customer_name AS ten_b
FROM   Customers a
JOIN   Customers b
      ON SOUNDEX(a.customer_name) = SOUNDEX(b.customer_name)
      AND a.customer_id < b.customer_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers a JOIN Customers b	Self-join — bảng Customers tự ghép với chính nó: hình dung có hai bản sao cùng bảng, đặt tên a và b , rồi xét từng cặp (một dòng ở a với một dòng ở b) xem tên có nghe giống nhau không. Lưu ý: điều kiện ghép bọc trong hàm SOUNDEX() nên MySQL không dùng được index — phải so mọi dòng với mọi dòng, rất nặng trên bảng lớn (xem Góc soi lỗi).
ON SOUNDEX(a.customer_name) = SOUNDEX(b.customer_name)	Điều kiện ghép cặp: SOUNDEX chuyển mỗi tên thành một mã phát âm — bắt đầu bằng chữ cái đầu, tiếp theo là các số mã hóa phụ âm (bỏ nguyên âm và ký tự không phải chữ). Hai tên đọc gần giống sẽ ra cùng mã và được ghép thành một cặp. Ví dụ: 'Nguyen Van A' và 'Nguyen Van A (2)' đều cho mã N2515 (SOUNDEX bỏ qua phần ' (2)') → ghép thành cặp nghi trùng.
AND a.customer_id < b.customer_id	Điều kiện chống đếm trùng cặp. Nếu bỏ dòng này: • mỗi cặp hiện 2 lần — cả (a ghép b) lẫn (b ghép a); • mỗi dòng còn tự khớp với chính nó (tên luôn trùng mã với chính nó). Điều kiện a.customer_id < b.customer_id chỉ giữ lại một chiều, loại cả hai trường hợp thừa trên.

Giải thích tổng thể

Điểm cốt lõi: câu này đổi **thước đo** so trùng từ **ký tự** sang **âm đọc**. Các cách trước cần hai chuỗi giống hệt nhau sau khi chuẩn hóa; còn ở đây, chỉ cần đọc lên nghe gần giống là bị ghép cặp — nhờ vậy bắt được loại trùng do lỗi đánh máy mà cách so ký tự bỏ sót.

Kết quả sau khi query (minh họa)

id_a	ten_a	id_b	ten_b
C001	Nguyen Van A	C009	Nguyen Van A (2)

1 cặp phát âm giống nhau. Đây là cùng cặp Câu 31 đã phát hiện qua dấu hiệu khác (ký tự số/ngoặc trong tên) — SOUNDEX hữu ích nhất khi lỗi gõ không để lại dấu hiệu ký tự rõ ràng như vậy, ví dụ 'Nguyen Van A' bị gõ nhầm thành 'Nguyenn Van A'.

GÓC SOI LỖI CỦA TESTER

Vài hạn chế cần biết trước khi dùng cách dò theo âm này:

- (1) **Kém với tiếng Việt có dấu**: SOUNDEX vốn cho tiếng Anh, nên bỏ dấu ('Nguyễn' → 'Nguyen') trước khi so để tăng độ chính xác.
- (2) **Dễ báo nhầm với tên ngắn**: nhiều tên ngắn khác nghĩa vẫn ra cùng mã → luôn xác nhận bằng mắt trước khi xử lý, tuyệt đối không tự động xóa hay gộp.
- (3) **Cảnh báo hiệu năng — đừng chạy nguyên câu này trên bảng lớn**: nó ghép mọi dòng với mọi dòng, lại bọc hàm nên không xài được index; số khách càng nhiều thì chi phí phình theo cấp số nhân (khoảng $N \times N$). 10 dòng demo thì tức thì, nhưng bảng khách thật hàng trăm nghìn dòng có thể chạy hàng giờ hoặc làm nghẽn DB. Cách chạy an toàn: lọc trước bằng **WHERE** để thu hẹp phạm vi (theo khu vực, theo ngày tạo gần đây...); hoặc thêm cột **soundex_name** tính sẵn + đánh index rồi so trên cột đó (hết bọc hàm); và luôn chạy trên bản sao read-only (xem chương 'Chạy SQL an toàn trên production').

38

Phát hiện đơn có tổng tiền nhưng không có sản phẩm nào

TÌNH HUỐNG

Câu 16 đã tìm đơn rỗng (không có items). Câu này thêm điều kiện: đơn có **total_amount > 0** nhưng lại không có item nào — nghĩa là hệ thống đã thu tiền (hoặc ghi nhận số tiền) cho một đơn không có sản phẩm cụ thể. Đây là mâu thuẫn dữ liệu nghiêm trọng.

Bảng Orders — dòng đỏ: ORD_004 ghi 5M nhưng Order_Items không có dòng nào:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.status
FROM   Orders o
LEFT JOIN Order_Items oi
      ON o.order_id = oi.order_id
WHERE  o.total_amount > 0
      AND oi.order_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

```
FROM Orders o
LEFT JOIN Order_Items oi
      ON o.order_id = oi.order_id
```

LEFT JOIN giữ lại **tất cả** đơn, kể cả đơn không có dòng item nào. Với đơn có items, mỗi item ghép thành một dòng; với đơn rỗng, phần cột của Order_Items bị bỏ trống — tức **oi.order_id = NULL**.

Ví dụ ORD_004 không có item nào → sau LEFT JOIN, dòng ORD_004 vẫn còn nhưng oi.order_id để trống. (Nếu dùng INNER JOIN, ORD_004 sẽ biến mất — không bắt được đơn rỗng.)

```
WHERE o.total_amount > 0
      AND oi.order_id IS NULL
```

Hai điều kiện phải thỏa cùng lúc:

- **total_amount > 0**: đơn có ghi nhận số tiền.
 - **oi.order_id IS NULL**: nhưng không có item nào — chính là dấu hiệu đơn rỗng lộ ra ở bước LEFT JOIN trên.
- Ghép lại: đơn **có tiền mà không có sản phẩm** — một mâu thuẫn nghiêm trọng.

```
SELECT o.order_id, o.customer_id,
       o.total_amount, o.status
```

Chiếu đủ thông tin để QA điều tra tiếp: đơn của khách nào, ghi bao nhiêu tiền, đang ở trạng thái gì.

Giải thích tổng thể

Vì sao đơn rỗng-có-tiền xếp mức nghiêm trọng hơn đơn rỗng thường? Vì hệ thống đang ghi nhận một khoản tiền mà **không có dòng hàng nào chống lưng** để đối chiếu — không cách nào biết 5M đó đúng hay sai, từ đâu ra. Số tiền 'lơ lửng' như vậy là rủi ro toàn vẹn tài chính, không chỉ là bản ghi thừa.

Một bài học soi lỗi đi kèm: **dữ liệu bản thật thường trượt nhiều phép kiểm cùng lúc**. ORD_004 vừa rỗng-có-tiền (câu này), vừa gán customer_id C999 không tồn tại (Câu 7) — hai lỗi độc lập chồng lên một dòng. Nên khi bắt được một đơn kiểu này, đừng dừng ở một phát hiện: soi luôn các dấu hiệu lân cận (khách có thật không, trạng thái có hợp lý không).

Kết quả sau khi query (minh họa)

order_id	customer_id	total_amount	status
ORD_004	C999	5.000.000	PENDING

ORD_004: 5M nhưng không có sản phẩm nào. Thêm vào đó: C999 không tồn tại. Đây là đơn hàng 'ma' — cần điều tra log xem được tạo như thế nào.

GÓC SOI LỖI CỦA TESTER

(1) **Cạm bẫy khi dùng LEFT JOIN để tìm đơn không có bản ghi con**: điều kiện lọc bảng con phải đặt **đúng chỗ**. `oi.order_id IS NULL` phải nằm ở **WHERE** (lọc sau khi đã LEFT JOIN); nếu vô ý chuyển nó lên **ON** thì không đơn nào bị loại và kết quả trả về sai bét. Và phải kiểm `IS NULL` trên **cột khóa** của bảng con (`oi.order_id`) — chọn nhầm một cột vốn cho phép NULL thì kết quả không còn tin được.

(2) **Hỏi trước khi kết luận là bug**: đơn có tiền mà không item thường do bước tạo đơn và bước thêm item **không nằm chung một transaction** — tạo đơn xong nhưng thêm item thất bại giữa chừng, để lại cái vỏ có tiền mà rỗng ruột. Cũng có thể là đơn điều chỉnh/hoàn phí cố ý không gán sản phẩm. Biết nhóm nguyên nhân giúp hỏi đúng người (dev hay vận hành).

39

Phát hiện tổng items vượt quá 1.5 lần total_amount

TÌNH HUỐNG

Câu 13 bắt mọi đơn có tổng items khác total_amount dù chỉ 1 đồng. Câu này dùng ngưỡng $1.5\times$ để bắt những trường hợp **lệch bất thường lớn**: tổng items cao hơn total_amount gấp rưỡi là dấu hiệu dữ liệu sai ở mức độ không thể do làm tròn hay phí vận chuyển.

Bảng Orders — dòng đồ: total_amount bất thường so với tổng items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount,
       SUM(oi.quantity * oi.price) AS tinh_tu_items
FROM   Orders o
JOIN   Order_Items oi
      ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total_amount
HAVING SUM(oi.quantity * oi.price)
       > o.total_amount * 1.5;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Order_Items oi ON o.order_id = oi.order_id	INNER JOIN ghép đơn hàng với items. Đơn không có item (ORD_004) bị loại — không có items thì không có tổng để so sánh. Nếu cần phát hiện cả đơn rỗng, dùng LEFT JOIN và lọc WHERE oi.item_id IS NULL.
GROUP BY o.order_id, o.total_amount	Gom tất cả items của cùng một đơn để tính tổng.
HAVING SUM(oi.quantity * oi.price) > o.total_amount * 1.5	HAVING so sánh sau aggregate: tổng items > 1.5 lần total_amount là bất thường. Ngưỡng 1.5× để bỏ qua chênh lệch nhỏ do phí vận chuyển, discount.

Giải thích tổng thể

Hai mức độ kiểm tra khác nhau: Câu 13 bắt **mọi** chênh lệch dù nhỏ, câu này chỉ bắt khi tổng items vượt **1.5 lần** total_amount.

Vì sao là 1.5×? Tiền hàng và total_amount thường lệch nhau chút ít vì lý do hợp lệ — phí vận chuyển, giảm giá, làm tròn — nhưng các khác biệt này hiếm khi vượt vài chục phần trăm. Đặt ngưỡng ở mức +50% là để bỏ qua vùng 'nhiều' đó: đã vượt 1.5× thì gần như chắc chắn là lỗi dữ liệu thật, không phải phí hay khuyến mãi. Đây là tham số nghiệp vụ — mỗi hệ thống nên tự chỉnh (các mức thay thế xem ở Góc soi lỗi).

ORD_001: items 62M > 48M (= 32M × 1.5) → bị bắt (do item trùng).

ORD_002: items 31M > 20M (= 20M × 1.5), tỷ lệ ≈ 1,55× — chỉ **vừa đủ** vượt ngưỡng → bị bắt (do Bug-B: total_amount ghi thấp). Nếu nâng ngưỡng lên 2× thì cả ORD_001 (1,94×) lẫn ORD_002 (1,55×) đều lọt lưới — cho thấy chọn con số nào quyết định ranh giới 'bao nhiêu thì coi là bất thường'.

Kết quả sau khi query (minh họa)

order_id	total_amount	tinh_tu_items
ORD_001	32.000.000	62.000.000
ORD_002	20.000.000	31.000.000

2 đơn vượt ngưỡng 1.5×. ORD_001: 62M vs 32M (do item trùng). ORD_002: 31M vs 20M (do Bug-B). Hai nguyên nhân khác nhau, cùng câu phát hiện.

GÓC SOI LỖI CỦA TESTER

Điều chỉnh ngưỡng tùy ngữ cảnh nghiệp vụ:

(1) **> 1.5×**: bắt lệch lớn, bỏ qua discount/phí nhỏ (câu này).

(2) **!= (bất kỳ lệch)**: bắt tất cả sai lệch dù nhỏ (Câu 13).

(3) **> 2×**: chỉ bắt lệch nghiêm trọng nhất.

Với hệ thống có discount, nên kết hợp thêm điều kiện loại trừ đơn có coupon trước khi áp ngưỡng.

BÀI TẬP TỰ LUYỆN — PHẦN 4

Bài 4.1. Tìm các sản phẩm bị thiếu dữ liệu giá HOẶC tồn kho (giá trị NULL).

Gợi ý: `WHERE price IS NULL OR stock IS NULL` — KHÔNG dùng `'= NULL'`.

Bài 4.2. Tìm khách hàng có tên chứa chữ số.

Gợi ý: `REGEXP '[0-9]'` bắt mọi ký tự là chữ số trong tên.

Bài 4.3. Tìm các tên sản phẩm bị trùng sau khi chuẩn hóa (bỏ khoảng trắng + đưa về chữ thường).

Gợi ý: `GROUP BY LOWER(TRIM(product_name))` rồi `HAVING COUNT(*) > 1`.

→ Lời giải đầy đủ ở **Phụ lục B** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 5 — Audit, log và dấu vết

Cột thời gian và mối quan hệ giữa các bảng là nơi kể lại lịch sử dữ liệu. Khi chúng mâu thuẫn — đơn đã xóa mềm vẫn để lại item, đơn hủy mà còn xóa chưa bật, dãy ID đứt quãng không rõ nguyên nhân — đó là dấu hiệu của bug logic hoặc lỗi hỏng dữ liệu.

PHẠM VI

Câu hỏi cốt lõi: **Lịch sử dữ liệu có kể đúng câu chuyện không?** — Khi nhìn thời gian và trạng thái mâu thuẫn nhau, đó là dấu vết của bug logic hoặc luồng xử lý bị gián đoạn giữa chừng.

40

Truy vết item còn sót của đơn đã xóa mềm

TÌNH HUỐNG

Câu 12 đã phát hiện đơn ORD_005 bị **xóa mềm** (cột deleted_at có giá trị) nhưng vẫn nằm trong bảng Orders. Vấn đề chưa dừng ở đó: các dòng chi tiết của đơn này trong Order_Items (item 8, 9) **không hề bị dọn**. Khi báo cáo doanh thu hoặc tồn kho quên lọc đơn đã xóa mềm, những item này vẫn bị cộng vào.

Bảng Order_Items — dòng đỏ: item 8, 9 thuộc ORD_005 (đơn đã xóa mềm):

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

CÂU LỆNH SQL

```
SELECT oi.item_id,
       oi.order_id,
       oi.product_id,
       oi.quantity,
       oi.price
FROM   Order_Items oi
JOIN   Orders o
      ON oi.order_id = o.order_id
WHERE  o.deleted_at IS NOT NULL
ORDER BY oi.item_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Orders o ON oi.order_id = o.order_id	INNER JOIN ghép mỗi dòng item với đơn cha của nó (khớp theo order_id), để đọc được cột deleted_at — cột này nằm bên bảng Orders, không có trong Order_Items. Ví dụ item 8 mang order_id = ORD_005 → JOIN sang Orders lấy được trạng thái xóa mềm của đơn ORD_005.
WHERE o.deleted_at IS NOT NULL	Chỉ giữ lại item thuộc đơn đã bị xóa mềm — tức đơn cha có deleted_at khác NULL (đã đánh dấu xóa, lẽ ra không còn hiệu lực). Với dữ liệu mẫu chỉ ORD_005 có deleted_at, nên chỉ item 8 và 9 lọt qua; item của các đơn khác (deleted_at trống) đều bị loại.
ORDER BY oi.item_id	Sắp xếp theo item_id để dễ đối chiếu ngược lại với bảng Order_Items gốc.

Giải thích tổng thể

Câu 12 soi **bảng cha** (Orders) để tìm đơn bị xóa mềm; câu này đi tiếp xuống **bảng con** (Order_Items) tìm những dòng chi tiết bị bỏ lại.

Đây là mẫu kiểm tra **tính nhất quán khi xóa mềm**: đã xóa (mềm) đơn cha thì các dòng con cũng phải được đánh dấu hoặc loại khỏi mọi phép tính. Nếu không, dữ liệu con 'sống' lâu hơn dữ liệu cha, và mọi con số cộng từ bảng con đều lệch.

Điểm mấu chốt: bug ở đây không nằm ở một dòng sai giá trị, mà ở **thao tác xóa làm nửa vời** — dọn cha nhưng quên con.

Kết quả sau khi query (mình họa)

item_id	order_id	product_id	quantity	price
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000

2 item thuộc đơn đã xóa mềm vẫn nằm trong Order_Items. Mọi báo cáo tính từ Order_Items phải thêm điều kiện loại đơn có deleted_at, nếu không doanh thu và lượng bán sẽ bị thổi phồng.

GÓC SOI LỖI CỦA TESTER

(1) **Điểm mù của câu lệnh**: INNER JOIN chỉ bắt được item mà đơn cha **còn** trong bảng Orders (bị xóa mềm). Nếu đơn cha bị **xóa cứng** (xóa hẳn dòng khỏi Orders), item con thành mồ côi thật sự nhưng câu này **KHÔNG** thấy — vì INNER JOIN đã loại luôn dòng không tìm được cha. Muốn bắt cả trường hợp đó, đối chiếu order_id của item với danh sách đơn tồn tại bằng **LEFT JOIN ... WHERE o.order_id IS NULL**.

(2) **Hỏi dev về ý đồ**: xóa mềm một đơn CÓ nên tự động dọn (hoặc đánh dấu) item con không? Nếu có mà code chưa làm → bug ở luồng xóa; nên xóa cha và con trong **cùng một transaction** để không bỏ sót nửa vời.

41**Đối chiếu trạng thái hủy với cờ xóa mềm****TÌNH HUỐNG**

Hệ thống dùng hai dấu vết để đánh dấu một đơn không còn hiệu lực: cột **status = 'CANCELLED'** và cột **deleted_at** (thời điểm xóa mềm). Theo quy ước, hai cột này phải đi cùng nhau. Khi chúng lệch nhau — hủy nhưng chưa xóa mềm, hoặc xóa mềm nhưng status chưa đổi — quy trình đã bỏ sót một bước.

Bảng Orders — dòng đỏ: ORD_003 hủy nhưng deleted_at vẫn trống:

order_id	status	deleted_at
ORD_001	COMPLETED	(NULL)
ORD_002	COMPLETED	(NULL)
ORD_003	CANCELLED	(NULL)
ORD_004	PENDING	(NULL)
ORD_005	CANCELLED	2026-06-25 10:30:00

CÂU LỆNH SQL

```
SELECT order_id,
       status,
       deleted_at,
       CASE
         WHEN status = 'CANCELLED'
           AND deleted_at IS NULL
         THEN 'Hủy nhưng chưa xóa mềm'
         WHEN status <> 'CANCELLED'
           AND deleted_at IS NOT NULL
         THEN 'Xóa mềm nhưng status chưa CANCELLED'
       END AS van_de
FROM   Orders
WHERE  (status = 'CANCELLED' AND deleted_at IS NULL)
      OR (status <> 'CANCELLED' AND deleted_at IS NOT NULL)
ORDER BY order_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL bắt đầu từ đây: lấy toàn bộ đơn trong bảng Orders, rồi các bước sau mới lọc và gán nhãn trên đó.
WHERE (status = 'CANCELLED' AND deleted_at IS NULL) OR (status <> 'CANCELLED' AND deleted_at IS NOT NULL)	Bước lọc, chạy ngay sau FROM — chỉ giữ những đơn có hai cột dấu vết lệch nhau . Hai vế OR bắt hai chiều lệch: • status = 'CANCELLED' AND deleted_at IS NULL : đã hủy nhưng chưa xóa mềm (ví dụ ORD_003). • status <> 'CANCELLED' AND deleted_at IS NOT NULL : đã xóa mềm nhưng trạng thái chưa chuyển sang hủy. Đơn nào có hai cột khớp nhau (cùng đã hủy, hoặc cùng còn bình thường) đều không lọt qua bước này.
CASE WHEN ... THEN ... END AS van_de	Thuộc phần SELECT nên chạy sau WHERE, dù được viết ở đầu câu lệnh. Trên các đơn đã lọc, CASE dán cho mỗi đơn một nhãn mô tả đúng chiều lệch của nó vào cột van_de — một trong hai chuỗi: 'Hủy nhưng chưa xóa mềm' hoặc 'Xóa mềm nhưng status chưa CANCELLED'. Nhờ nhãn sẵn này, QA đọc kết quả là biết ngay đơn sai kiểu gì, khỏi phải tự đối chiếu.
ORDER BY order_id	Sắp xếp theo mã đơn cho dễ tra cứu.

Giải thích tổng thể

Khi một sự kiện nghiệp vụ được ghi ở **nhiều cột dấu vết**, các cột đó phải luôn đồng bộ. Câu này kiểm tra sự đồng bộ giữa status và deleted_at.

ORD_003 đã CANCELLED nhưng deleted_at vẫn trống — bước xóa mềm bị bỏ quên.

ORD_005 thì chuẩn: vừa CANCELLED vừa có deleted_at, nên không bị bắt.

Kết quả sau khi query (minh họa)

order_id	status	deleted_at	van_de
ORD_003	CANCELLED	(NULL)	Hủy nhưng chưa xóa mềm

ORD_003 hủy nhưng chưa được xóa mềm — nếu báo cáo lọc theo deleted_at, đơn này vẫn lọt vào như đơn còn hiệu lực. Cần đồng bộ lại hai cột.

GÓC SOI LỖI CỦA TESTER

(1) **Hỏi dev trước khi kết luận:** đơn CANCELLED mà deleted_at trống là **bug của một bước bị thiếu** trong luồng hủy đơn, hay quy trình cố ý tách riêng hai bước? Hai câu trả lời dẫn tới hai hướng xử lý khác nhau — nếu là cố ý, phải rà lại mọi báo cáo/màn hình đang lọc theo deleted_at xem có bỏ sót đơn đã hủy không.

(2) **Gốc rễ thường nằm ở code:** nên cập nhật status và deleted_at trong **cùng một transaction**, tránh cập nhật nửa vời (half-update — sửa được cột này nhưng cột kia chưa kịp) khiến hai cột lệch nhau.

42

Phát hiện đơn treo (PENDING) tồn đọng quá lâu

TÌNH HUỐNG

Một đơn ở trạng thái **PENDING** vài giờ là bình thường. Nhưng nếu nó treo nhiều ngày, rất có thể luồng thanh toán bị kẹt, job xử lý đã chết, hoặc đơn bị bỏ quên. Câu này đo số ngày tồn đọng tính đến một **mốc chốt sổ cố định** (2026-06-30) để kết quả ổn định, không đổi theo ngày chạy.

Bảng Orders — dòng đỏ: ORD_004 vẫn PENDING từ 2026-06-24:

order_id	customer_id	status	order_date
ORD_001	C001	COMPLETED	2026-06-20
ORD_002	C002	COMPLETED	2026-06-22
ORD_003	C003	CANCELLED	2026-06-23
ORD_004	C999	PENDING	2026-06-24
ORD_005	C001	CANCELLED	2026-06-25
ORD_006	C003	PENDING	2026-06-23
ORD_007	C001	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       status,
       order_date,
       DATEDIFF('2026-06-30', order_date)
       AS so_ngay_ton_dong
FROM   Orders
WHERE  status = 'PENDING'
      AND DATEDIFF('2026-06-30', order_date) > 3
ORDER BY so_ngay_ton_dong DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL bắt đầu từ đây: lấy toàn bộ đơn trong bảng Orders làm tập dữ liệu, rồi các bước sau mới lọc và tính toán trên đó.
WHERE status = 'PENDING' AND DATEDIFF('2026-06-30', order_date) > 3	Bước lọc, chạy ngay sau FROM — chỉ giữ đơn thỏa cả hai điều kiện: • status = 'PENDING': đơn đang treo, chưa xử lý xong. • DATEDIFF('2026-06-30', order_date) > 3: đã treo quá 3 ngày. Trong đó DATEDIFF(mốc, ngày_đặt) là số ngày từ ngày đặt đơn đến mốc chốt sổ 30-06. Ví dụ đơn đặt 24-06 → cách 30-06 là 6 ngày, lớn hơn 3 nên được giữ lại. Ngưỡng 3 ngày chỉ là ví dụ — tùy SLA (cam kết thời gian xử lý) của mỗi hệ thống. Lưu ý: DATEDIFF bọc quanh cột order_date khiến điều kiện này chậm trên bảng lớn (non-sargable) — chi tiết và cách viết nhanh hơn xem ở Góc soi lỗi.
DATEDIFF('2026-06-30', order_date) AS so_ngay_ton_dong	Thuộc phần SELECT nên chạy sau WHERE, dù được viết ở đầu câu lệnh. Nó lấy chính con số 'số ngày treo' vừa dùng để lọc và hiển thị thành một cột riêng (đặt tên so_ngay_ton_dong) để đọc và sắp xếp. Dùng mốc cố định 2026-06-30 (thay vì ngày hôm nay) để con số trong sách không đổi mỗi lần chạy.
ORDER BY so_ngay_ton_dong DESC	Sắp xếp giảm dần theo số ngày treo — đơn treo lâu nhất lên đầu, để ưu tiên xử lý trước.

Giải thích tổng thể

Câu này săn đơn **tồn đọng** — đơn có ngày đặt hợp lệ nhưng trạng thái mắc kẹt ở PENDING quá lâu. (Khác Câu 34: câu đó bắt ngày **vô lý** như ngày tương lai.)

Bộ lọc gồm hai điều kiện phải thỏa cùng lúc: trạng thái là **PENDING**, và số ngày treo (tính đến mốc 30-06) phải **lớn hơn 3**. Bảng dưới cho thấy câu lệnh quyết định giữ hay loại từng đơn thế nào:

Đơn	Trạng thái	Số ngày treo (đến 30-06)	Kết quả
ORD_006	PENDING	7 (> 3)	Giữ lại
ORD_004	PENDING	6 (> 3)	Giữ lại
ORD_001	COMPLETED	10	Loại — không PENDING
ORD_007	PENDING	-185 (đơn tương lai)	Loại — không quá 3 ngày

Kết quả sau khi query (minh họa)

order_id	customer_id	status	order_date	so_ngay_ton_dong
ORD_006	C003	PENDING	2026-06-23	7
ORD_004	C999	PENDING	2026-06-24	6

ORD_006 treo 7 ngày — đây cũng là đơn double order với ORD_003 (Câu 29). ORD_004 treo 6 ngày — customer_id C999 không tồn tại (Câu 7). Đơn treo lâu là nơi nên soi kỹ vì thường đi kèm nhiều lỗi khác.

GÓC SOI LỖI CỦA TESTER

(1) **Đừng quên mốc thời gian:** mốc 30-06 ở đây là cố định để ví dụ chạy ra kết quả ổn định. Nếu bê nguyên lên hệ thống thật mà quên đổi thành **CURDATE()**, câu lệnh sẽ đo theo một ngày đứng yên — 'tuổi đơn' tính ra ngày càng sai mà không báo lỗi gì. Khi giám sát thực tế, luôn đo theo ngày hiện tại.

(2) **Cạm bẫy hiệu năng (non-sargable):** bọc hàm quanh cột — **DATEDIFF(order_date) > 3** — khiến dù order_date có index, MySQL vẫn phải tính DATEDIFF cho **từng dòng** thay vì dùng index, nên chậm trên bảng lớn. Viết lại dạng dùng được index (cùng kết quả):

WHERE status = 'PENDING'

AND order_date < DATE_SUB(CURDATE(), INTERVAL 3 DAY);

43**Dựng dòng thời gian đơn hàng — khoảng cách giữa các đơn****TÌNH HUỐNG**

Mỗi đơn hàng đến vào một thời điểm, và khoảng cách giữa các đơn liên tiếp cho biết **nhịp đặt đơn** của hệ thống. Câu này xếp đơn theo thời gian rồi tính, với mỗi đơn, đã cách đơn liền trước bao nhiêu ngày — biến một danh sách đơn thành đường nhịp để nhìn ra chỗ bất thường. Có hai kiểu bất thường đáng soi: **khoảng lặng dài** (nhiều ngày liền không có đơn — có thể job tạo đơn hoặc API đã ngừng chạy mà không ai hay) và **cụm dày đặc** (nhiều đơn dồn trong tích tắc — nghi vấn bot đặt hàng loạt hoặc khách bấm gửi hai lần).

Bảng Orders — dòng thời gian 7 đơn theo order_date:

order_id	order_date
ORD_001	2026-06-20
ORD_002	2026-06-22
ORD_003	2026-06-23
ORD_004	2026-06-24
ORD_005	2026-06-25
ORD_006	2026-06-23
ORD_007	2027-01-01

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.order_date,
       DATEDIFF(
         o.order_date,
         (SELECT MAX(o2.order_date)
          FROM Orders o2
          WHERE o2.order_date < o.order_date)
       ) AS ngay_ke_tu_don_truoc
FROM Orders o
ORDER BY o.order_date;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o	MySQL bắt đầu từ đây: lấy toàn bộ đơn trong bảng Orders, gán alias o cho mỗi đơn đang xét. Bước này xác định 'sẽ duyệt trên những dòng nào' — chạy trước , rồi phần SELECT mới tính toán trên từng dòng đã chọn.
(SELECT MAX(o2.order_date) FROM Orders o2 WHERE o2.order_date < o.order_date)	Subquery tương quan — trái tim của câu lệnh. Nhiệm vụ của nó rất gọn: tìm ngày của đơn liền ngay trước đơn đang xét. Cách làm: với mỗi đơn (gọi là o), nó nhìn qua mọi đơn khác, chỉ giữ những đơn có ngày trước ngày của o, rồi lấy ngày mới nhất (MAX) trong số đó — đúng là đơn ngay trước o. Ví dụ đơn ORD_004 (24-06): các đơn trước nó rơi vào ngày 20, 22, 23, 23 → lấy MAX = 23-06. Hai điểm dễ vấp: • Dùng dấu < (ngày nhỏ hơn), không phải ≤ — để đơn không tự so với chính nó, và để hai đơn cùng ngày cùng nhìn lùi về ngày khác gần nhất. • Gọi là 'tương quan' (correlated) vì bên trong có nhắc o.order_date của truy vấn ngoài. Do đó MySQL phải chạy lại subquery này cho từng đơn (EXPLAIN gắn nhãn dependent) — chính là lý do câu chậm khi bảng lớn (xem Góc soi lỗi).
DATEDIFF(o.order_date, ...) AS ngay_ke_tu_don_truoc	DATEDIFF(a, b) trả về số ngày của a trừ b. Ở đây a = ngày đơn hiện tại, b = ngày đơn liền trước (do subquery trả về) → kết quả là 'đơn này cách đơn trước bao nhiêu ngày'. Ví dụ ORD_004: DATEDIFF(24-06, 23-06) = 1. Riêng đơn đầu tiên (ORD_001) không có đơn nào trước → subquery trả NULL → DATEDIFF(ngày, NULL) = NULL, nên cột này hiển thị trống.
ORDER BY o.order_date	Sắp xếp kết quả hiển thị theo thời gian tăng dần, để đọc bảng như một dòng sự kiện từ cũ đến mới. Mệnh đề này không ảnh hưởng cách tính khoảng cách — bỏ đi thì các con số vẫn đúng, chỉ là thứ tự dòng xáo trộn nên khó đọc hơn.

Giải thích tổng thể

Cách đọc kết quả: cột 'số ngày cách đơn trước' chính là thước đo nhịp bán hàng. Với dữ liệu mẫu, các khoảng 1–2 ngày là đều đặn — bình thường; chỉ con số vọt lên bất thường (như 190 ngày) mới đáng điều tra. Nói ngắn gọn: bản thân con số không phải bug — **chỗ lệch khỏi nhịp thường ngày** mới là dấu hiệu cần soi.

Một điểm hay của cách viết này: nó dựng được dòng thời gian mà **không cần công cụ đặc biệt nào**, chạy được cả trên các bản MySQL cũ. Từ MySQL 8.0 trở lên có 'window function' (giới thiệu ở Câu 49) giúp viết kiểu bài này ngắn gọn hơn.

Kết quả sau khi query (minh họa)

order_id	order_date	ngay_ke_tu_don_truoc
ORD_001	2026-06-20	(NULL)
ORD_002	2026-06-22	2
ORD_003	2026-06-23	1
ORD_006	2026-06-23	1
ORD_004	2026-06-24	1
ORD_005	2026-06-25	1
ORD_007	2027-01-01	190

ORD_003 và ORD_006 cùng ngày 2026-06-23 — subquery lấy max ngày TRƯỚC 2026-06-23 là 2026-06-22, nên cả hai đều cho khoảng cách = 1. Hai đơn cùng ngày là dấu hiệu double-submit (xem Câu 29). ORD_007 (2027-01-01) có khoảng cách = 190 ngày — khoảng lặng cực dài, đây là đơn ngày tương lai (xem Câu 34). Đơn đầu tiên ORD_001 có khoảng cách NULL vì không có đơn nào trước.

GÓC SOI LỖI CỦA TESTER

(1) **Muốn tự động hoá:** thêm điều kiện lọc theo khoảng cách (ví dụ chỉ lấy đơn cách đơn trước hơn 7 ngày, hoặc bằng 0) để câu lệnh tự chỉ ra điểm bất thường, khỏi phải dò tay.

(2) **Cảnh báo hiệu năng:** subquery ở đây phải quét lại bảng cho từng đơn, nên số đơn càng nhiều thì chi phí phình rất nhanh (khoảng $N \times N$ — cùng kiểu rủi ro với Câu 37). Vài nghìn đơn vẫn ổn; nhưng bảng Orders thật hàng triệu dòng thì câu này có thể chạy rất lâu. Khi đó nên dùng **window function LAG()** (MySQL 8.0+; xem Câu 49) — chỉ quét bảng một lần.

(3) **Khác biệt cần nhớ nếu đổi sang LAG:** với hai đơn **trùng ngày**, LAG cho khoảng cách 0 (so đúng dòng liền trước), còn subquery ở đây (dùng dấu <) cho 1 (đếm lùi về ngày khác gần nhất) — chọn cách khớp với điều bạn muốn đo.

44 Phát hiện item_id bị nhảy — dấu vết của bản ghi bị xóa

TÌNH HUỐNG

Trên hệ thống thật, item_id thường là khóa chính tự tăng (AUTO_INCREMENT) — cấp số liên tục 1, 2, 3... Nếu dãy bị nhảy (có 1, 2, 4 nhưng thiếu 3), thường là dấu vết một bản ghi từng tồn tại rồi bị xóa. Đây là kỹ thuật audit đơn giản để phát hiện dữ liệu bị xóa mà không để lại log. (Trên DB mẫu, gap được tạo sẵn bằng INSERT tường minh để mô phỏng.)

Bảng Order_Items — thiếu item_id 3, 10 và 11 trong dãy 1→14:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000

item_id	order_id	product_id	quantity	price
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT a.item_id + 1      AS id_bi_mat
FROM   Order_Items a
LEFT JOIN Order_Items b
      ON b.item_id = a.item_id + 1
WHERE  b.item_id IS NULL
AND    a.item_id <
      (SELECT MAX(item_id)
       FROM Order_Items);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items a LEFT JOIN Order_Items b ON b.item_id = a.item_id + 1	Self-join: bảng tự join với chính nó. Alias a là bản ghi gốc, b là bản ghi tiếp theo liền kề. Nếu b.item_id = NULL → không có bản ghi nào có id = a.item_id + 1.
WHERE b.item_id IS NULL AND a.item_id < (SELECT MAX(item_id) FROM Order_Items)	Hai điều kiện kết hợp: tiếp theo bị thiếu (IS NULL) VÀ chưa phải bản ghi cuối (< MAX). Bỏ điều kiện MAX sẽ trả thêm id = MAX+1, không phải gap.
SELECT a.item_id + 1 AS id_bi_mat	Kết quả là id bị thiếu — tức là a.item_id + 1.

Giải thích tổng thể

Logic của câu: với mỗi item_id = n đang có, kiểm tra xem n+1 có tồn tại không. Nếu không có → n+1 chính là một số bị thiếu.

Ví dụ: có item_id=2 nhưng không có 3 → báo thiếu 3; có 9 nhưng không có 10 → báo thiếu 10. Riêng item_id=14 là số lớn nhất nên không xét tiếp.

Điểm mù cần biết: câu chỉ soi được số ngay sau một id ĐANG tồn tại. Số 11 cũng thiếu, nhưng vì 10 cũng không có trong bảng nên không có dòng nào để 'nhìn sang' 11 — thành ra 11 bị bỏ sót. Nói cách khác, với một khoảng thiếu liên tiếp, câu này chỉ báo số đầu tiên của khoảng.

Kết quả sau khi query (minh họa)

id_bi_mat
3
10

Query báo 2 số bị thiếu: item_id=3 (giữa 2 và 4) và item_id=10 (giữa 9 và 12). Đó là các vị trí từng có bản ghi rồi biến mất — cần đối chiếu audit log để biết vì sao mất.

GÓC SOI LỖI CỦA TESTER

Gap trong AUTO_INCREMENT không phải lúc nào cũng là bug:

- (1) **INSERT thất bại:** transaction rollback, nhưng AUTO_INCREMENT không rollback lại — tạo gap bình thường.
- (2) **Xóa dữ liệu:** hard DELETE không để lại dấu vết gì khác.
- (3) **Bulk insert lỗi giữa chừng:** một số ID đã được cấp nhưng bản ghi tương ứng không được lưu. Để biết nguyên nhân chính xác, cần có bảng audit log riêng.

45

Phân tích đơn hàng bị hủy — khách nào hay hủy nhất

TÌNH HUỐNG

Đơn bị hủy không chỉ là chuyện kinh doanh — còn là tín hiệu kỹ thuật. Nếu một khách cụ thể hủy nhiều đơn bất thường, có thể luồng thanh toán gặp lỗi riêng với profile đó, hoặc là dữ liệu test chưa dọn sau sprint. Câu này đếm số đơn hủy theo từng khách và xếp ai hủy nhiều nhất lên đầu.

Bảng Orders — dòng đỏ: ORD_003 có status = CANCELLED:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS so_don_huy
FROM   Orders o
JOIN   Customers c
       ON o.customer_id = c.customer_id
WHERE  o.status = 'CANCELLED'
GROUP BY c.customer_id, c.customer_name
ORDER BY so_don_huy DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id	INNER JOIN lấy tên khách cho mỗi đơn. Đơn mờ côi (customer_id không có trong Customers) sẽ bị loại — nhưng ở câu này không ảnh hưởng, vì đơn mờ côi ORD_004 vốn là PENDING nên đã bị mệnh đề WHERE lọc trước rồi.
WHERE o.status = 'CANCELLED'	Chỉ giữ đơn bị hủy trước khi gom nhóm.
GROUP BY c.customer_id, c.customer_name ORDER BY so_don_huy DESC	Gom các đơn hủy về từng khách rồi đếm (COUNT), sắp xếp giảm dần — khách hủy nhiều nhất lên đầu.

Giải thích tổng thể

Đây là mẫu **đếm-rồi-xếp-hạng trên một tập đã lọc**: lọc đúng loại đơn (CANCELLED), gom theo khách, đưa ai nhiều nhất lên đầu — dùng được cho mọi câu hỏi 'ai/cái gì nhiều nhất trong nhóm X'.

Nhớ rằng kết quả là **tín hiệu, không phải kết luận**: 1 đơn hủy chưa nói lên gì. Trên data mẫu ai cũng chỉ ~1 nên bảng này chưa có ý nghĩa — nó chỉ phát huy trên production đủ lớn, thường kèm **HAVING COUNT(*) > N** để bỏ nhiễu, chỉ giữ khách hủy nhiều đáng ngờ.

Kết quả sau khi query (minh họa)

customer_id	customer_name	so_don_huy
C003	Le Thi C	1
C001	Nguyen Van A	1

C001 và C003 mỗi khách hủy 1 đơn (C001 hủy ORD_005, C003 hủy ORD_003). Trên production, câu này giúp theo dõi và cảnh báo sớm khi số đơn hủy của một khách tăng đột biến.

GÓC SOI LỖI CỦA TESTER

'Số đơn hủy' khác 'tỷ lệ hủy'. Câu này đếm **số** đơn hủy — một khách hủy 1/1 đơn (100%) và một khách hủy 1/50 đơn (2%) đều hiện `so_don_huy = 1`. Muốn đo mức bất thường thật, phải tính tỷ lệ: số đơn hủy chia tổng đơn của khách đó.

Khi thấy khách hủy nhiều, phân biệt trước khi báo bug: hủy **tập trung một khoảng thời gian** → nghi release lỗi, soi luồng checkout đột đó; hủy **rải đều** → có thể là dữ liệu test chưa dọn hoặc lỗi UX mãn tính.

BÀI TẬP TỰ LUYỆN — PHẦN 5

Bài 5.1. Liệt kê các đơn đã bị xóa mềm (`deleted_at` có giá trị) kèm số ngày từ lúc đặt đến lúc xóa (`DATEDIFF`) — đơn 'sống' quá ngắn là dấu vết đáng soi.

Gợi ý: `WHERE deleted_at IS NOT NULL`; dùng `DATEDIFF(deleted_at, order_date)`.

Bài 5.2. Tìm các item vẫn còn trong `Order_Items` nhưng thuộc đơn hàng đã bị xóa mềm (`deleted_at IS NOT NULL`) — dấu vết dọn dẹp không triệt để.

Gợi ý: `JOIN Order_Items` với `Orders`, lọc `WHERE o.deleted_at IS NOT NULL`.

→ Lời giải đầy đủ ở **Phụ lục B** (cuối sách). Hãy tự viết trước khi xem.

PHẦN 6 — Truy vấn nâng cao cho QA

Năm câu lệnh cuối dùng tới window function, CTE và UNION. Chúng giải quyết những bài toán kiểm thử khó: lấy bản ghi mới nhất mỗi nhóm, phân hạng khách hàng, tổng hợp nhiều loại lỗi trong một báo cáo duy nhất.

PHẠM VI

Câu hỏi cốt lõi: **Cần kỹ thuật nào khi câu trả lời đòi hỏi nhiều bước tính toán?** — Window function, CTE và UNION giải những bài toán mà WHERE và GROUP BY đơn thuần không đủ: xếp hạng theo nhóm, chuỗi thời gian, tổng hợp nhiều loại lỗi cùng lúc.

46 Dùng ROW_NUMBER() phát hiện item trùng trong cùng một đơn

TÌNH HUỐNG

Câu 4 tìm được item trùng bằng GROUP BY, nhưng chỉ cho biết **nhóm nào** bị trùng — không chỉ ra **dòng nào** mới là bản thừa cần xóa. Câu này dùng **ROW_NUMBER()** để đánh số thứ tự cho từng dòng trong mỗi nhóm; dòng nào bị đánh số 2 trở lên chính là bản trùng.

ROW_NUMBER() thuộc nhóm **window function**: khác GROUP BY (gộp nhiều dòng thành một), nó giữ nguyên từng dòng và chỉ thêm một cột đánh số bên cạnh.

Bảng Order_Items — dòng đỏ: item 1 và item 7 cùng ORD_001/PROD_001:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000
8	ORD_005	PROD_004	1	1.000.000
9	ORD_005	PROD_002	1	2.000.000
12	ORD_003	PROD_002	0	1.500.000
13	ORD_006	PROD_003	1	8.000.000
14	ORD_007	PROD_004	20	1.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       ROW_NUMBER() OVER (
         PARTITION BY order_id, product_id
         ORDER BY item_id
       ) AS so_lan_trong_don
FROM   Order_Items
ORDER  BY order_id, product_id, item_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	Nạp toàn bộ bảng Order_Items — 11 dòng. Window function không lọc bớt dòng nào, giữ nguyên tất cả rồi thêm cột đánh số bên cạnh.
ROW_NUMBER() OVER (...) AS so_lan_trong_don	ROW_NUMBER() gán một số thứ tự tăng dần (1, 2, 3...) cho từng dòng. Phần OVER (...) khai báo 'cửa sổ' — phạm vi dòng mà hàm nhìn vào để đánh số. Nếu để OVER () trống thì cả bảng là một cửa sổ (đánh 1→11); ở đây ta chia nhỏ cửa sổ bằng PARTITION BY ngay dưới.
PARTITION BY order_id, product_id	Chia dữ liệu thành từng nhóm theo cặp (đơn + sản phẩm) — giống cách gom của GROUP BY nhưng KHÔNG gộp dòng . Mỗi khi sang một nhóm mới (cặp order_id/product_id khác), ROW_NUMBER đếm lại từ 1 . Nhờ vậy con số cho biết 'đây là lần xuất hiện thứ mấy của cặp này'.
ORDER BY item_id (bên trong OVER)	Trong mỗi nhóm, quyết định dòng nào được số 1: item_id nhỏ hơn xếp trước → nhận số 1 (coi là bản gốc), dòng sau nhận số 2 (bản trùng). Đây là ORDER BY của cửa sổ — đừng nhầm với ORDER BY sắp xếp kết quả ở cuối câu.
ORDER BY order_id, product_id, item_id (cuối câu)	ORDER BY này chỉ sắp xếp bảng kết quả cho dễ đọc — đưa các dòng cùng nhóm nằm liền nhau. Không ảnh hưởng gì đến số ROW_NUMBER đã tính ở trên.

Giải thích tổng thể

Ý tưởng cốt lõi: con số **so_lan_trong_don** cho biết mỗi dòng là **lần xuất hiện thứ mấy** của cặp (đơn + sản phẩm). Số **1** = bản đầu tiên (giữ lại); số **2 trở lên** = bản lặp (thừa).

Nhờ vậy việc dọn trùng trở nên an toàn: chỉ cần xóa các dòng có số > 1, giữ nguyên bản gốc số 1 — không sợ xóa nhầm cả cặp.

Đó là lợi thế so với Câu 4: Câu 4 chỉ nói 'cặp này bị trùng', còn câu này chỉ thẳng **dòng nào** phải xóa (item 7).

Kết quả sau khi query (minh họa)

item_id	order_id	product_id	so_lan_trong_don
1	ORD_001	PROD_001	1
7	ORD_001	PROD_001	2
2	ORD_001	PROD_002	1
4	ORD_002	PROD_001	1
5	ORD_002	PROD_004	1
12	ORD_003	PROD_002	1
6	ORD_003	PROD_003	1
9	ORD_005	PROD_002	1
8	ORD_005	PROD_004	1
13	ORD_006	PROD_003	1
14	ORD_007	PROD_004	1

Item_id=7 có so_lan_trong_don=2 — bản ghi trùng của item_id=1 trong ORD_001/PROD_001. 11 dòng tổng cộng; chỉ item 7 có số thứ tự > 1 → là bản ghi trùng duy nhất. Lọc WHERE so_lan_trong_don > 1 để chỉ xem bản trùng.

GÓC SOI LỖI CỦA TESTER

Trước khi xóa các dòng có số > 1, phải chốt 'giữ dòng nào'. ORDER BY trong OVER quyết định dòng nào là bản gốc được giữ — ở đây là item_id nhỏ nhất. Nhưng có khi bản mới hơn mới là bản đã sửa đúng; hỏi dev/PO quy tắc giữ bản nào trước khi DELETE.

Cảnh giác khi cột ORDER BY không duy nhất: nếu các dòng trùng có cùng giá trị ORDER BY, dòng nào nhận số 1 là ngẫu nhiên — dễ xóa nhầm dòng cần giữ. Luôn ORDER BY theo một cột xác định (id, thời gian tạo).

Ngoài bắt trùng, ROW_NUMBER() còn hay dùng để **lấy bản ghi mới nhất mỗi nhóm** (PARTITION BY nhóm, ORDER BY thời gian DESC, giữ dòng số 1) — mẫu rất thường gặp khi kiểm thử báo cáo.

Lưu ý: ROW_NUMBER() cần MySQL 8.0+ (kiểm tra: **SELECT VERSION();**).

47 Dùng CTE + RANK() xếp hạng sản phẩm bán chạy

TÌNH HUỐNG

Gần như báo cáo bán hàng nào cũng có bảng **'top sản phẩm bán chạy'** — xếp hạng sản phẩm theo doanh số từ cao xuống thấp. Câu này dựng đúng bảng đó qua hai bước: trước tiên tính tổng doanh số mỗi sản phẩm (dùng **CTE** đặt tên cho kết quả trung gian), rồi gán thứ hạng bằng **RANK()** — hàm này còn xử lý được cả khi hai sản phẩm bằng doanh số (cùng nhận một hạng). QA dựng lại bảng này để kiểm tra xếp hạng trên dashboard có đúng không, và quan trọng hơn: con số xếp hạng có bị dữ liệu bẩn thổi phồng không.

Bảng Products — PROD_001 xếp hạng 1 nhưng doanh số bị thổi phồng do item trùng:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30

product_id	product_name	category	price	stock
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
WITH doanh_so AS (  
  SELECT p.product_id,  
         p.product_name,  
         SUM(oi.quantity * oi.price)  
           AS tong_doanh_so  
  FROM   Products p  
  JOIN   Order_Items oi  
        ON p.product_id = oi.product_id  
  GROUP BY p.product_id, p.product_name  
)  
SELECT product_id,  
       product_name,  
       tong_doanh_so,  
       RANK() OVER  
         (ORDER BY tong_doanh_so DESC)  
         AS hang  
FROM   doanh_so;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

<pre>WITH doanh_so AS (SELECT product_id, product_name, SUM(quantity*price) AS tong_doanh_so FROM Products JOIN Order_Items GROUP BY product_id, product_name)</pre>	Cụm 1 — Dựng bảng tạm (CTE): khối WITH tạo một bảng tạm tên doanh_so — mỗi sản phẩm một dòng kèm tổng doanh số (giống Câu 22). Đặt tên để dùng lại ở cụm sau.
<pre>SELECT ..., RANK() OVER (ORDER BY tong_doanh_so DESC) AS hang FROM doanh_so</pre>	Cụm 2 — Truy vấn trên bảng tạm: đọc từ doanh_so và thêm cột thứ hạng bằng RANK() . RANK khác ROW_NUMBER: nếu hai sản phẩm bằng doanh số, cả hai cùng nhận một hạng và hạng kế bị bỏ qua (1, 1, 3...).

Giải thích tổng thể

Kết quả xếp hạng đúng về kỹ thuật, nhưng **bảng xếp hạng chỉ đáng tin khi dữ liệu nền sạch**:
PROD_001 đứng hạng 1 với 90M — nhưng con số này bị thổi phồng do item trùng (Câu 4/46), doanh số thực chỉ khoảng 60M.
PROD_004 vọt lên hạng 2 với 22M do item 14 có quantity=20 — cần kiểm tra là đơn thật hay dữ liệu test chưa dọn.
Bài học QA: một 'bảng bán chạy' trông chuyên nghiệp vẫn có thể sai nếu chưa đối soát dữ liệu gốc.

Kết quả sau khi query (minh họa)

product_id	product_name	tong_doanh_so	hang
PROD_001	iPhone 15 Pro Max	90.000.000	1
PROD_004	Sac du phong Anker	22.000.000	2
PROD_003	Tai nghe Sony WH-1000XM5	16.000.000	3
PROD_002	Ban phim co Logitech	4.000.000	4

4 sản phẩm có doanh số (PROD_005, 006, 007 chưa bán). PROD_001 dẫn đầu 90M (bị phóng đại do item 7 trùng). PROD_004 hạng 2 với 22M (do item 14 qty=20). PROD_003 hạng 3 với 16M. Xếp hạng này chỉ đáng tin sau khi làm sạch dữ liệu.

GÓC SOI LỖI CỦA TESTER

Bảng xếp hạng dễ 'trông đúng mà sai': thứ tự có thể chuẩn nhưng con số lại bị dữ liệu bẩn thổi phồng — luôn soi cả con số, đừng chỉ soi thứ hạng.

Khi hai sản phẩm bằng doanh số, xác nhận với spec: báo cáo muốn **RANK()** (đồng hạng, bỏ hạng kế) hay **ROW_NUMBER()** (số liên tiếp, không đồng hạng)? Nhìn giao diện không suy ra được — phải hỏi.

48

Dùng lại một CTE nhiều lần: tìm khách chi tiêu trên mức trung bình

TÌNH HUỐNG

Nhiều hệ thống cần lọc ra nhóm **khách chi tiêu cao hơn mặt bằng** — để chăm sóc VIP, mời ưu đãi, phân tích khách giá trị. 'Cao hơn mặt bằng' nghĩa là chi nhiều hơn **mức trung bình của tất cả khách**. Câu này làm ba việc: tính tổng chi mỗi khách, lấy trung bình các con số đó, rồi giữ lại ai vượt trung bình.

Vì mức trung bình phải tính từ chính danh sách vừa dựng, ta đặt tên danh sách đó bằng **CTE** (bảng tạm) để **dùng lại hai lần** — một lần lấy danh sách, một lần tính trung bình — thay vì viết lại phép tính tổng hai lần.

Bảng Customers — C001 là khách duy nhất chi tiêu trên mức trung bình COMPLETED:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
WITH tong_chi AS (  
  SELECT c.customer_id,  
         c.customer_name,  
         SUM(o.total_amount) AS tong_da_mua  
  FROM   Customers c  
  JOIN   Orders o  
        ON c.customer_id = o.customer_id  
  WHERE  o.status = 'COMPLETED'  
  GROUP  BY c.customer_id, c.customer_name  
)  
SELECT customer_id,  
       customer_name,  
       tong_da_mua  
FROM   tong_chi  
WHERE  tong_da_mua >  
       (SELECT AVG(tong_da_mua)  
        FROM   tong_chi)  
ORDER  BY tong_da_mua DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

<pre>WITH tong_chi AS (SELECT customer_id, SUM(total_amount) AS tong_da_mua ... WHERE status='COMPLETED' GROUP BY customer_id)</pre>	Cụm 1 — Dựng bảng tạm (CTE): khối WITH tạo một bảng tạm tên tong_chi — mỗi khách một dòng kèm tổng chi tiêu, chỉ tính đơn COMPLETED (bỏ CANCELLED, PENDING). Xong cụm này, coi tong_chi như một bảng thật để dùng tiếp.
<pre>SELECT ... FROM tong_chi -- lần 1 WHERE tong_da_mua > (SELECT AVG(tong_da_mua) FROM tong_chi) -- lần 2</pre>	Cụm 2 — Truy vấn trên bảng tạm: ở đây tong_chi được gọi tên hai lần : • Lần 1 (FROM tong_chi ở câu chính): lấy danh sách khách kèm tổng chi. • Lần 2 (FROM tong_chi trong subquery): tính mức trung bình để làm ngưỡng so sánh. Câu chỉ giữ khách có tổng chi lớn hơn mức trung bình đó. Định nghĩa CTE một lần, dùng lại hai lần — khỏi phải chép lại khối GROUP BY.
<pre>ORDER BY tong_da_mua DESC</pre>	Sắp xếp kết quả giảm dần — khách chi nhiều nhất lên đầu.

Giải thích tổng thể

Câu này lọc theo một **ngưỡng động**: mức trung bình được tính ngay từ chính tập dữ liệu, không phải con số cố định gõ tay — nên khi dữ liệu đổi, ngưỡng tự đổi theo.
Trong data mẫu (chỉ tính đơn COMPLETED): C001 = 32M, C002 = 20M → trung bình = 26M → chỉ C001 (32M) vượt, C002 (20M) rớt.
Mẫu 'so với trung bình của chính tập' là nền của phân tích outlier — tìm cái bất thường so với mặt bằng chung.

Kết quả sau khi query (minh họa)

customer_id	customer_name	tong_da_mua
C001	Nguyen Van A	32.000.000

Chỉ C001 (32M) vượt mức trung bình 26M. C002 có 20M < 26M nên bị loại. Đây là khách VIP tiềm năng — dữ liệu test nên cover luồng upgrade membership.

GÓC SOI LỖI CỦA TESTER

'Trên mức trung bình' — trung bình của cái gì? Câu này lấy trung bình của **tổng chi từng khách** (gộp về mỗi khách một số rồi mới tính trung bình), khác với trung bình của **từng đơn**. Hai cách cho hai ngưỡng khác nhau — xác nhận với PO đang cần loại nào.

Ngưỡng này còn 'động': thêm/bớt khách là trung bình đổi theo, nên một khách 'trên trung bình' hôm nay có thể rớt xuống sau — đừng chốt danh sách VIP cứng từ một lần chạy.

49 Tính doanh thu tích lũy theo thời gian với SUM() OVER()**TÌNH HUỐNG**

Nhiều báo cáo hiện con số **lũy kế** — 'tính đến thời điểm này đã cộng được tổng bao nhiêu'. Giống cột số dư trên sao kê ngân hàng: mỗi dòng không phải số của riêng nó, mà là tổng dồn tất cả các dòng từ đầu đến đó (ngày 1 bán 32M → lũy kế 32M; ngày 2 bán 20M → lũy kế 52M; cứ thế). QA gặp con số lũy kế này trên dashboard 'doanh thu từ đầu tháng', biểu đồ tăng trưởng... và cần tự tính lại bằng SQL để đối chiếu — lịch là có bug. **SUM() OVER()** là công cụ để ra cột lũy kế đó chỉ trong một câu lệnh.

Bảng Orders — doanh thu tích lũy tính theo order_date tăng dần:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25
ORD_006	C003	8.000.000	PENDING	2026-06-23
ORD_007	C001	20.000.000	PENDING	2027-01-01

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       order_date,
       total_amount,
       SUM(total_amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING
                AND CURRENT ROW
       ) AS luy_ke
FROM   Orders
ORDER BY order_date;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders — tất cả 7 đơn, kể cả CANCELLED và PENDING.
SUM(total_amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS luy_ke	SUM() OVER() là kiểu 'tổng chạy dồn': với mỗi đơn, cộng total_amount của nó với mọi đơn đứng trước — khác SUM() thường vốn gộp tất cả thành một con số duy nhất. ORDER BY order_date (đặt bên trong OVER) quyết định thứ tự cộng dồn: theo ngày, cũ trước mới sau. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW là phạm vi cộng — từ dòng đầu tiên (<i>UNBOUNDED PRECEDING</i> = không giới hạn về trước) đến dòng đang xét (<i>CURRENT ROW</i>). Có thể viết gọn OVER (ORDER BY order_date), cùng kết quả khi mỗi ngày chỉ có một đơn. Nhưng khi nhiều đơn cùng ngày thì hai kiểu khác nhau: • RANGE (bản gọn ngầm dùng): các đơn cùng ngày bị gộp chung một mốc tích lũy. • ROWS (viết tường minh như trên): cộng lần lượt từng dòng, mỗi đơn một mốc riêng.
ORDER BY order_date	Sắp xếp kết quả cuối cùng theo ngày — đảm bảo luy_ke tăng dần theo thời gian khi đọc từ trên xuống.

Giải thích tổng thể

SUM() OVER() tính tổng tích lũy mà không gộp dòng — khác GROUP BY (gộp tất cả thành một con số): window function vừa giữ chi tiết đủ 7 đơn, vừa thêm cột tổng dồn. Cách cột luy_ke hình thành — mỗi dòng = số của chính nó cộng lũy kế của dòng ngay trên:

Đơn (theo ngày)	Bán trong đơn	Lũy kế = trước + nay
ORD_001	32M	32M
ORD_002	20M	52M (32 + 20)
ORD_003	8M	60M (52 + 8)
ORD_006	8M	68M (60 + 8)
ORD_004	5M	73M (68 + 5)
ORD_005	15M	88M (73 + 15)
ORD_007	20M	108M (88 + 20)

Kết quả sau khi query (minh họa)

order_id	customer_id	order_date	total_amount	luy_ke
ORD_001	C001	2026-06-20	32.000.000	32.000.000
ORD_002	C002	2026-06-22	20.000.000	52.000.000
ORD_003	C003	2026-06-23	8.000.000	60.000.000
ORD_006	C003	2026-06-23	8.000.000	68.000.000
ORD_004	C999	2026-06-24	5.000.000	73.000.000
ORD_005	C001	2026-06-25	15.000.000	88.000.000
ORD_007	C001	2027-01-01	20.000.000	108.000.000

luy_ke = 108M nhưng đây là tổng thô gồm cả CANCELLED (ORD_003/8M, ORD_005/15M), PENDING bất thường (ORD_006 double order/8M, ORD_007 ngày tương lai/20M), và orphan (ORD_004/5M). Doanh thu thực COMPLETED: chỉ 52M.

GÓC SOI LỖI CỦA TESTER

Khi kiểm thử báo cáo tích lũy, hỏi spec rõ ngay từ đầu: cộng dồn theo **tất cả đơn** hay chỉ đơn đã hoàn tất? Nếu chỉ tính đơn hoàn tất, thêm **WHERE status = 'COMPLETED'** trước khi cộng dồn — nếu không, tổng dồn sẽ gồm cả đơn hủy và đơn treo, làm sai báo cáo.

Với nhiều đơn cùng ngày, kết quả tích lũy còn phụ thuộc cách viết frame ROWS vs RANGE (đã giải thích ở phần phân tích mệnh đề bên trên).

50

Báo cáo tổng hợp: nhiều loại lỗi trong một câu UNION ALL

TÌNH HUỐNG

Câu cuối gộp nhiều kiểm tra vào **một báo cáo duy nhất**. Mỗi loại lỗi là một câu SELECT riêng — email trùng (Câu 3), tồn kho âm (Câu 14), khách hàng ma (Câu 7) — rồi **UNION ALL** xếp chồng ba kết quả đó lại thành một bảng. QA dùng câu này như một 'health check' nhanh: chạy một lần, thấy ngay hệ thống đang có bao nhiêu loại lỗi tồn đọng.

Bảng Orders (1 trong 3 nguồn lỗi minh họa) — câu lệnh quét cả Customers, Products và Orders:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24
ORD_005	C001	15.000.000	CANCELLED	2026-06-25

CÂU LỆNH SQL

```
SELECT 'Email trùng' AS loại_loi,
       customer_id    AS doi_tuong,
       'Customers'    AS bang
FROM   Customers
WHERE  email IN (
  SELECT email FROM Customers
  WHERE email IS NOT NULL
  AND   email != ''
  GROUP BY email
  HAVING COUNT(*) > 1)
UNION ALL
SELECT 'Tồn kho âm', product_id, 'Products'
FROM   Products WHERE stock < 0
UNION ALL
SELECT 'Khách hàng ma', order_id, 'Orders'
FROM   Orders
WHERE  customer_id NOT IN
      (SELECT customer_id FROM Customers)
ORDER BY loại_loi;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers WHERE email IN (SELECT email FROM Customers GROUP BY email HAVING COUNT(*) > 1)	Khối 1 — quét & lọc: subquery bên trong tìm các email xuất hiện > 1 lần; câu ngoài giữ lại những khách có email nằm trong danh sách trùng đó (kỹ thuật Câu 3; bọc LOWER() như Câu 8 nếu muốn an toàn với mọi collation).
SELECT 'Email trung' AS loai_loi, customer_id AS doi_tuong, 'Customers' AS bang	Khối 1 — chiếu kết quả: mỗi dòng gán một nhãn cố định 'Email trung' + đối tượng (customer_id) + tên bảng, để trong báo cáo gộp biết dòng này thuộc loại lỗi gì.
UNION ALL SELECT 'Ton kho am', ... UNION ALL SELECT 'Khach hang ma', ...	UNION ALL xếp chồng thêm hai khối kiểm tra nữa (tồn kho âm, khách hàng ma) xuống dưới khối 1. Điều kiện: cả ba khối cùng số cột và kiểu tương thích. Dùng UNION ALL (không phải UNION) để giữ nguyên mọi dòng — nhanh hơn và không vô tình gộp mất dòng.
ORDER BY loai_loi	Sắp xếp theo loại lỗi — gom các lỗi cùng loại lại gần nhau để dễ đọc báo cáo.

Giải thích tổng thể

Sức mạnh của câu này là **gộp nhiều kiểm tra độc lập vào một lần chạy**: mỗi khối UNION ALL là một câu kiểm tra hoàn chỉnh, không phụ thuộc khối khác — thêm hay bớt một loại lỗi chỉ là thêm/bớt một khối, rất dễ biến thành 'bảng theo dõi sức khỏe dữ liệu' chạy định kỳ.

Một điểm đáng chú ý ở kết quả: nhóm email bắt **4 dòng (C001, C004, C005, C009)** thay vì 2 như nhìn bằng mắt. Vì MySQL 8.0+ mặc định so sánh chuỗi **không phân biệt hoa/thường**, 'a.nguyen@email.com' (C001) và 'A. NGUYEN@EMAIL.COM' (C009) bị coi là cùng một email → cả hai bị bắt. Đây là bug ẩn mà Câu 8 cũng phát hiện — cần xác nhận policy: hệ thống có phân biệt hoa/thường trong email không?

Kết quả sau khi query (minh họa)

loai_loi	doi_tuong	bang
Email trung	C001	Customers
Email trung	C004	Customers
Email trung	C005	Customers
Email trung	C009	Customers
Khach hang ma	ORD_004	Orders
Ton kho am	PROD_003	Products

6 lỗi từ 3 loại: 4 email trùng (C001+C009 vì MySQL case-insensitive), 1 đơn mờ côi (ORD_004), 1 tồn kho âm (PROD_003). Chạy câu này mỗi ngày để theo dõi dữ liệu sạch hay không.

GÓC SOI LỖI CỦA TESTER

Bẫy 'sạch giả': nếu một khối kiểm tra bị viết sai điều kiện, nó lặng lẽ trả 0 dòng — cả báo cáo trông 'không có lỗi' trong khi thực ra khối đó đã hỏng, không còn bắt được gì. Nên chạy thử từng khối riêng ít nhất một lần để chắc nó thật sự phát hiện được lỗi.

Khi thêm khối dùng **NOT IN (subquery)** như 'Khach hang ma': nếu cột trong subquery có thể NULL, đổi sang **NOT EXISTS** — cùng bẫy NULL đã học ở Câu 11, nay ở dạng subquery.

BÀI TẬP TỰ LUYỆN — PHẦN 6

Bài 6.1. Dùng RANK() xếp hạng sản phẩm theo tổng doanh số bán ra (tính từ Order_Items).

Gợi ý: `RANK() OVER (ORDER BY SUM(quantity*price) DESC) kết hợp GROUP BY product_id.`

Bài 6.2. Dùng UNION ALL tạo báo cáo đếm nhanh: bao nhiêu khách lỗi email, bao nhiêu sản phẩm giá NULL, bao nhiêu đơn mờ côi.

Gợi ý: Mỗi dòng là một `SELECT COUNT(*)` kèm nhãn chuỗi cố định, nối bằng `UNION ALL`.

→ Lời giải đầy đủ ở **Phụ lục B** (cuối sách). Hãy tự viết trước khi xem.

Case study: Một ngày điều tra dữ liệu của QA

BỐI CẢNH

Sáng thứ Hai, dashboard báo doanh số iPhone 15 Pro Max (PROD_001) tháng này là **90.000.000đ** — tương đương 3 máy. Nhưng bộ phận kho khẳng định chỉ xuất **2 máy** (60.000.000đ). Lẽch 30 triệu. QA được nhờ soi xem dữ liệu sai ở đâu.

Thay vì đoán, QA lần theo dấu vết bằng chính các câu lệnh trong sách — mỗi bước thu hẹp phạm vi nghi ngờ cho tới khi chạm nguyên nhân gốc.

Bước 1 · Chụp toàn cảnh (Câu 21)

Đếm số dòng items mỗi đơn để tìm đơn có số dòng bất thường.

Câu lệnh lỗi: `GROUP BY order_id → COUNT(*)`

Kết quả: **ORD_001 = 3 dòng** (các đơn khác chỉ 1–2) — một đơn 2 sản phẩm sao có 3 dòng?

→ **Bước tiếp:** Biết có đơn bất thường, nhưng chưa rõ sản phẩm nào bị thổi số — xếp hạng doanh số để lộ nghi phạm.

Bước 2 · Định vị nghi phạm (Câu 22)

Xếp hạng doanh số từng sản phẩm để xem cái nào cao bất thường.

Câu lệnh lỗi: `GROUP BY product_id → SUM(quantity*price), ORDER BY ... DESC`

Kết quả: **PROD_001 = 90M** = đúng 3×30 triệu, nhưng nghiệp vụ chỉ bán 2 lần → có 1 lần bán 'ảo'.

→ **Bước tiếp:** Biết PROD_001 bị thổi, nhưng cái 'ảo' nằm ở dòng nào? — đánh số từng dòng để chỉ mặt bản trùng.

Bước 3 · Truy tới gốc (Câu 46)

Đánh số các dòng trong cùng nhóm (order_id, product_id): dòng nào bị đánh số > 1 là bản trùng.

Câu lệnh lỗi: `ROW_NUMBER() OVER (PARTITION BY order_id, product_id)`

Kết quả: Trong ORD_001/PROD_001: item 1 → 1, **item 7 → 2** — item 7 là bản sao của item 1. Đây chính là chiếc iPhone thứ ba 'ảo'.

→ **Bước tiếp:** Đã chỉ mặt thủ phạm (item 7), giờ đo thiệt hại bằng tiền — đối soát tổng đơn.

Bước 4 · Đo đúng phần lệch (Câu 13)

Đối soát tổng ghi trên đơn với tổng cộng từ từng dòng items.

Câu lệnh lỗi: `HAVING total_amount <> SUM(quantity*price)`

Kết quả: **ORD_001: ghi 32M nhưng items cộng 62M — lệch đúng 30 triệu**, khớp giá một iPhone đếm thừa.

→ **Bước tiếp:** Biết một đơn lệch 30M, nhưng con số sai này đã lan tới báo cáo nào? — tổng theo danh mục xem phạm vi.

Bước 5 · Đánh giá phạm vi ảnh hưởng (Câu 27)

Tổng doanh số theo danh mục để xem lỗi lan tới cấp báo cáo nào.

Câu lệnh lỗi: `GROUP BY category → SUM(quantity*price)`

Kết quả: **Diện thoai = 90M** thay vì 60M thực — báo cáo danh mục và toàn hệ thống đều bị thổi từ 1 dòng lỗi.

→ **Bước tiếp:** Xong vụ chính; tiện lúc mở DB, quét luôn xem còn lỗi nào khác đang ẩn.

Bước 6 · Quét thêm lỗi khác (Câu 50)

Việc chính đã xong. Trước khi đóng DB, chạy một câu gộp nhiều kiểm tra vào một lần — thói quen tốt để không bỏ lỡ lỗi khác đang ẩn.

Câu lệnh lỗi: `SELECT 'Email trùng'... UNION ALL SELECT 'Tồn kho âm'... UNION ALL SELECT 'Khách mờ côi'...`

Kết quả: **6 dòng thuộc 3 loại lỗi:** 4 email trùng · 1 sản phẩm tồn kho âm · 1 đơn mờ côi (ORD_004 trở khách C999 không tồn tại) — không liên quan vụ iPhone, nhưng ghi nhận để xử lý riêng.

KẾT LUẬN & KIẾN NGHỊ

Nguyên nhân gốc: bảng Order_Items thiếu ràng buộc duy nhất trên (order_id, product_id), khiến cùng một dòng bị ghi hai lần.

- Báo dev thêm **UNIQUE(order_id, product_id)** (hoặc kiểm tra ở tầng ứng dụng) để chặn lặp.
- Dọn dữ liệu: xóa item_id = 7 bị trùng, tính lại total_amount và các báo cáo liên quan.
- Báo kế toán con số đúng: doanh số iPhone là **60 triệu**, không phải 90 triệu.

Điều đáng nhớ không phải sáu câu lệnh, mà là **trình tự điều tra**: chụp toàn cảnh → định vị nghi phạm → truy tới gốc → đo đúng phần lệch → đánh giá phạm vi → quét lỗi đi kèm. SQL là đèn pin; tư duy điều tra mới là hướng soi.

Kiểm thử ghi dữ liệu: hành động trên ứng dụng → xác minh database

50 câu phía trước soi **dữ liệu đã có sẵn**. Nhưng phần lớn công việc kiểm thử chạm tới database lại là một câu hỏi khác: “**Tôi vừa thao tác trên ứng dụng — database có ghi lại đúng không?**” Tạo một đơn hàng, sửa hồ sơ, hủy giao dịch — mỗi hành động phải để lại đúng dấu vết trong DB. Đây là kỹ năng QA chức năng dùng hằng ngày.

Khuôn ba bước: Trước → Hành động → Sau (Arrange – Act – Assert)

TRƯỚC (Arrange) — ghi lại trạng thái hiện tại

Trước khi thao tác, ghi lại những giá trị sắp bị thay đổi — nhất là các con số sẽ biến động (tồn kho, số dư, số dòng). Không có mốc 'trước' để so, bạn không thể khẳng định 'sau' đúng hay sai.

HÀNH ĐỘNG (Act) — thao tác qua app/API

Thực hiện đúng một thao tác trên giao diện hoặc API như người dùng thật — **không** sửa DB bằng tay. Mục tiêu là kiểm xem hệ thống ghi đúng không, chứ không phải tự ghi hộ nó.

SAU (Assert) — soi DB và so với kỳ vọng

Chạy SELECT kiểm từng thứ và so với kỳ vọng: bản ghi chính, các bảng liên quan, các giá trị được tính ra từ số khác (vd tổng tiền = tổng các dòng), và quan trọng — **không có bản ghi thừa hay thiếu**.

Ví dụ — đặt một đơn hàng mới rồi xác minh

HÀNH ĐỘNG

Trên giao diện, khách **C001** đặt mua **2** chiếc iPhone 15 Pro Max (PROD_001, giá 30.000.000). Hệ thống sinh đơn mới **ORD_100**. Tồn kho PROD_001 trước khi đặt là **50**. Sau thao tác, DB phải thể hiện đúng bốn điều dưới đây.

1. Bản ghi đơn được tạo đúng — đúng khách, tổng tiền, trạng thái, ngày.

```
SELECT order_id, customer_id, total_amount, status, order_date
FROM   Orders
WHERE  order_id = 'ORD_100';
```

Kỳ vọng: customer_id = C001, total_amount = 60.000.000, status hợp lệ (vd PENDING), order_date = hôm nay.

2. Dòng chi tiết đúng và KHÔNG thừa — số lượng dòng phải khớp.

```
SELECT product_id, quantity, price
FROM   Order_Items
WHERE  order_id = 'ORD_100';
```

Kỳ vọng đúng 1 dòng: PROD_001, quantity = 2, price = 30.000.000 (giá tại thời điểm mua). Nếu ra 2 dòng → double-submit (người dùng bấm gửi hai lần), xem Câu 4, 29; nếu rỗng → đơn tạo mà thiếu chi tiết (Câu 16, 38).

3. Tồn kho bị trừ đúng số lượng — so với mốc 'trước'.

```
SELECT stock
FROM Products
WHERE product_id = 'PROD_001';
```

Trước khi đặt stock = 50, bán 2 → kỳ vọng 48. Vẫn 50 → quên trừ kho; còn 46 → trừ hai lần (xem Câu 20, 28).

4. Số tổng khớp với chi tiết: $\text{total_amount} = \text{SUM}(\text{quantity} \times \text{price})$.

```
SELECT o.total_amount,
       SUM(oi.quantity * oi.price) AS tinh_tu_items
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
WHERE o.order_id = 'ORD_100'
GROUP BY o.total_amount;
```

Hai con số phải bằng nhau ($60.000.000 = 2 \times 30.000.000$). Lỗi là lỗi tính tiền — đúng mẫu Câu 13, nay áp cho một đơn vừa tạo.

CHECKLIST — SAU MỖI THAO TÁC GHI, KIỂM GÌ?

- Bản ghi chính được tạo / sửa / xóa đúng như mong đợi?
- Mọi bảng **liên quan** cập nhật đúng (tồn kho, audit log, bảng đối soát)?
- Số dòng đúng — không thừa (double-submit), không thiếu?
- Số tổng khớp với chi tiết ($\text{total} = \text{SUM items}$, số dư, điểm thưởng)?
- Cột tự động đúng (timestamp, status mặc định, khóa ngoại)?
- Nếu thao tác **thất bại**: DB có rollback sạch, không để lại bản ghi rác?

GÓC SOI LỖI CỦA TESTER

Những bug 'ghi dữ liệu' hay gặp nhất:

- App báo 'thành công' nhưng DB không ghi — hoặc ngược lại, DB ghi mà app báo lỗi (để lại rác).
- Cập nhật **một phần**: đơn được tạo nhưng quên trừ kho hoặc quên ghi audit log.
- Trừ kho hai lần / tạo đơn trùng do double-submit (Câu 4, 29).
- **Soft-delete**: 'xóa' trên app nhưng DB chỉ đánh dấu cờ — phải kiểm đúng cơ chế, đừng tưởng mất là mất.
- Sai múi giờ hoặc timestamp khi ghi (Câu 34).

Luôn kiểm trên môi trường test/staging, và chạy lại file **ecommerce_test_setup.sql** để đưa dữ liệu về mốc gốc trước mỗi lượt kiểm — nhờ vậy con số 'trước' luôn xác định và kết quả tái lập được.

Chạy SQL an toàn trên production

50 câu lệnh chính trong sách đều là **SELECT** — chỉ đọc, không sửa. Nhưng khi chạy trên database thật đang phục vụ người dùng, cần vài nguyên tắc tối thiểu để không gây hại.

Nguyên tắc an toàn

- Ưu tiên chạy trên **replica / bản sao chỉ đọc**; dùng **tài khoản read-only** để không thể vô tình UPDATE/DELETE.
- Luôn thêm **LIMIT** khi thăm dò; tránh giờ cao điểm — câu nặng có thể chiếm nhiều tài nguyên và làm chậm cả hệ thống.
- Đặt **EXPLAIN** trước câu **SELECT** để xem có quét cả bảng không — dấu hiệu **'Table scan'** (EXPLAIN dạng cây, mặc định MySQL 8.0+) hoặc **type = ALL** (dạng bảng). Trên bảng lớn, đây là tín hiệu báo dev/DBA cân nhắc đánh index trên cột đang lọc — QA không tự thêm index trên production.
- Nếu buộc phải sửa: chạy **SELECT** với đúng **WHERE** trước, xem kết quả, rồi mới đổi thành UPDATE/DELETE trong transaction để còn ROLLBACK nếu sai.
- Sách viết theo cú pháp MySQL — vài chỗ khác trên PostgreSQL/SQL Server (LIMIT vs TOP, IFNULL vs ISNULL, phân biệt hoa/thường).

QUYỀN RIÊNG TƯ KHI QUERY DỮ LIỆU THẬT

- **KHÔNG SELECT *** trên bảng chứa thông tin cá nhân; che email/số điện thoại khi đính kết quả vào defect report.
- Không copy dữ liệu thật ra ngoài môi trường kiểm soát; cần quyền truy cập thì xin đúng quy trình, đừng mượn tài khoản người khác.

PHỤ LỤC — Tra cứu nhanh

Ba phần tra cứu nhanh: bảng cú pháp lỗi để tra khi viết câu lệnh, đáp án các bài tập tự luyện, và bảng giải thích thuật ngữ.

A · Cheat sheet cú pháp lỗi

Mục tiêu	Cú pháp lỗi	Câu mẫu
Khám phá schema chưa có tài liệu	FROM information_schema .columns / .table_constraints	1, 2
Tìm bản ghi trùng	GROUP BY cot HAVING COUNT(*) > 1	3, 4, 8, 29
Tìm mò côi — an toàn với NULL	WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.fk = a.id)	BT 6.2
Tìm mò côi — viết ngắn	LEFT JOIN b ON ... WHERE b.id IS NULL	7, 16, 17
Bẫy cần tránh	NOT IN (subquery có NULL) → luôn rỗng	11
Xử lý NULL khi tính toán	COALESCE(x, 0)	28
Chuẩn hóa chuỗi trước khi so	TRIM(x), LOWER(x)	6, 8, 35
Kiểm tra định dạng chuỗi	x REGEXP 'mau'	31, 32
Kiểm tra ngày bất thường	WHERE ngay > CURDATE() DATEDIFF(a, b)	34, 42, 43
Đếm có điều kiện	SUM(CASE WHEN ... THEN 1 ELSE 0 END)	9
Đối soát tổng cha – con	JOIN + GROUP BY HAVING SUM(chi tiết) ≠ cha	13, 21, 39
Tính phần trăm / tỷ lệ	COUNT(*) * 100.0 / (SELECT COUNT(*) ...)	26
Đánh số thứ tự trong nhóm	ROW_NUMBER() OVER (PARTITION BY g ORDER BY c)	46
Xếp hạng (cho phép đồng hạng)	RANK() / DENSE_RANK() OVER (ORDER BY c)	22, 47
Cộng dồn theo thời gian	SUM(x) OVER (ORDER BY ngay)	49
Tái dùng kết quả trung gian	WITH cte AS (...) SELECT ... FROM cte	47, 48
Ghép nhiều báo cáo theo cột dọc	SELECT ... UNION ALL SELECT ...	50, BT 6.2

Mục tiêu	Cú pháp lỗi	Câu mẫu
Ngưỡng động tính từ chính bảng	WHERE x > (SELECT AVG(x)*k FROM t)	30, 39, 48

B · Đáp án bài tập tự luyện

Lời giải dưới đây chỉ là MỘT cách viết đúng — cách của bạn có thể khác mà vẫn cho cùng kết quả. Đáp án được kiểm chứng trên bộ dữ liệu mẫu nhỏ trong sách.

PHẦN 1 · Toàn vẹn và trùng lặp dữ liệu

Bài 1.1. Đếm số khách theo từng hạng thành viên (membership_tier) và chỉ ra hạng nào nằm NGOÀI danh sách hợp lệ (Standard, Silver, Gold, Platinum).

```
SELECT membership_tier,
       COUNT(*) AS so_khach,
       CASE WHEN membership_tier NOT IN
            ('Standard', 'Silver', 'Gold', 'Platinum')
            THEN 'NGOAI DANH SACH' ELSE 'OK' END AS trang_thai
FROM   Customers
GROUP BY membership_tier
ORDER BY so_khach DESC;
```

ĐÁP ÁN

Standard (5), Silver (2), Gold (2), VIP (1). Chỉ VIP bị đánh dấu 'NGOAI DANH SACH' — đây mới là giá trị cần điều tra (Bug-H). Đếm theo tier là bình thường; điều đáng quan tâm với QA là tier lạ, không phải tier trùng.

Bài 1.2. Tìm những khách hàng có customer_name chứa khoảng trắng thừa ở đầu hoặc cuối.

```
SELECT customer_id,
       customer_name,
       CHAR_LENGTH(customer_name) AS do_dai
FROM   Customers
WHERE  customer_name <> TRIM(customer_name);
```

ĐÁP ÁN

C008 (' Pham Van D ', dài 14 ký tự — sau TRIM còn 10). Khoảng trắng thừa không nhìn thấy bằng mắt nhưng phá vỡ so khớp, gây trùng ẩn và lỗi tìm kiếm.

Bài 1.3. Sản phẩm nào được mua trong nhiều hơn một đơn hàng khác nhau?

```
SELECT product_id,
       COUNT(DISTINCT order_id) AS so_don
FROM   Order_Items
GROUP BY product_id
HAVING COUNT(DISTINCT order_id) > 1;
```

ĐÁP ÁN

4 sản phẩm đạt điều kiện: PROD_001 (2 đơn: ORD_001+ORD_002), PROD_002 (3 đơn: ORD_001+ORD_003+ORD_005), PROD_003 (2 đơn: ORD_003+ORD_006), PROD_004 (3 đơn: ORD_002+ORD_005+ORD_007). Lưu ý COUNT(DISTINCT order_id) khác COUNT(*): PROD_001 có 3 dòng item nhưng item 1 và 7 cùng thuộc ORD_001 — chỉ tính 1 đơn cho lần đó.

PHẦN 2 · Ràng buộc nghiệp vụ

Bài 2.1. Liệt kê các sản phẩm có giá cao hơn giá trung bình của toàn bộ sản phẩm.

```
SELECT product_id,
       product_name,
       price
FROM   Products
WHERE  price > (SELECT AVG(price) FROM Products)
ORDER BY price DESC;
```

ĐÁP ÁN

PROD_001 (30M) và PROD_003 (8M). Trung bình $\approx 7.4M$. Lưu ý: AVG tự bỏ qua PROD_006 (giá NULL) nên ngưỡng chỉ tính trên 6 sản phẩm có giá.

Bài 2.2. Trong các đơn COMPLETED, đơn nào có total_amount KHÁC tổng tiền tính từ Order_Items?

```
SELECT o.order_id,
       o.total_amount,
       SUM(oi.quantity * oi.price) AS tong_items
FROM   Orders o
JOIN   Order_Items oi ON o.order_id = oi.order_id
WHERE  o.status = 'COMPLETED'
GROUP BY o.order_id, o.total_amount
HAVING SUM(oi.quantity * oi.price) <> o.total_amount;
```

ĐÁP ÁN

ORD_001 (32M vs 62M — item trùng) và ORD_002 (20M vs 31M — Bug-B). ORD_003 bị loại vì CANCELLED. Đây là Câu 13 thu hẹp vào đơn đã hoàn tất — nơi sai số gây hậu quả tài chính thật.

Bài 2.3. Sản phẩm nào CHƯA từng được bán nhưng vẫn còn tồn kho lớn hơn 0?

```
SELECT p.product_id,
       p.product_name,
       p.stock
FROM   Products p
WHERE  NOT EXISTS (
    SELECT 1 FROM Order_Items oi
    WHERE  oi.product_id = p.product_id)
AND    p.stock > 0;
```

ĐÁP ÁN

PROD_005 (30) và PROD_006 (10). PROD_007 bị loại vì stock = NULL — so sánh 'NULL > 0' cho UNKNOWN, không phải TRUE. Đây là lý do hàng tồn 'tàng hình' dễ bị bỏ sót.

Bài 2.4. Tìm tất cả sản phẩm có giá (price) bằng NULL hoặc bằng 0.

```
SELECT product_id,
       product_name,
       price
FROM   Products
WHERE  price IS NULL
      OR price <= 0;
```

ĐÁP ÁN

Chỉ PROD_006 (Loa Bluetooth JBL, price = NULL) khớp; trong data mẫu không sản phẩm nào có price ≤ 0. Tương tự Câu 14 nhưng áp cho cột price thay vì stock: NULL = chưa nhập giá; ≤ 0 = giá âm hoặc miễn phí ngoài ý muốn.

PHẦN 3 · Đối soát và tính toán

Bài 3.1. Tính tổng doanh thu thực (số lượng × giá) của từng đơn hàng, sắp xếp giảm dần.

```
SELECT o.order_id,
       SUM(oi.quantity * oi.price) AS doanh_thu
FROM   Orders o
JOIN   Order_Items oi ON o.order_id = oi.order_id
GROUP BY o.order_id
ORDER BY doanh_thu DESC;
```

ĐÁP ÁN

6 đơn có item (ORD_004 rỗng nên không hiện): ORD_001=62M · ORD_002=31M · ORD_007=20M · ORD_003=8M · ORD_006=8M · ORD_005=3M. ORD_001 đứng đầu nhưng bị thổi phồng do item trùng (62M vs total_amount ghi 32M) — luôn đối soát hai con số này (Câu 13).

Bài 3.2. Mỗi danh mục (category) có bao nhiêu sản phẩm, kể cả sản phẩm chưa bán?

```
SELECT category,
       COUNT(*) AS so_sp
FROM   Products
GROUP BY category
ORDER BY so_sp DESC;
```

ĐÁP ÁN

Phu kien (6) · Dien thoai (1). Vì đếm trên Products nên bao gồm cả sản phẩm chưa bán — khác Câu 27 (chỉ đếm danh mục có phát sinh đơn qua JOIN).

Bài 3.3. Khách hàng nào có tổng chi tiêu cao hơn mức trung bình của các khách đã từng mua?

```
SELECT c.customer_id,
       c.customer_name,
       SUM(o.total_amount) AS tong
FROM   Customers c
JOIN   Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(o.total_amount) > (
    SELECT AVG(t) FROM (
        SELECT SUM(o2.total_amount) AS t
        FROM   Orders o2
        JOIN   Customers c2 ON o2.customer_id = c2.customer_id
        GROUP BY o2.customer_id) x);
```

ĐÁP ÁN

C001 (67M). Trung bình ba khách có đơn $\approx 34.3M$ $((67+20+16)/3)$; chỉ C001 vượt. C001 có 3 đơn (ORD_001+ORD_005+ORD_007=67M), C003 có 2 đơn (ORD_003+ORD_006=16M). Mẫu 'so với trung bình của chính tập' là nền của phân tích outlier — Câu 48 dùng CTE để viết gọn hơn (lưu ý: Câu 48 chỉ tính đơn COMPLETED nên số khác bài này).

PHẦN 4 · Biên và dữ liệu bất thường**Bài 4.1.** Tìm các sản phẩm bị thiếu dữ liệu giá HOẶC tồn kho (giá trị NULL).

```
SELECT product_id,
       product_name,
       price,
       stock
FROM   Products
WHERE  price IS NULL
       OR stock IS NULL;
```

ĐÁP ÁN

PROD_006 (price NULL) và PROD_007 (stock NULL). Phải dùng IS NULL — viết 'price = NULL' luôn cho kết quả rỗng. Hai sản phẩm này phá vỡ mọi phép tính liên quan đến giá/kho.

Bài 4.2. Tìm khách hàng có tên chứa chữ số.

```
SELECT customer_id,
       customer_name
FROM   Customers
WHERE  customer_name REGEXP '[0-9]';
```

ĐÁP ÁN

C009 ('Nguyen Van A (2)'). Tên người thật hiếm khi có chữ số; '(2)' là dấu hiệu tài khoản nhân bản hoặc dữ liệu test được đánh số thủ công (Câu 31).

Bài 4.3. Tìm các tên sản phẩm bị trùng sau khi chuẩn hóa (bỏ khoảng trắng + đưa về chữ thường).

```
SELECT LOWER(TRIM(product_name)) AS ten_chuan,
       COUNT(*) AS so_lan
FROM   Products
GROUP BY LOWER(TRIM(product_name))
HAVING COUNT(*) > 1;
```

ĐÁP ÁN

'ban phim co logitech' xuất hiện 2 lần (PROD_002 + PROD_005). Chuẩn hóa trước khi GROUP giúp bắt trùng mà so khớp thô bỏ sót (Câu 35).

PHẦN 5 · Audit, log và dấu vết

Bài 5.1. Liệt kê các đơn đã bị xóa mềm (deleted_at có giá trị) kèm số ngày từ lúc đặt đến lúc xóa (DATEDIFF) — đơn 'sống' quá ngắn là dấu vết đáng soi.

```
SELECT order_id,
       order_date,
       deleted_at,
       DATEDIFF(deleted_at, order_date) AS ngay_ton_tai
FROM   Orders
WHERE  deleted_at IS NOT NULL;
```

ĐÁP ÁN

ORD_005 — đặt và xóa cùng ngày 2026-06-25 → ngay_ton_tai = 0. Đơn bị hủy ngay trong ngày thường là đặt nhầm, test, hoặc thao tác gian lận — dấu vết audit cần đối chiếu với log thao tác.

Bài 5.2. Tìm các item vẫn còn trong Order_Items nhưng thuộc đơn hàng đã bị xóa mềm (deleted_at IS NOT NULL) — dấu vết dọn dẹp không triệt để.

```
SELECT oi.item_id,
       oi.order_id,
       oi.product_id
FROM   Order_Items oi
JOIN   Orders o ON oi.order_id = o.order_id
WHERE  o.deleted_at IS NOT NULL;
```

ĐÁP ÁN

Item 8 (PROD_004) và item 9 (PROD_002) — cả hai thuộc ORD_005 đã xóa mềm nhưng item chưa được dọn. Nếu báo cáo doanh số tính thẳng từ Order_Items mà không JOIN sang Orders để lọc deleted_at, hai item này vẫn bị cộng vào — đúng mẫu rò rỉ ở Câu 40.

PHẦN 6 · Truy vấn nâng cao cho QA

Bài 6.1. Dùng RANK() xếp hạng sản phẩm theo tổng doanh số bán ra (tính từ Order_Items).

```
SELECT product_id,
       SUM(quantity * price) AS doanh_so,
       RANK() OVER (
         ORDER BY SUM(quantity * price) DESC) AS hang
FROM   Order_Items
GROUP BY product_id;
```

ĐÁP ÁN

PROD_001 90M (hạng 1) · PROD_004 22M (hạng 2) · PROD_003 16M (hạng 3) · PROD_002 4M (hạng 4).
Xếp hạng đúng kỹ thuật nhưng 90M của PROD_001 bị thổi phồng do item trùng — dữ liệu nền vẫn bẩn (Câu 47). PROD_002 chỉ 4M vì item 12 có quantity=0 (không đóng góp doanh số).

Bài 6.2. Dùng UNION ALL tạo báo cáo đếm nhanh: bao nhiêu khách lỗi email, bao nhiêu sản phẩm giá NULL, bao nhiêu đơn mờ côi.

```
SELECT 'Email NULL/rong' AS loai, COUNT(*) AS so_luong
FROM   Customers WHERE email IS NULL OR TRIM(email) = ''
UNION ALL
SELECT 'Gia NULL', COUNT(*)
FROM   Products WHERE price IS NULL
UNION ALL
SELECT 'Don mo coi', COUNT(*)
FROM   Orders o WHERE NOT EXISTS (
  SELECT 1 FROM Customers c
  WHERE c.customer_id = o.customer_id);
```

ĐÁP ÁN

Email NULL/rỗng = 2 (C006, C007) · Giá NULL = 1 (PROD_006) · Đơn mờ côi = 1 (ORD_004). Mẫu health-check: một câu trả ra nhiều loại lỗi (Câu 50), dễ mở rộng bằng cách thêm UNION ALL.

C · Giải thích thuật ngữ

Các thuật ngữ kỹ thuật thường gặp trong sách — sắp xếp theo thứ tự bảng chữ cái.

Thuật ngữ	Giải thích
AUTO_INCREMENT	Cột DB tự tăng số thứ tự mỗi khi thêm dòng mới (1, 2, 3...). Nếu một INSERT thất bại hoặc có dòng bị xóa, chuỗi số sẽ có gap — đây là dấu hiệu cần điều tra (Câu 44).
Bản ghi mồ côi (orphan record)	Dòng dữ liệu tham chiếu đến một đối tượng không tồn tại ở bảng kia. Ví dụ: đơn hàng có customer_id C999 nhưng C999 không có trong bảng Customers (Câu 7).
Collation	Cài đặt quy định cách database so sánh chuỗi ký tự. Chế độ mặc định MySQL 8.0 (utf8mb4_0900_ai_ci) không phân biệt hoa/thường: 'a.nguyen@email.com' và 'A.NGUYEN@EMAIL.COM' được coi là như nhau (Câu 3, 8).
Composite key (khóa tổ hợp)	Khóa gồm nhiều cột kết hợp thay vì một cột đơn. Ví dụ: tổ hợp (order_id, product_id) xác định duy nhất mỗi dòng trong Order_Items (Câu 4).
CTE (Common Table Expression)	Tên đặt tạm cho một kết quả trung gian trong câu SQL, khai báo bằng WITH ... AS (...). Giúp viết SQL theo từng bước rõ ràng thay vì lồng nhiều subquery vào nhau (Câu 47, 48).
FK / Foreign Key (khóa ngoại)	Cột dùng để liên kết sang bảng khác. Khi có FK constraint, DB kiểm tra tự động: giá trị nhập vào phải tồn tại ở bảng tham chiếu. Nếu constraint bị tắt, bản ghi mồ côi có thể lọt vào (Câu 2, 7).
Hard delete (xóa cứng)	Xóa hẳn dòng khỏi DB bằng lệnh DELETE. Không thể phục hồi nếu không có backup. Xem thêm: Soft-delete.
Index (chỉ mục)	Cấu trúc DB tạo sẵn để tăng tốc tìm kiếm, giống mục lục cuối sách. Câu lệnh bọc hàm quanh cột (LOWER, TRIM, DATEDIFF...) thường không dùng được index, khiến DB phải quét toàn bộ bảng — chậm khi dữ liệu lớn.
NULL	Giá trị đặc biệt nghĩa là 'chưa có thông tin' — khác hoàn toàn với chuỗi rỗng "" và số 0. Mọi phép so sánh với NULL (=, !=, >) đều trả về 'không xác định', nên bắt buộc dùng IS NULL hoặc IS NOT NULL (Câu 5, 9, 14).
Outlier (giá trị ngoại lệ)	Bản ghi có giá trị bất thường so với phần còn lại của tập dữ liệu. Ví dụ: đơn hàng 32M trong khi trung bình chỉ 15M. Outlier không nhất thiết là lỗi — nhưng luôn cần điều tra thêm (Câu 30).
Rollback (hoàn tác giao dịch)	Khi một giao dịch DB (transaction) gặp lỗi giữa chừng, rollback hủy bỏ toàn bộ thao tác đã thực hiện trong giao dịch đó và khôi phục DB về trạng thái trước. Lưu ý: AUTO_INCREMENT không rollback theo — tạo gap trong chuỗi ID.
Soft-delete (xóa mềm)	Không xóa thật mà đánh dấu bằng cột (deleted_at, is_deleted). Dữ liệu vẫn còn trong DB, dùng để tra cứu lịch sử hoặc phục hồi. Bug thường gặp: quên lọc bản ghi đã soft-delete dẫn đến tính toán sai (Câu 12, 40, 41).
Subquery (truy vấn con)	Câu SELECT lồng bên trong một câu SQL khác. Dùng để tính ngưỡng động, kiểm tra sự tồn tại, hoặc lọc theo kết quả trung gian. Khi cần dùng lại nhiều lần, thay bằng CTE để code rõ hơn (Câu 30, 47, 48).
Transaction (giao dịch DB)	Nhóm thao tác DB được thực hiện cùng nhau theo nguyên tắc 'tất cả hoặc không'. Nếu bất kỳ bước nào thất bại, toàn bộ giao dịch bị rollback. Đảm bảo dữ liệu không bị ghi dở giữa chừng.
Window function (hàm cửa sổ)	Nhóm hàm SQL tính toán trên một tập dữ liệu nhưng giữ nguyên từng dòng — khác với GROUP BY vốn gộp nhiều dòng thành một. Gồm: ROW_NUMBER(), RANK(), SUM() OVER, LAG()... (Câu 46, 47, 49).

Tóm lại

SQL không thay thế tư duy kiểm thử — nó khuếch đại tư duy ấy. Một câu lệnh chỉ mạnh khi đứng sau nó là một câu hỏi tốt: "Điều gì trong hệ thống này lẽ ra phải luôn đúng?". 50 câu lệnh ở đây là 50 câu hỏi như vậy, đã được viết thành truy vấn. Khi gặp một nghiệp vụ mới, hãy tự đặt câu hỏi của riêng bạn rồi dịch nó sang SQL theo đúng các mẫu trong sách.

Ba điều mang theo:

- Mọi truy vấn đối soát đều dựa trên một **bất biến** — tìm cái luôn đúng, rồi tìm cái vi phạm nó.
- Kết quả SQL là **danh sách cần điều tra**, không phải bản án. Luôn xác minh trước khi kết luận bug.
- Cẩn thận với **NULL**, **collation**, **khoảng trắng ẩn** và **khác biệt dialect** — đó là nơi bug ẩn nấp ngay trong chính câu lệnh của bạn.

Biên soạn bởi maiqai.com